IDAT7215
Computer Programming for Product Development and Applications

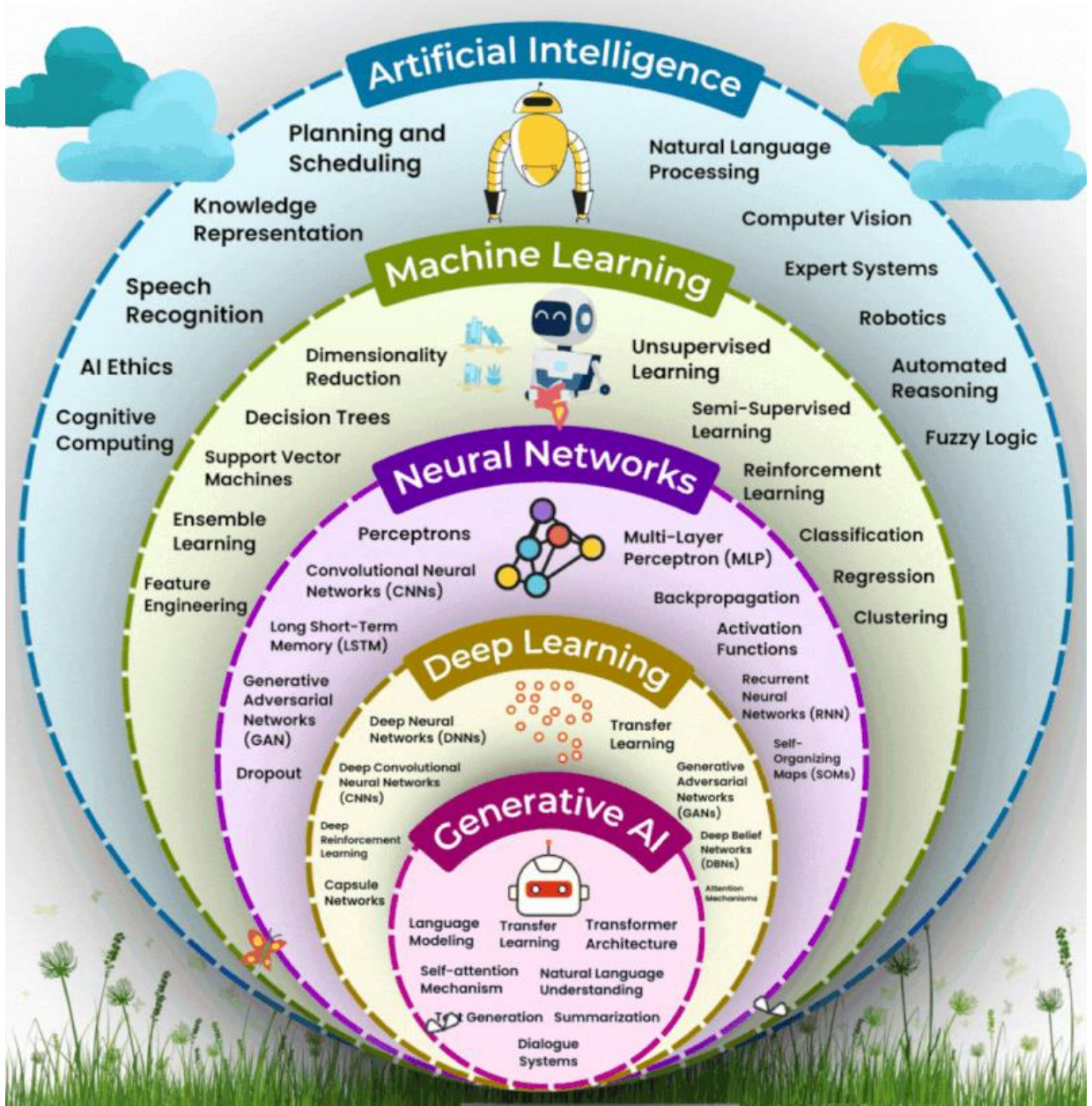
Lecture 4-1: Introduction to Machine Learning Using Scikit-learn Python Library

Dr. Zulfiqar Ali

Outline

- Why Machine Learning
- What is Machine Learning
- Machine Learning vs Deep Learning
- Training and Test Data
- Types of Machine Learning
- Introduction to Scikit-Learn Library
- Supervised Machine Learning Algorithms

The AI Universe



Artificial Intelligence



Engineering of making Intelligent Machines and Programs

1950's 1960's 1970's

Machine Learning



Ability to learn without being explicitly programmed

1980's 1990's 2000's 2006's

Deep Learning



Learning based on Deep Neural Network

2010's 2012's 2017's

Why Machine Learning was Introduced

Statistics: How to efficiently train large complex models?

Computer Science & Artificial Intelligence: How to train more robust versions of the AI systems.

Neuroscience: How to design operational models of the brain.



What is Machine Learning

Machine learning is a subset of AI techniques that use statistical methods to enable machines to learn automatically and improve from experience.





Machine Learning

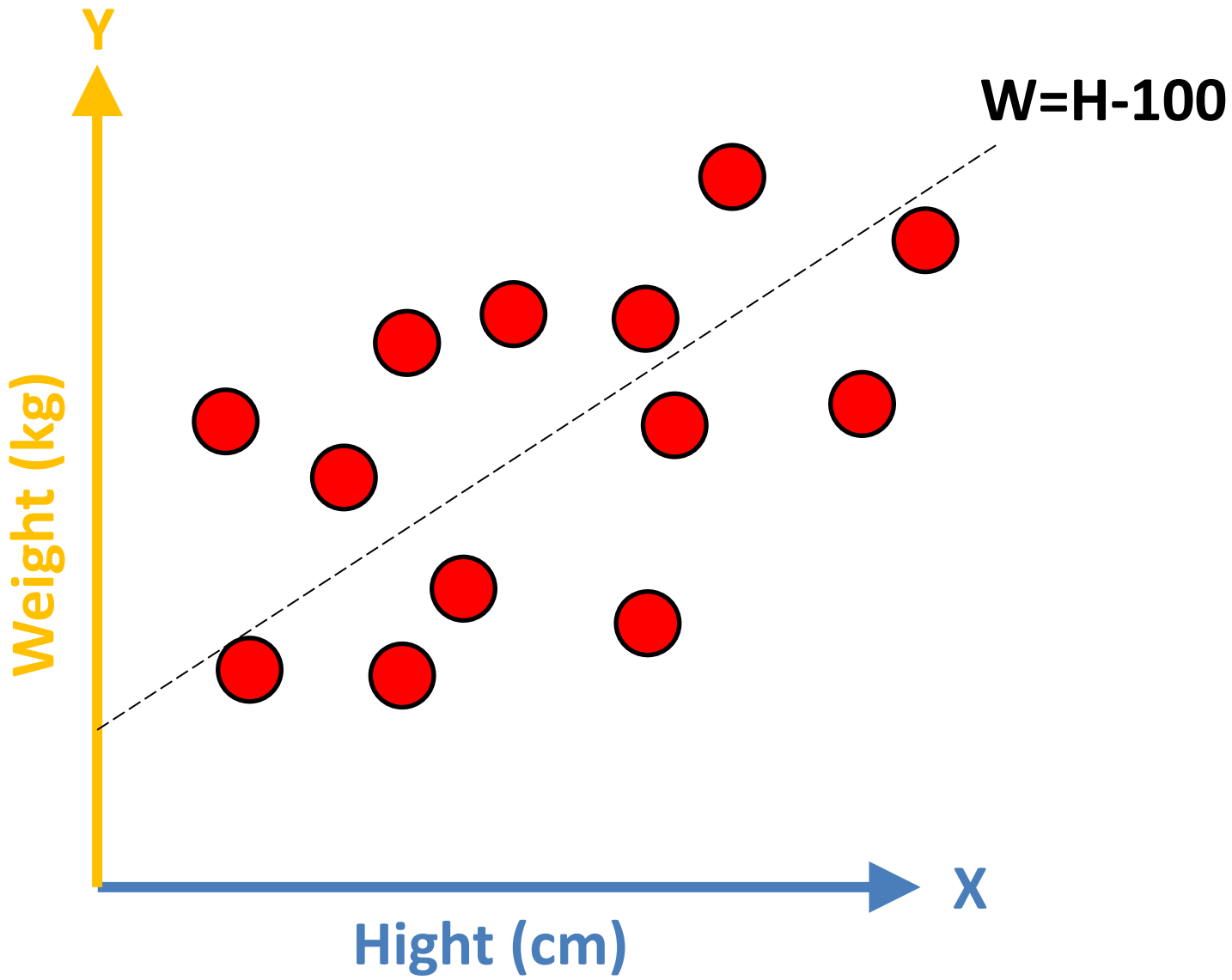
- Herbert Alexander Simon: “ Learning is any process by which a system improves performance from experience.”

- “Machine Learning is concerned with computer programs that automatically improve their performance through experience. “

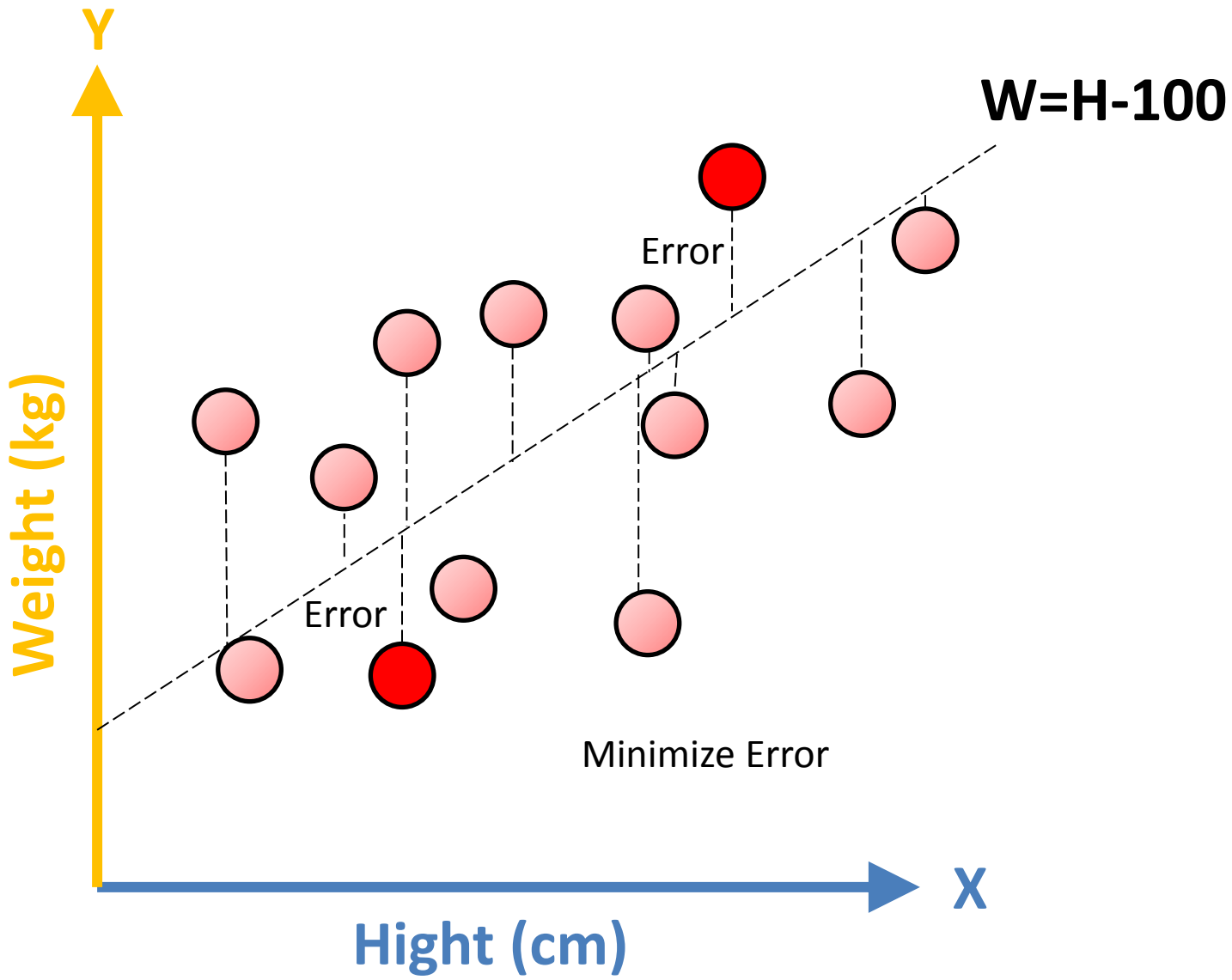


Herbert Simon
Turing Award 1975
Nobel Prize in Economics 1978

Example of Machine Learning



Example of Machine Learning



What is Deep Learning

Deep learning is a particular kind of machine learning that is inspired by the functionality of brain cells called neurons which led to the concept of artificial neural network.





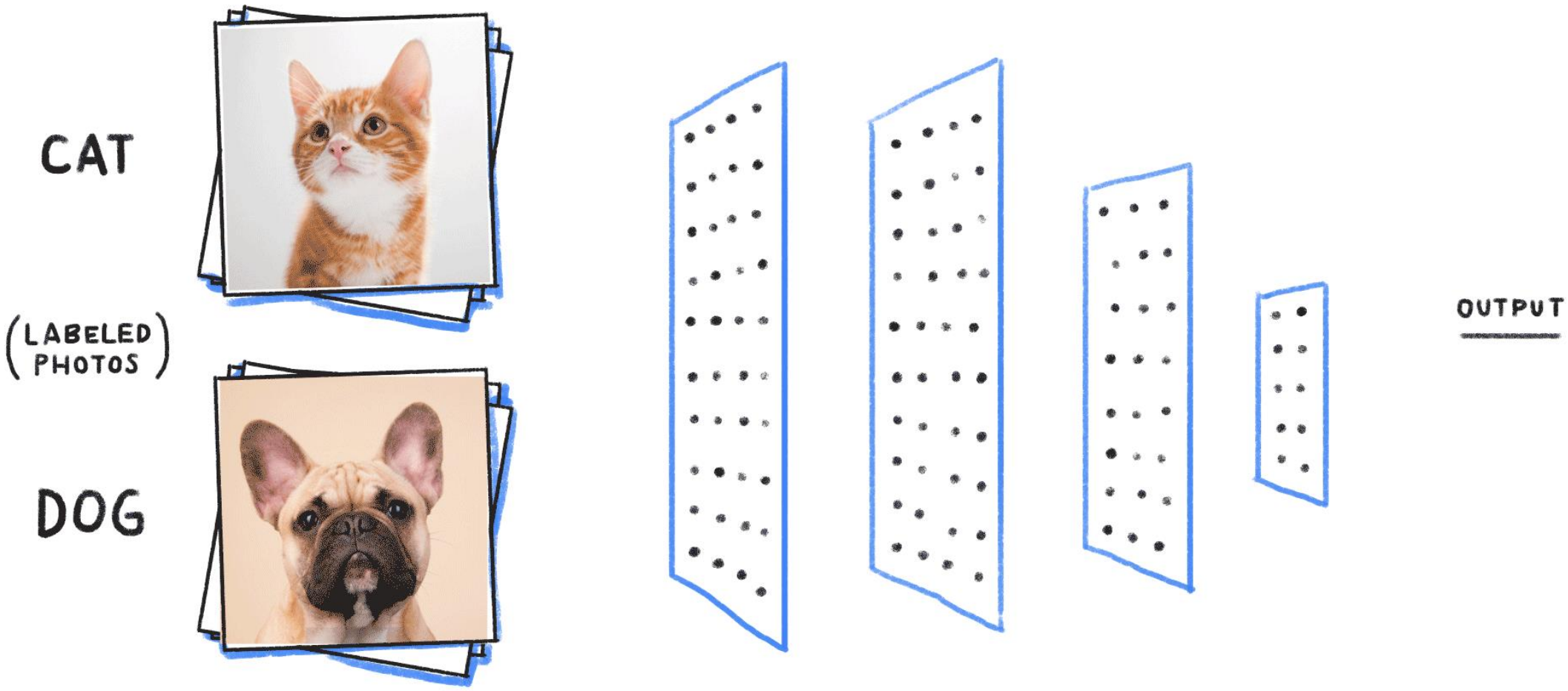
Example of Deep Learning



- Sides = 4 ✓
- Closed ✓
- Perpendicular ✓
- Equal sizes ✓

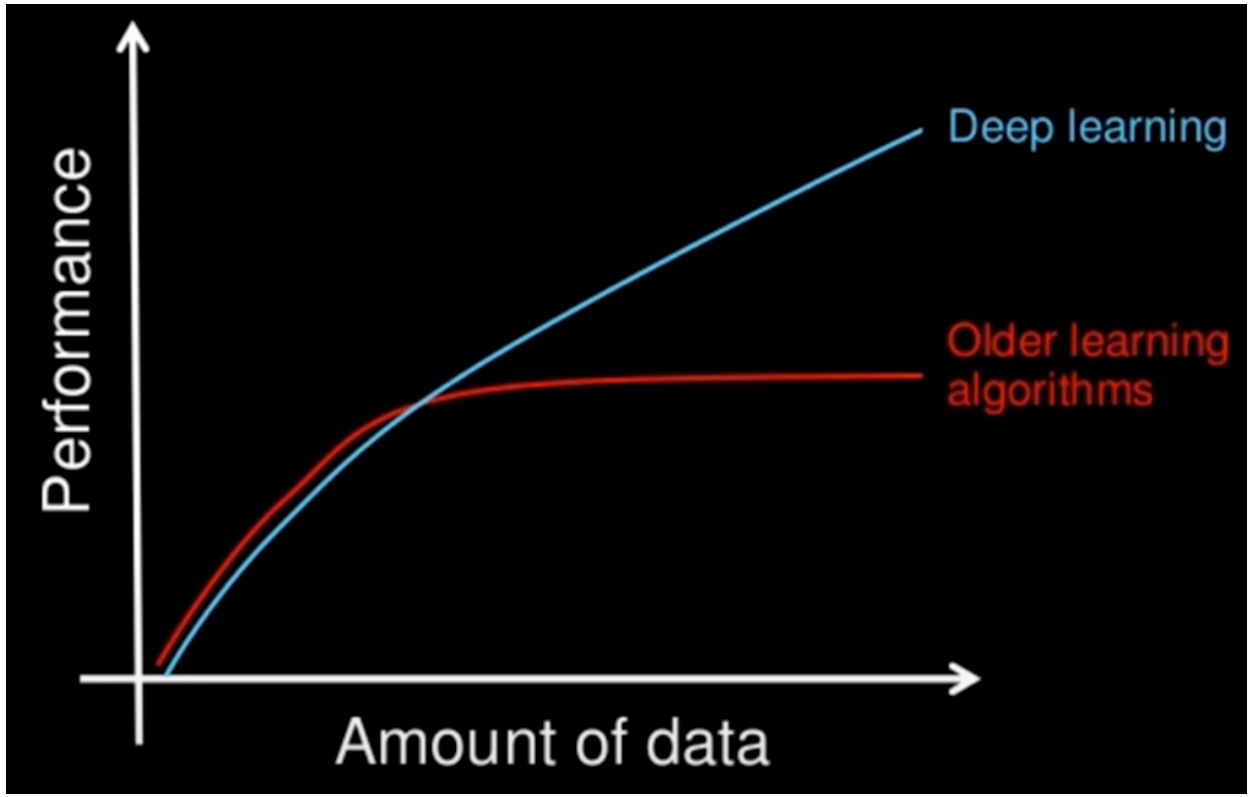
Well, it is nothing but a nested hierarchy of basic concepts.

Example of Deep Learning



Deep Learning Vs Machine Learning

Deep learning **is** a machine learning



Data dependency

Deep learning depend on the high-end machine where Machine learning can work with a low-end machine

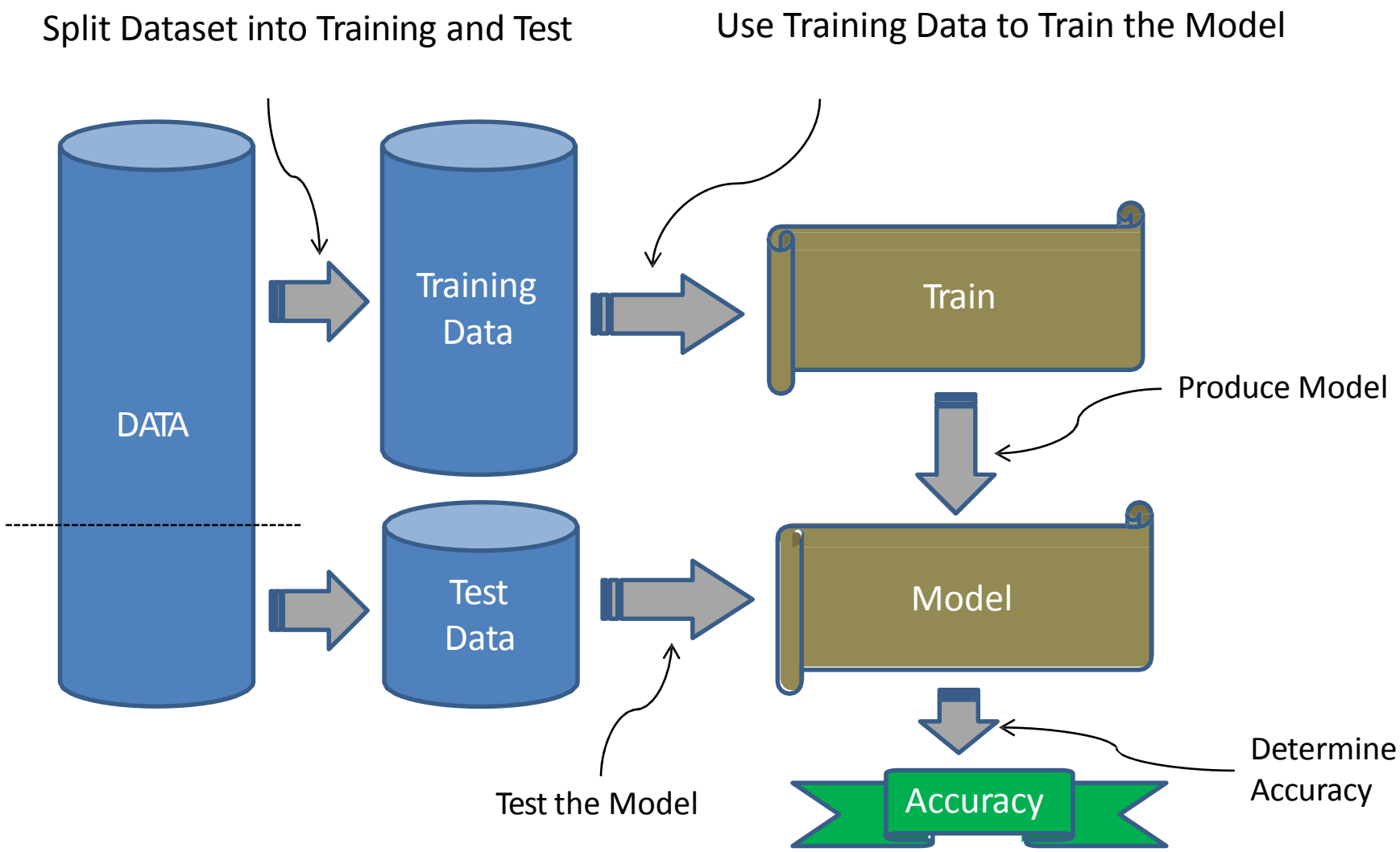


Hardware dependency

Splitting Datasets

- To use a dataset in Machine Learning, the dataset is split into a training and test set.
- The training set is used to train the model.
- The test set is used to test the accuracy of the model.
- Typically, split 80% training, 20% test.

It's About Training



Serial Splitting of the Dataset

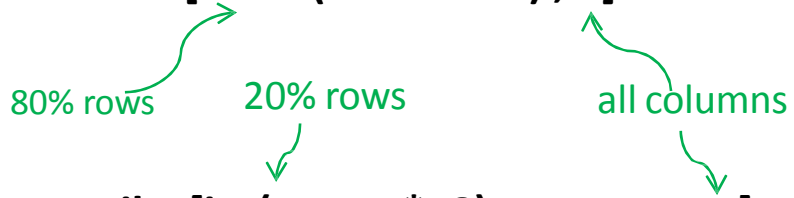
The simplest method of splitting data is to split it serially.

- Take the first 80% rows and put them into the training set.
- Take the remaining 20% rows and put them into the test set.

```
import pandas as pd                    # pandas library
dataset = pd.read_csv("Data.csv")      # read in data as panda dataframe

nrows = dataset.shape[ 0 ]             # property shape[ 0 ] is the number of rows

train = dataset.iloc[ 1: int(nrows * .8) , : ]
test  = dataset.iloc[int(nrows * .8) +1, nrows, : ]
```



Random Splitting of the Dataset

Another method is too pick rows at random.

- Sci-kit learn has a built-in method

```
from sklearn.cross_validation import train_test_split

ncols = dataset.shape[ 1 ]        # property shape[ 0 ] is the number of columns

# Assume label is the last column in the dataset
X = dataset.iloc[ :, :-1 ]        # X is all the features (exclude last column)
y = dataset.iloc[ :, ncols ]      # Y is the label (last column)


# Split the data, with 80% train and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

split size

seed for random
Number generator

Data Imbalance-Overfitting

- If the training data is overly unbalanced, then the model will predict a non-meaningful result.

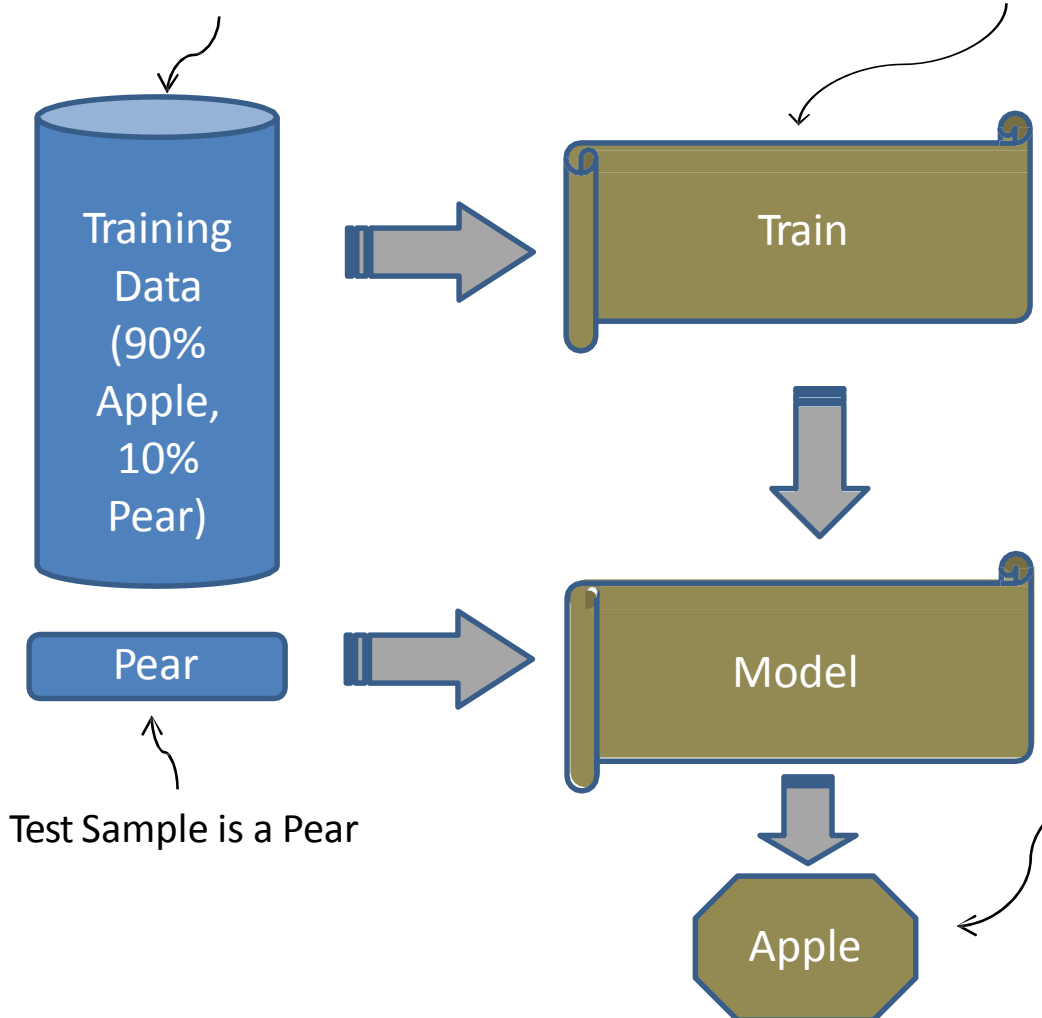
- For example, if the model is a binary classifier (e.g., apple vs. pear), and nearly all the samples are of the same label (e.g., apple), then the model will simply learn that everything is a label (apple).

- This is called overfitting. To prevent overfitting, there needs to be a fairly equal distribution of training samples for each classification, or range if the label is a real value.

Data Imbalance-Overfitting

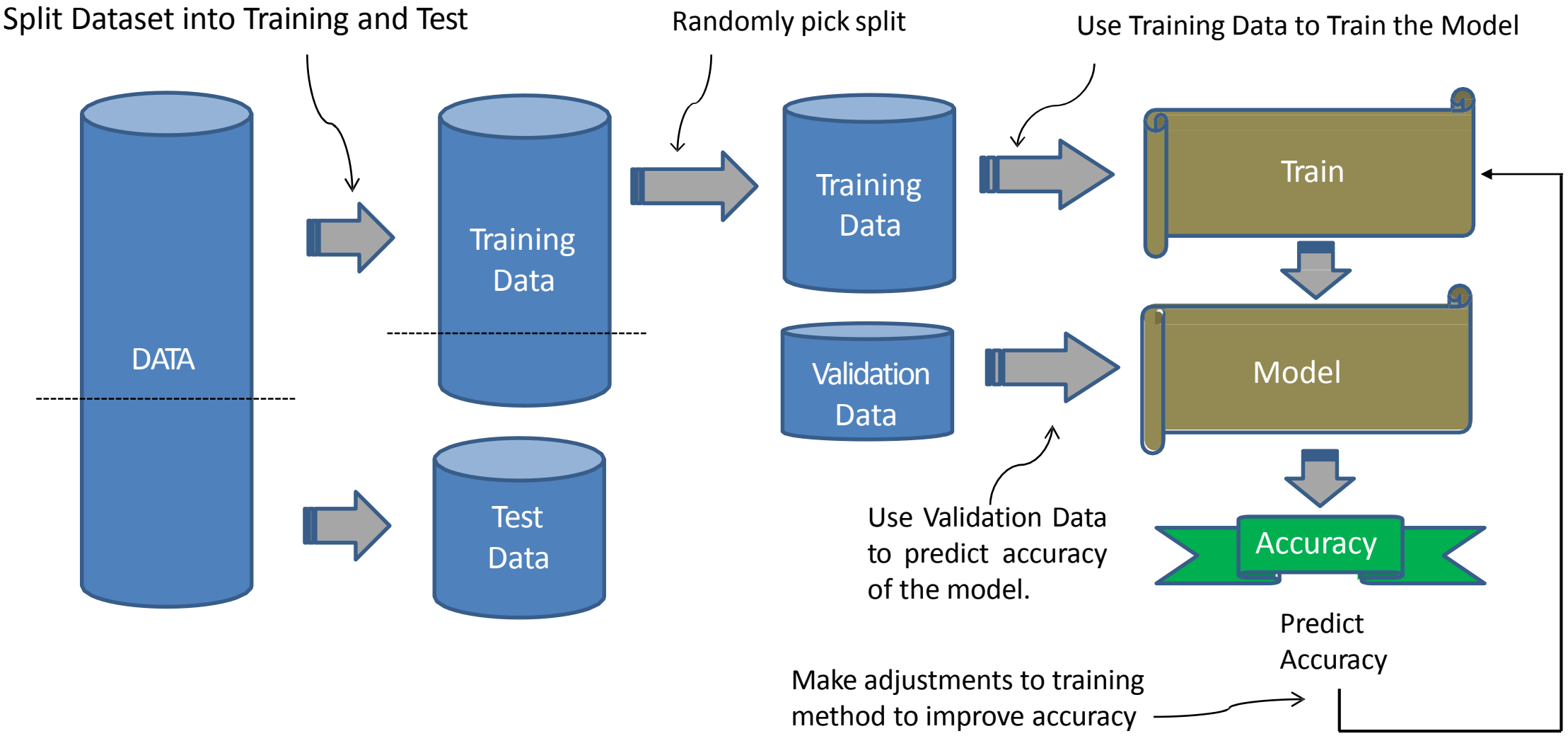
Nearly all samples are an Apple

In an imbalance, the model will fit itself to the imbalance, not the predictor.



The model will most likely predict the pear as an apple.

Cross Validation



K-Fold Cross Validation

- K-Fold is a well-known form of cross-validation.

- Steps:

1. Partition the dataset into k equal-sized partitions.
2. Select one partition as the validation data.
3. Use the remaining k-1 as the training data.
4. Train the model and determine accuracy from the validation data.
5. Repeat the process k times, selecting a different partition each time for the validation data.
6. Average the accuracy results.

Types of Machine Learning



Supervised Learning



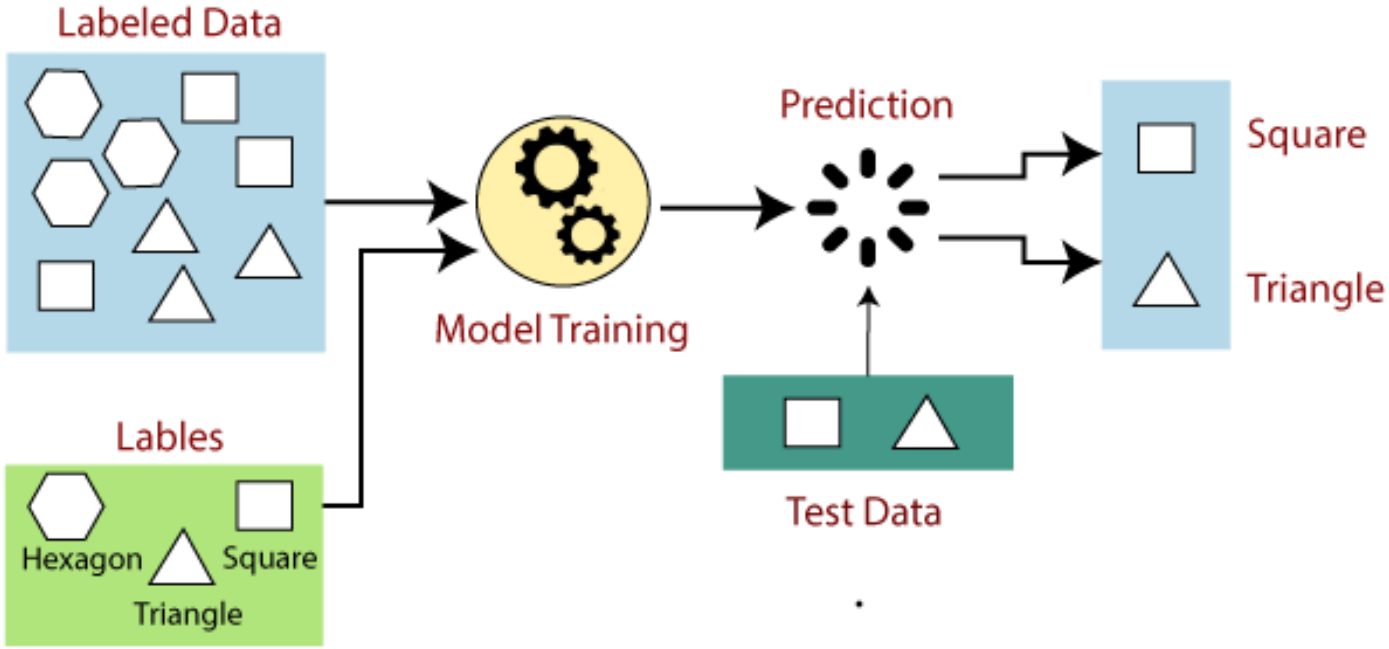
Unsupervised Learning



Reinforcement Learning

Supervised Learning

- The computer learns by making use of labeled data.
- In supervised learning, the algorithm "learns" from the training dataset by iteratively making predictions on the data and adjusting for the correct answer.



Example IRIS Dataset

IRIS

https://archive.ics.uci.edu/ml/datasets/Iris

Instances (samples, observations)



	sepal_length	sepal_width	petal_length	petal_width	class
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
...	...	...	...	...	...
50	6.4	3.2	4.5	1.5	veriscolor
...	...	...	...	...	...
150	5.9	3.0	5.1	1.8	virginica

Features (attributes, dimensions)

Classes (targets)

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1D} \\ x_{21} & x_{22} & \cdots & x_{2D} \\ x_{31} & x_{32} & \cdots & x_{3D} \\ \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{ND} \end{bmatrix}$$

$$\mathbf{y} = [y_1, y_2, y_3, \cdots y_N]$$

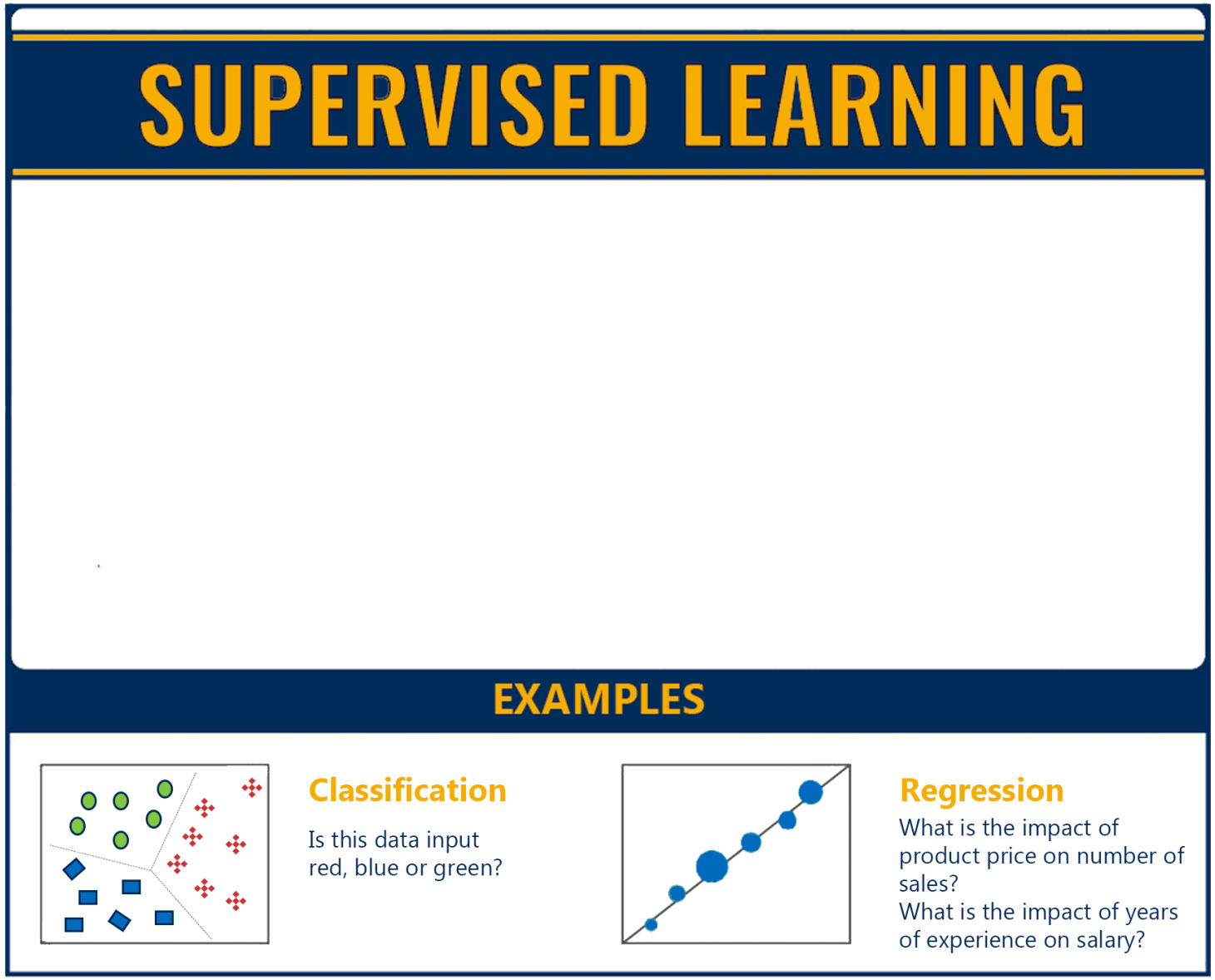
The loan data (Example)

Approved or not

ID	Age	Has_Job	Own_House	Credit_Rating	Class
1	young	false	false	fair	No
2	young	false	false	good	No
3	young	true	false	good	Yes
4	young	true	true	fair	Yes
5	young	false	false	fair	No
6	middle	false	false	fair	No
7	middle	false	false	good	No
8	middle	true	true	good	Yes
9	middle	false	true	excellent	Yes
10	middle	false	true	excellent	Yes
11	old	false	true	excellent	Yes
12	old	false	true	good	Yes
13	old	true	false	good	Yes
14	old	true	false	excellent	Yes
15	old	false	false	fair	No

Supervised Learning

- The computer learns by making use of labeled data.
- In supervised learning, the algorithm "learns" from the training dataset by iteratively making predictions on the data and adjusting for the correct answer.



Applications of supervised Learning

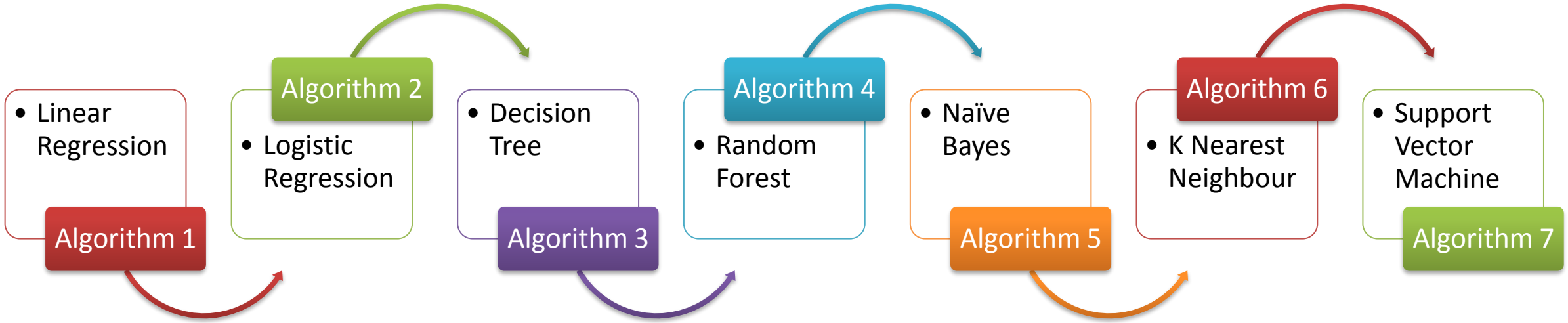


Risk Evaluation



Forecast Sales

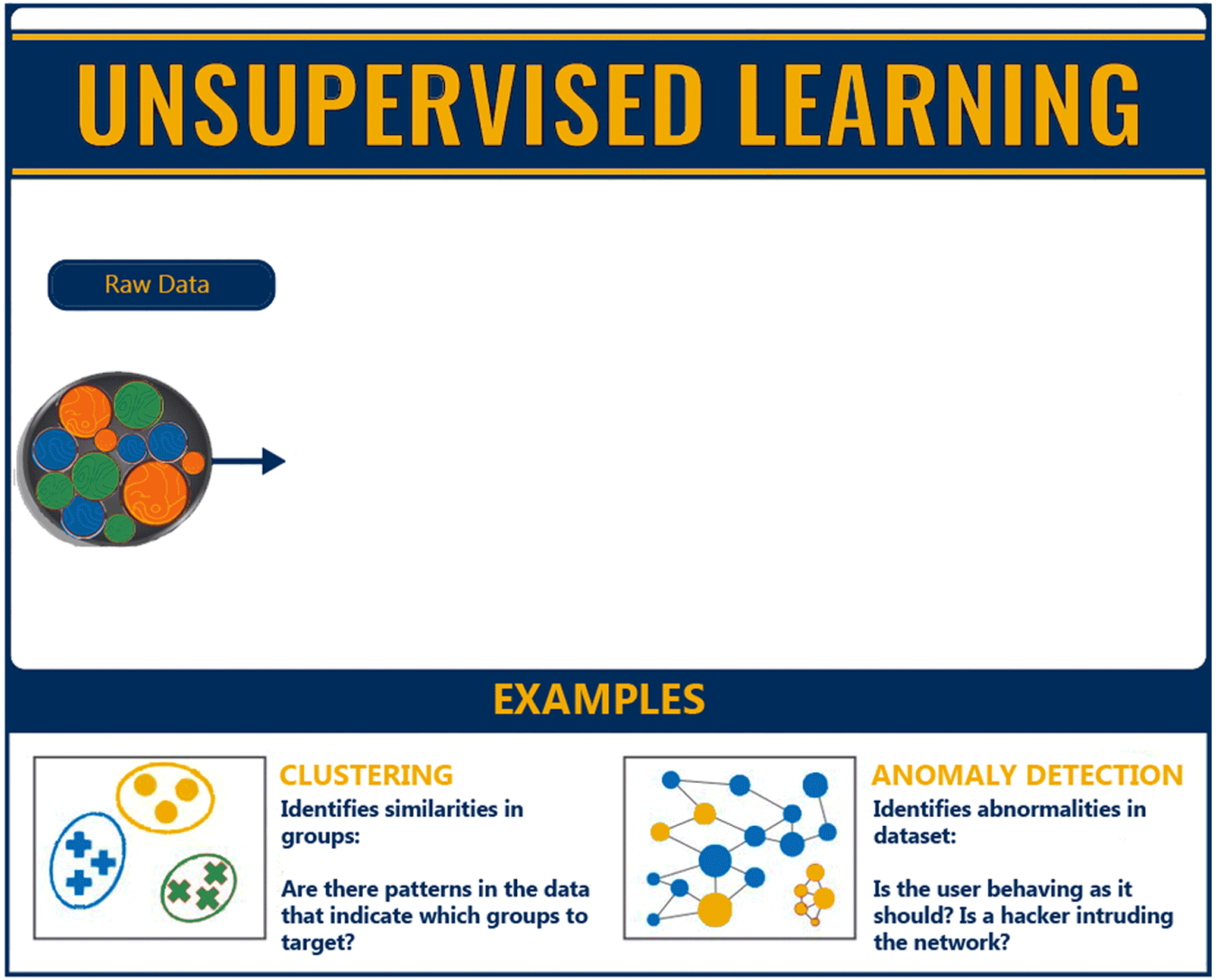
Popular Algorithms in Supervised Machine Learning



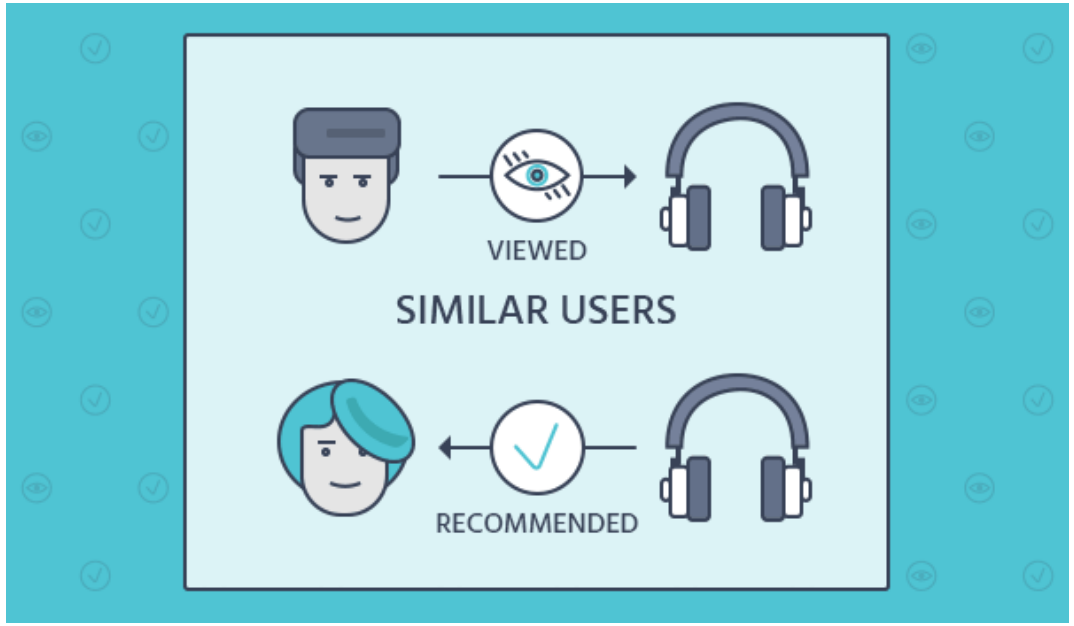
Unsupervised Learning

- Computer learns by making use of unlabeled data without any guidence.

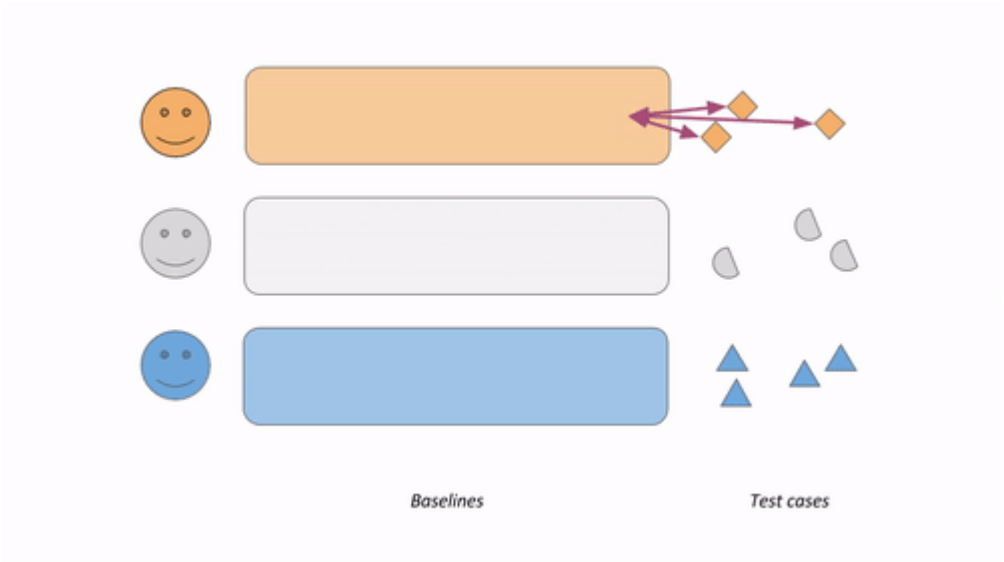
- It uses machine learning algorithms to analyze and cluster unlabeled datasets. These algorithms discover hidden patterns or data groupings without the need for human intervention.



Applications of Unsupervised Learning



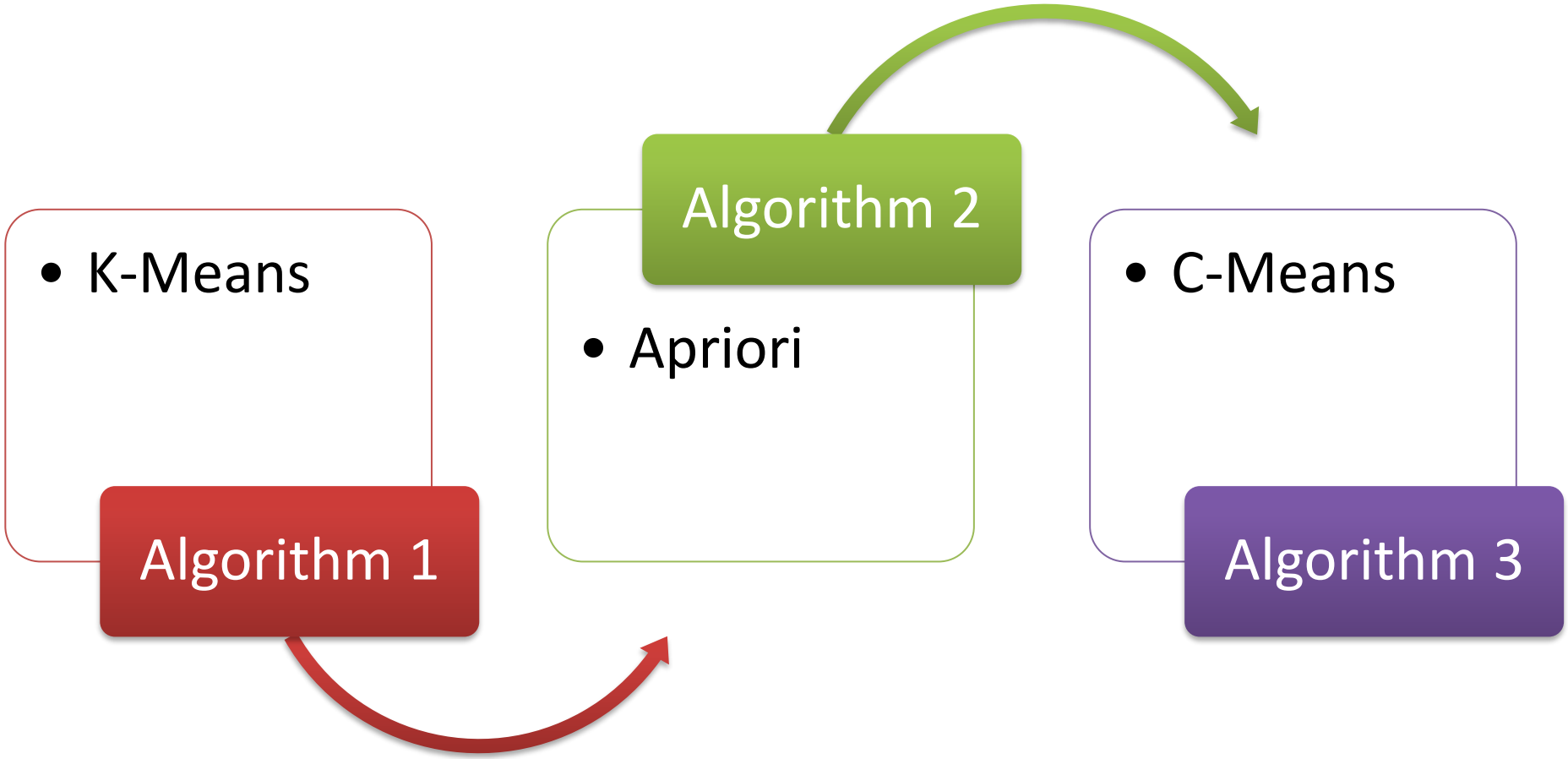
Recommendation Systems



Anomaly Detection

Anomaly detection (aka outlier analysis) is a step in data mining that identifies data points, events, and/or observations that deviate from a dataset's

Popular Algorithms in Unsupervised Machine Learning



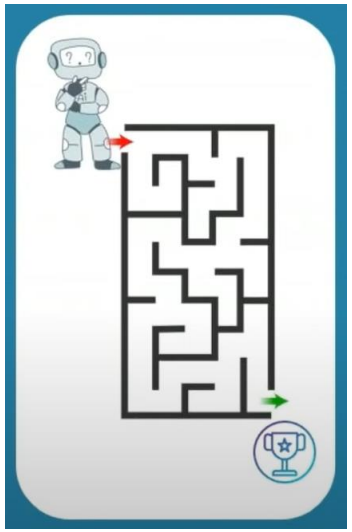
Reinforcement Learning

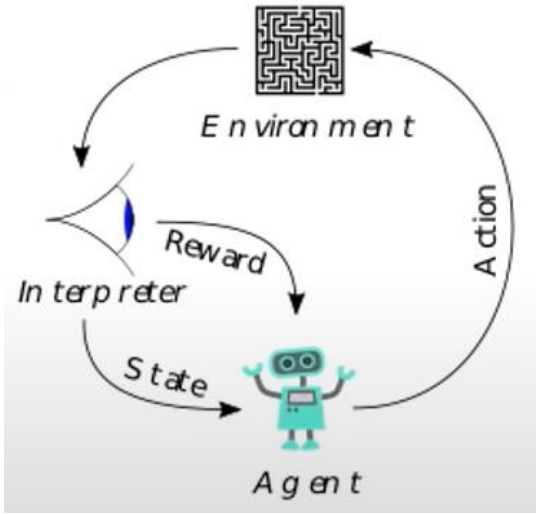
- In Reinforcement learning an agent interacts with its environment by producing actions & discovers errors or rewards.

- In reinforcement learning there is no predefined data given and agents learn most of the time from the environment.

- No supervision



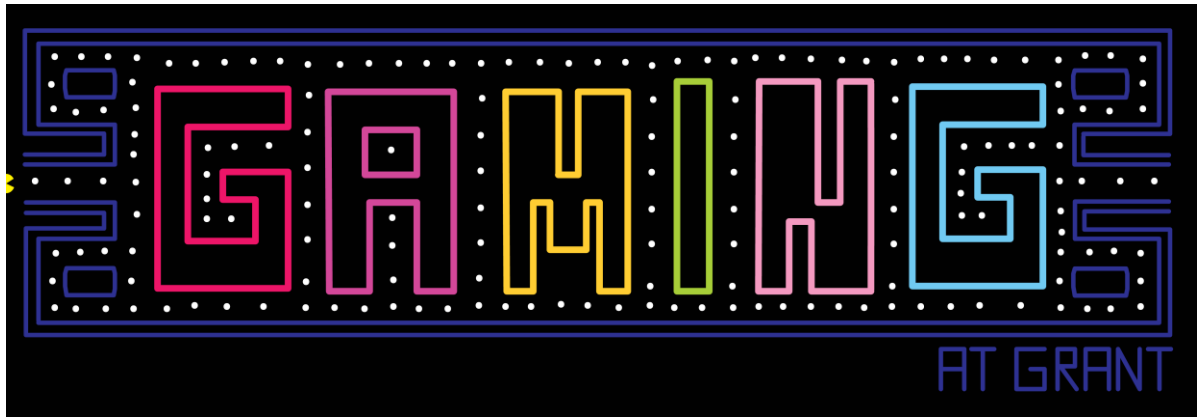




Applications of Reinforcement Learning

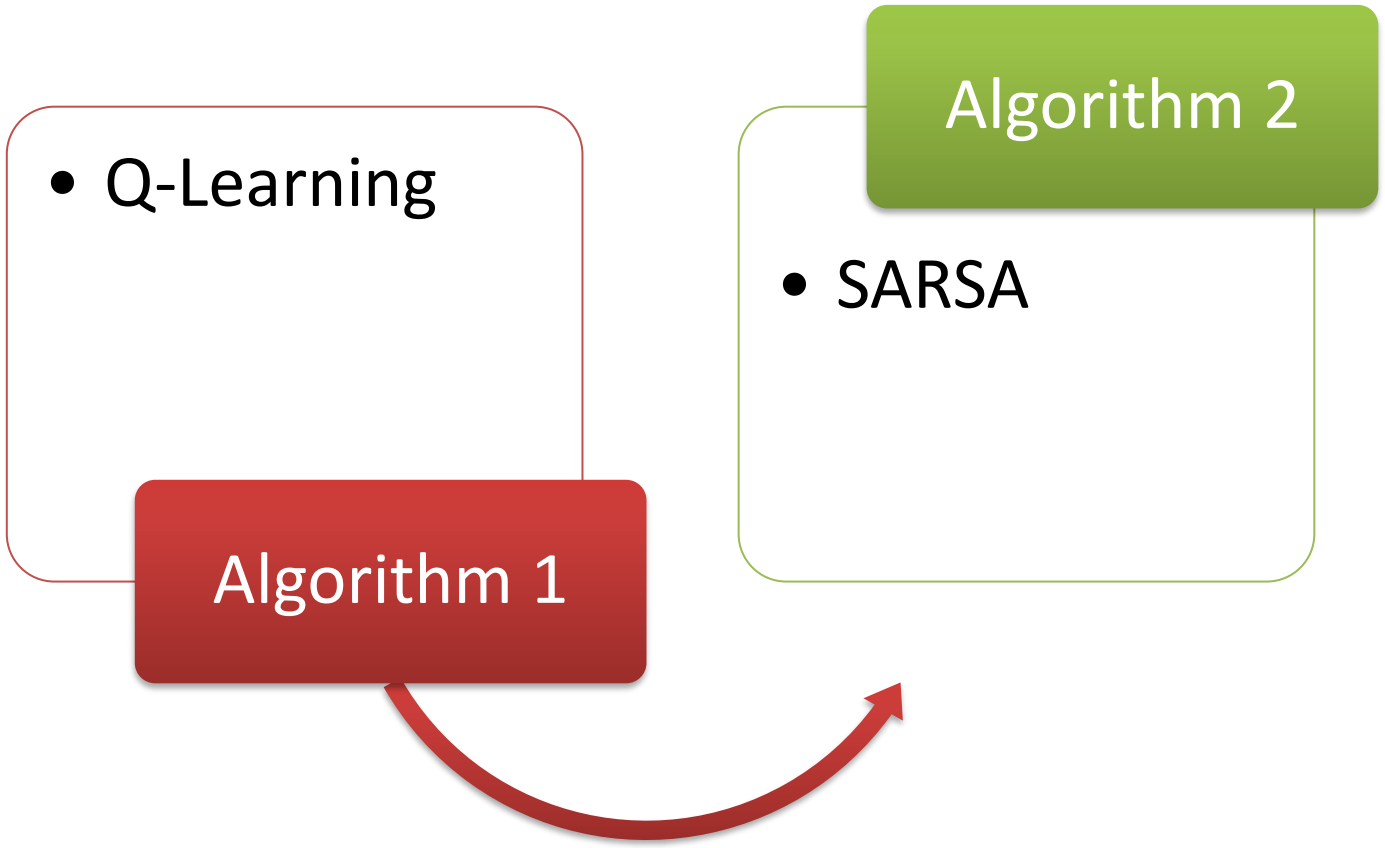


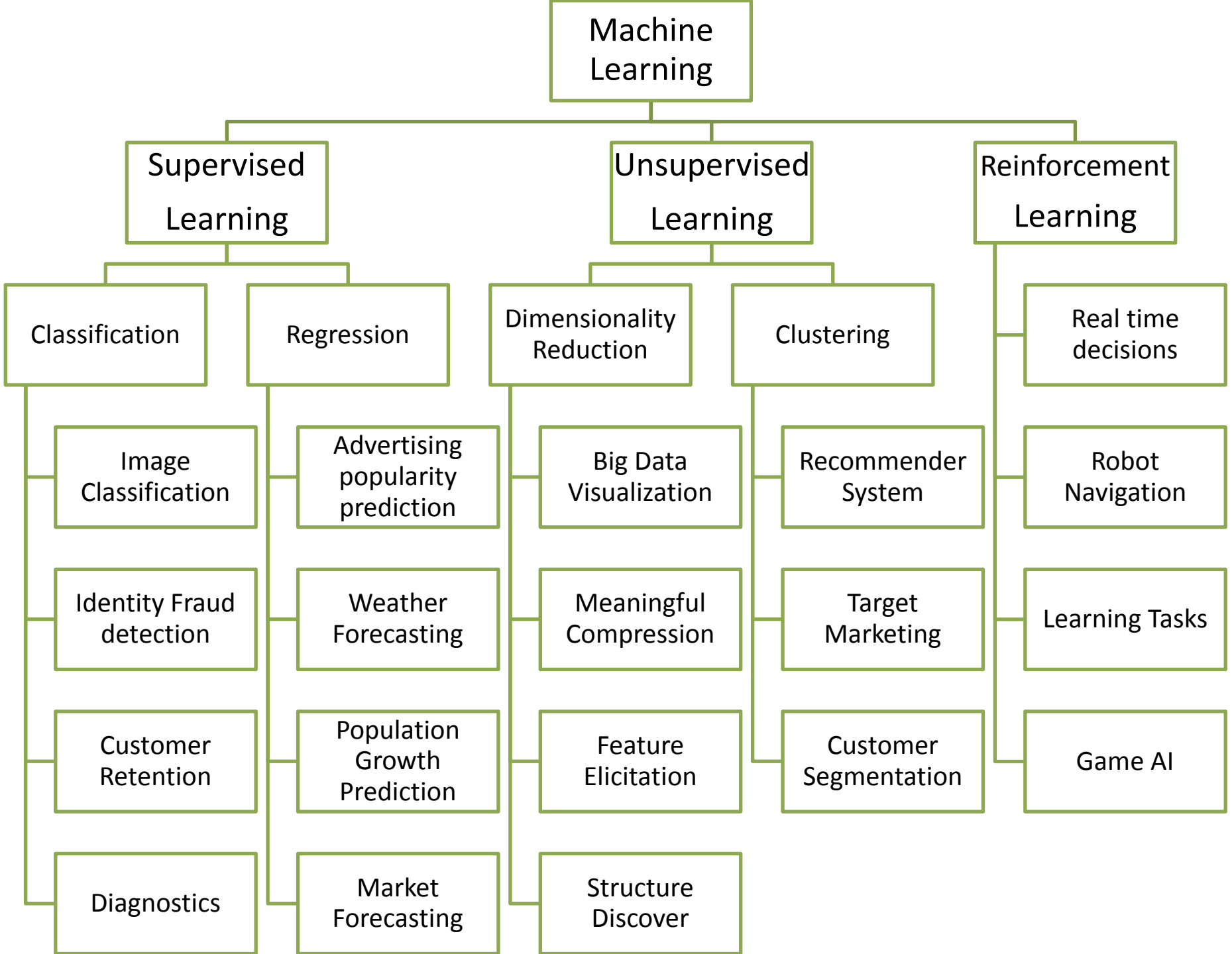
Self driving cars



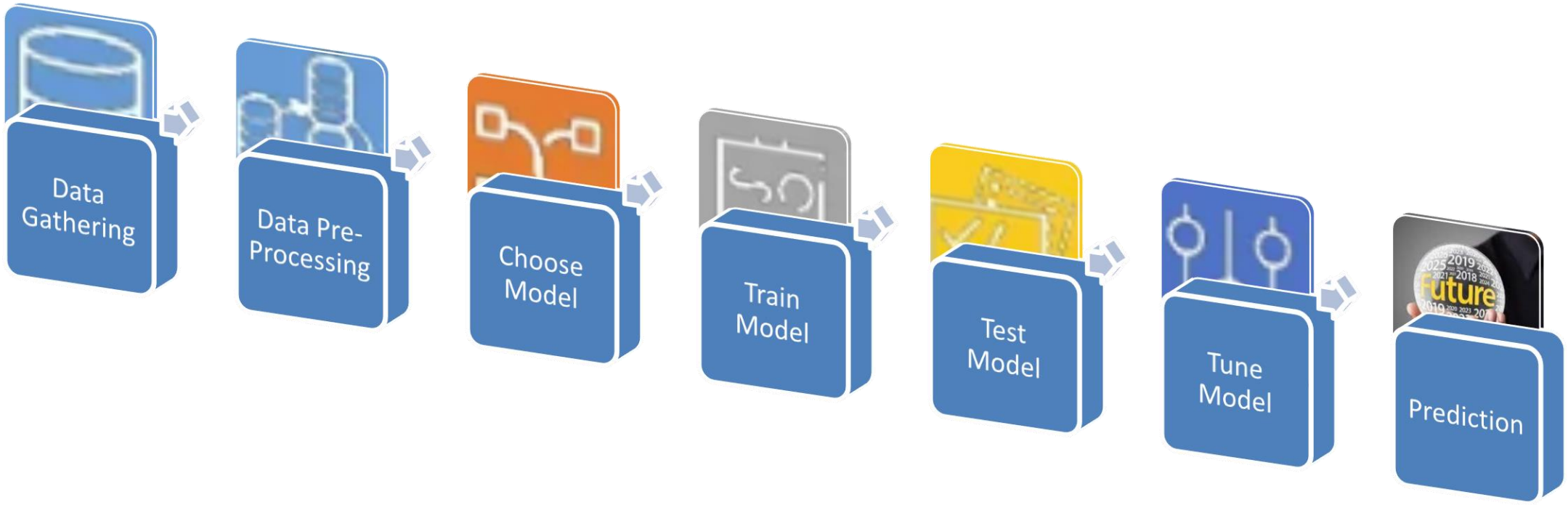
Gaming

Popular Algorithms in Reinforcement Machine Learning




Machine Learning
Supervised Learning
Unsupervised Learning
Reinforcement Learning
Classification
Regression
Dimensionality Reduction
Clustering
Real time decisions
Image Classification
Identity Fraud detection
Customer Retention
Diagnostics
Advertising popularity prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Big Data Visualization
Meaningful Compression
Feature Elicitation
Structure Discover
Recommender System
Target Marketing
Customer Segmentation
Robot Navigation
Learning Tasks
Game AI

Processes involved in Machine Learning



Introduction to SciKit-Learn library for machine learning algorithms

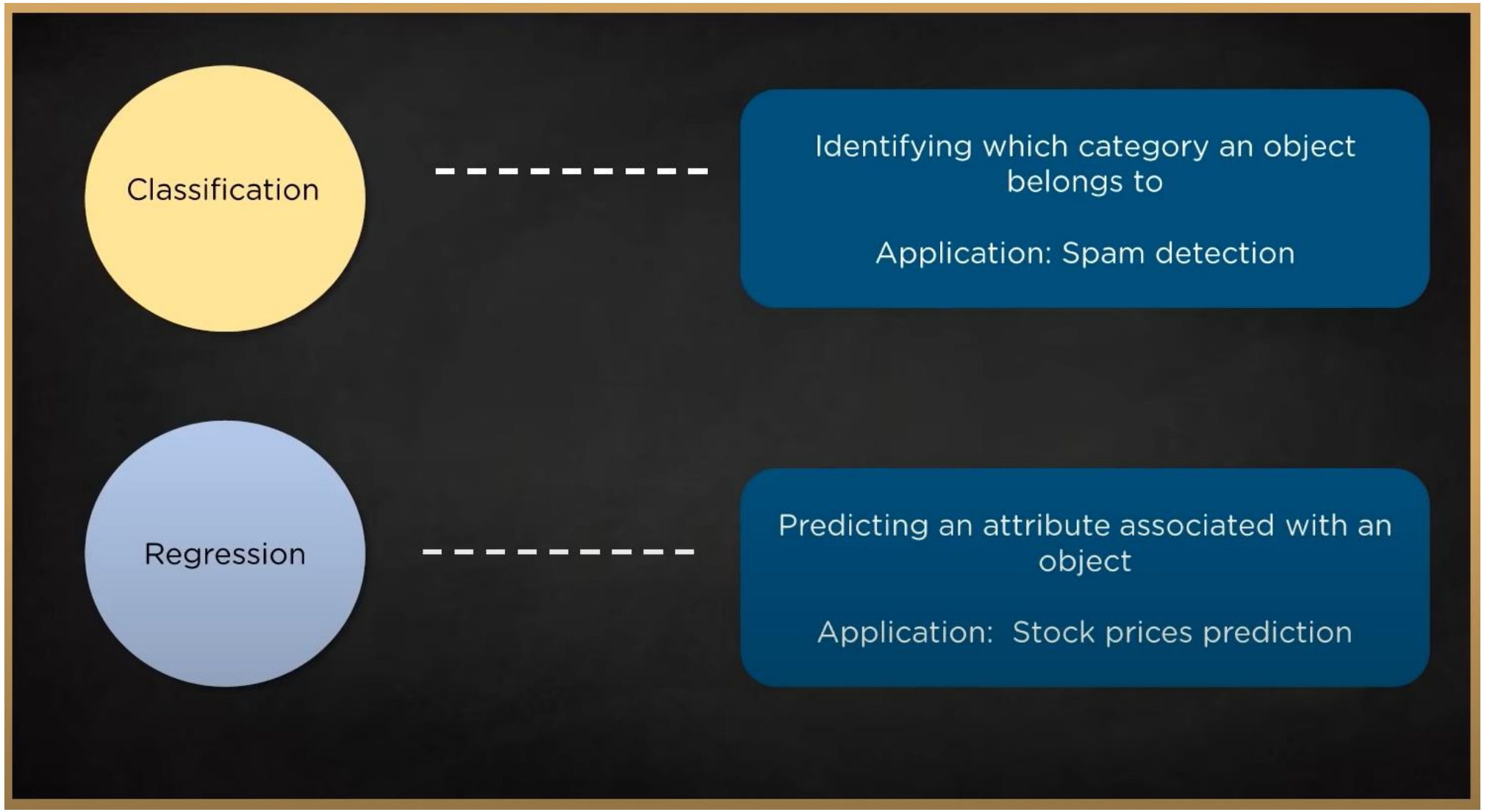


What is Scikit-learn?

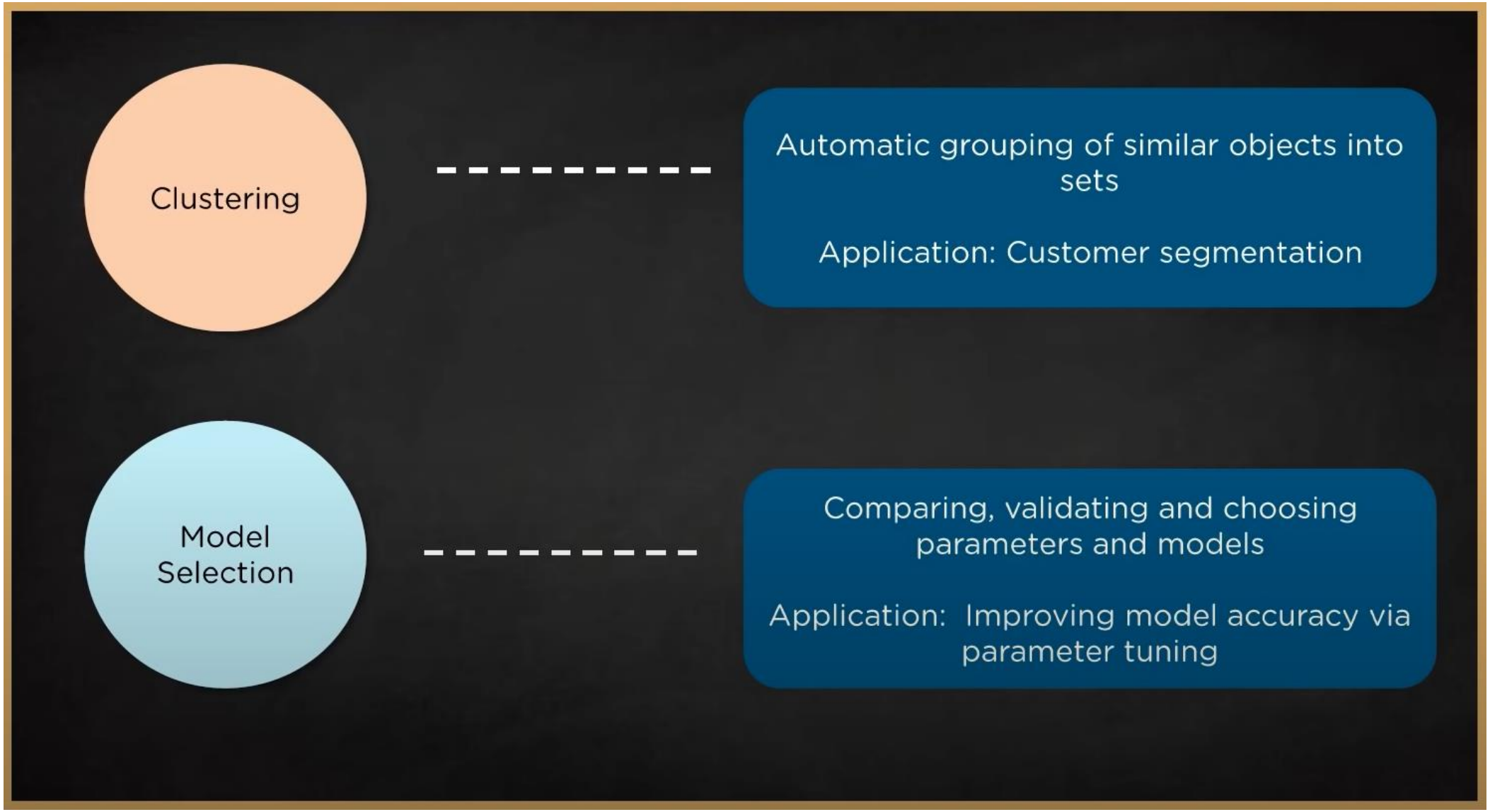
- Popular machine learning toolkit in Python.
- Simple and efficient tool for data mining and data analysis.
- Built on NumPy, SciPy, and matplotlib.
- Open source, commercially usable-BSD license.



What we can achieve using scikit-learn

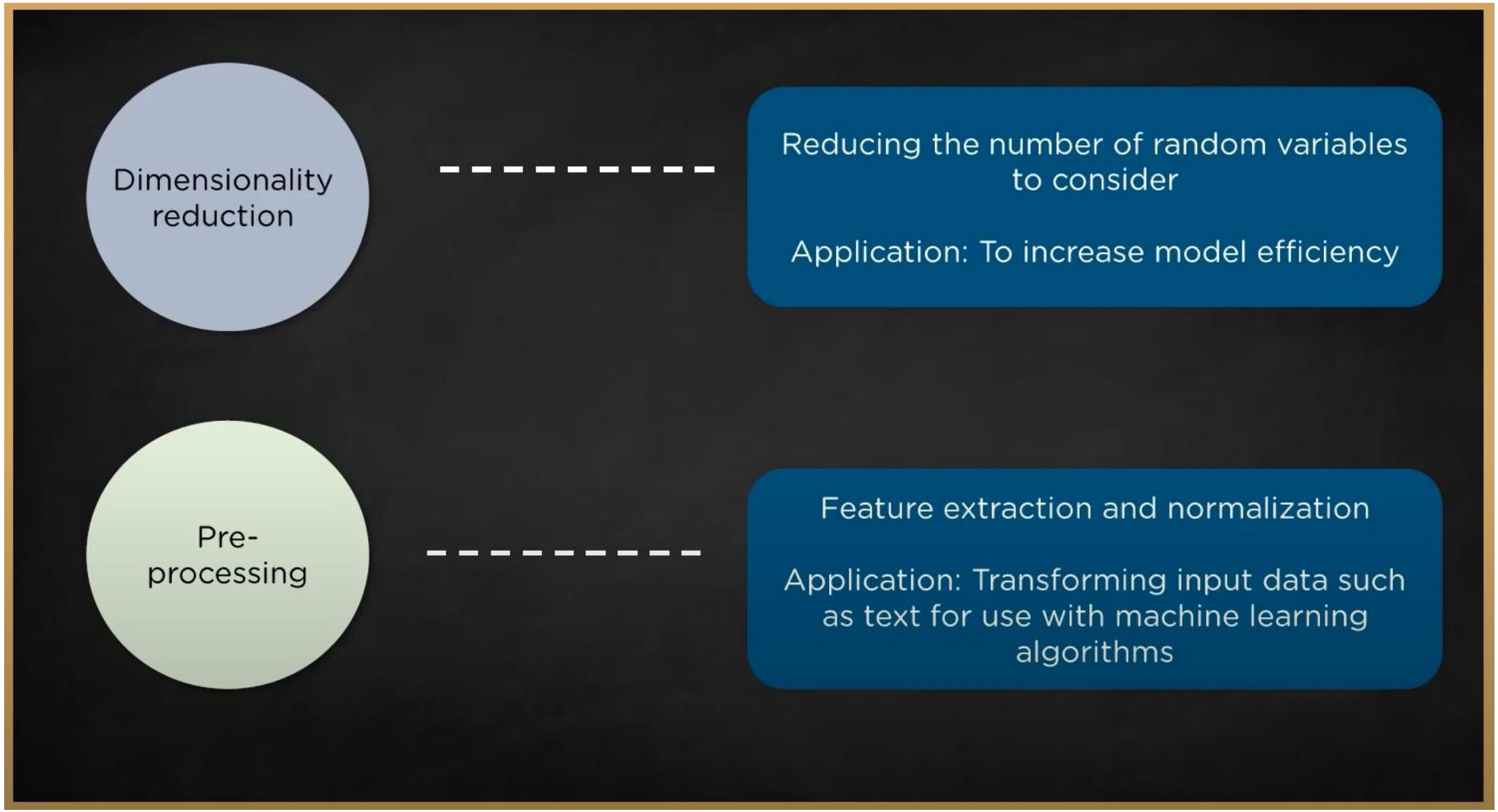


What we can achieve using scikit-learn



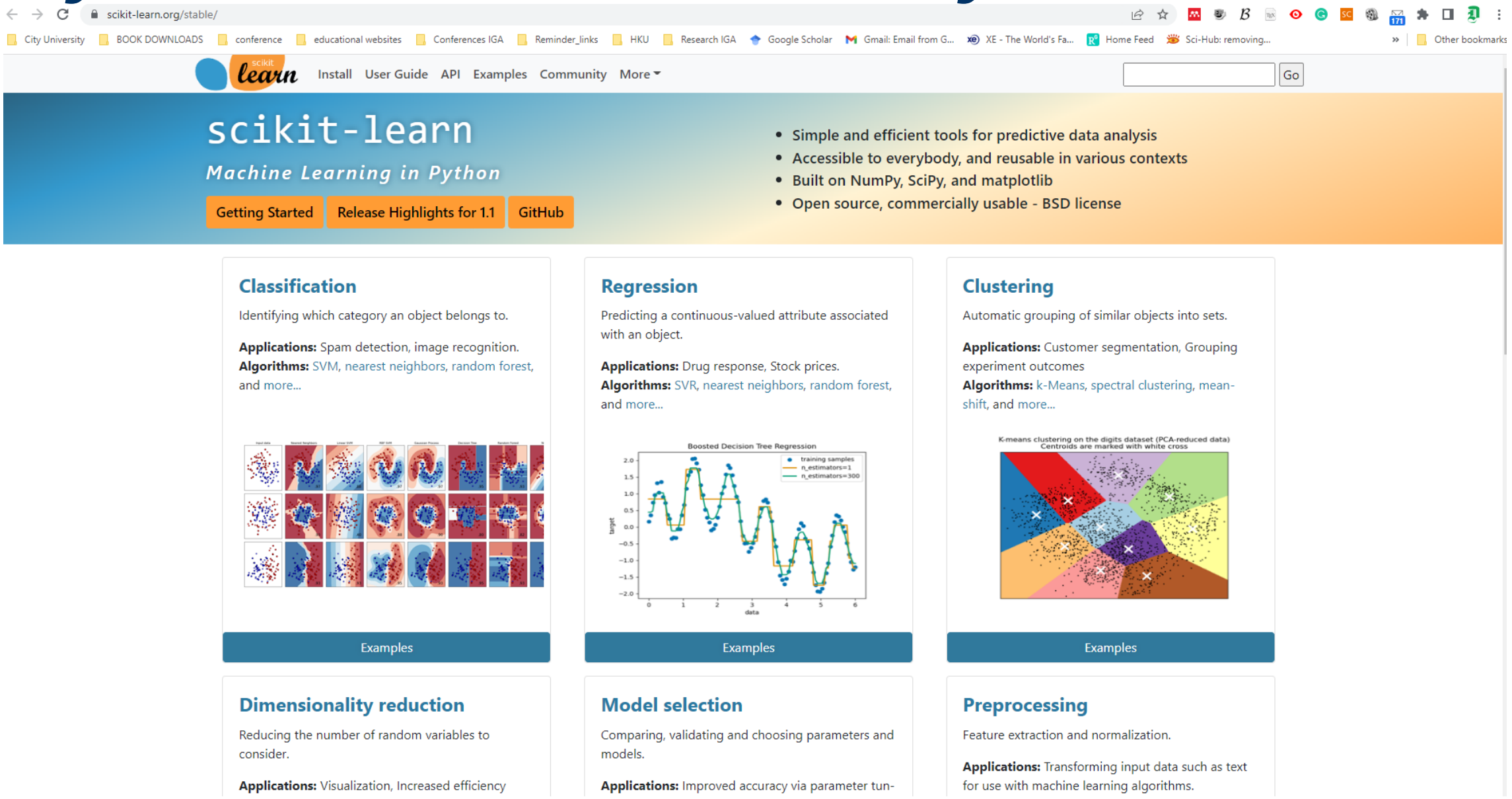
scikit learn

What we can achieve using scikit-learn

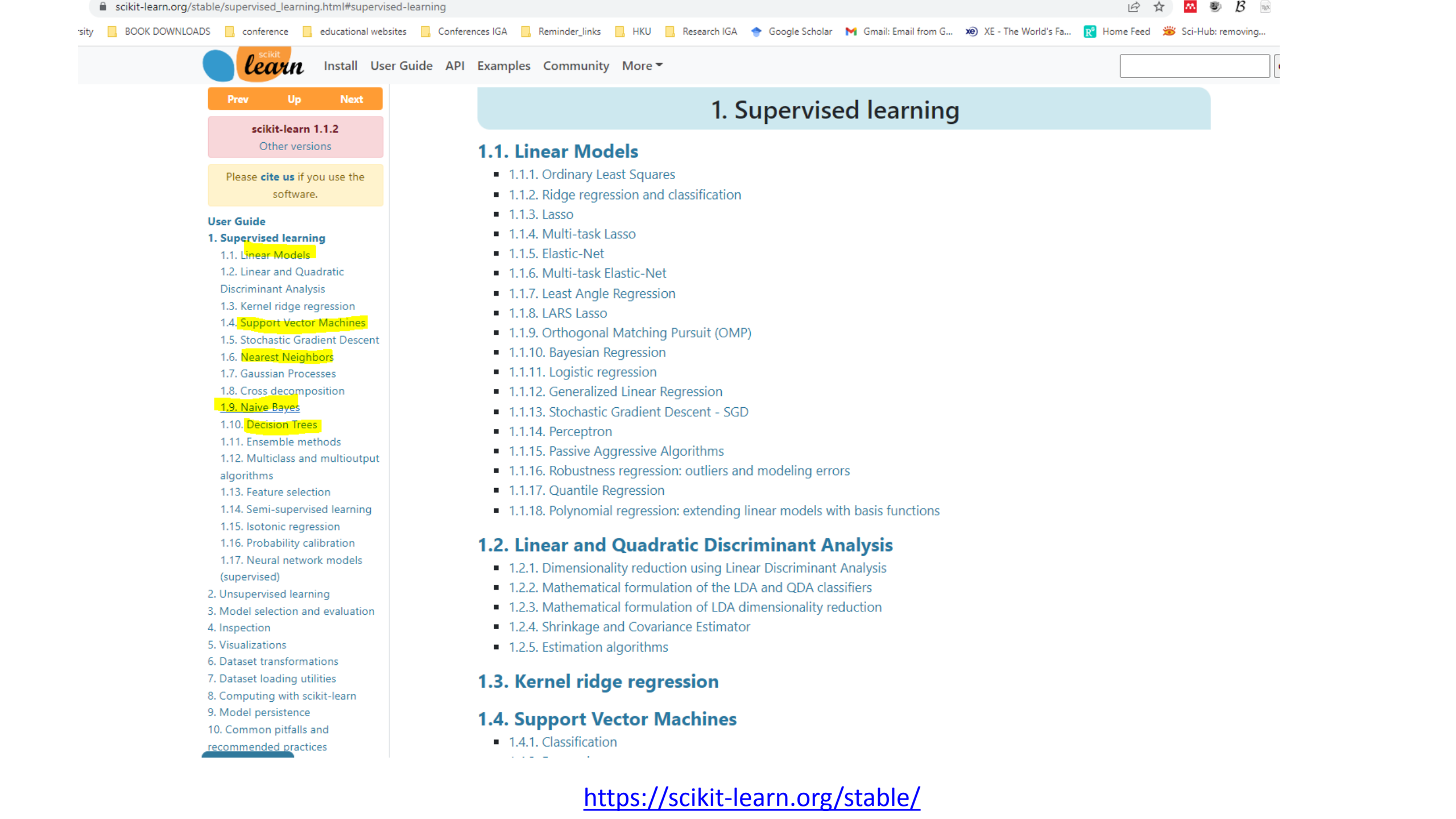


scikit learn

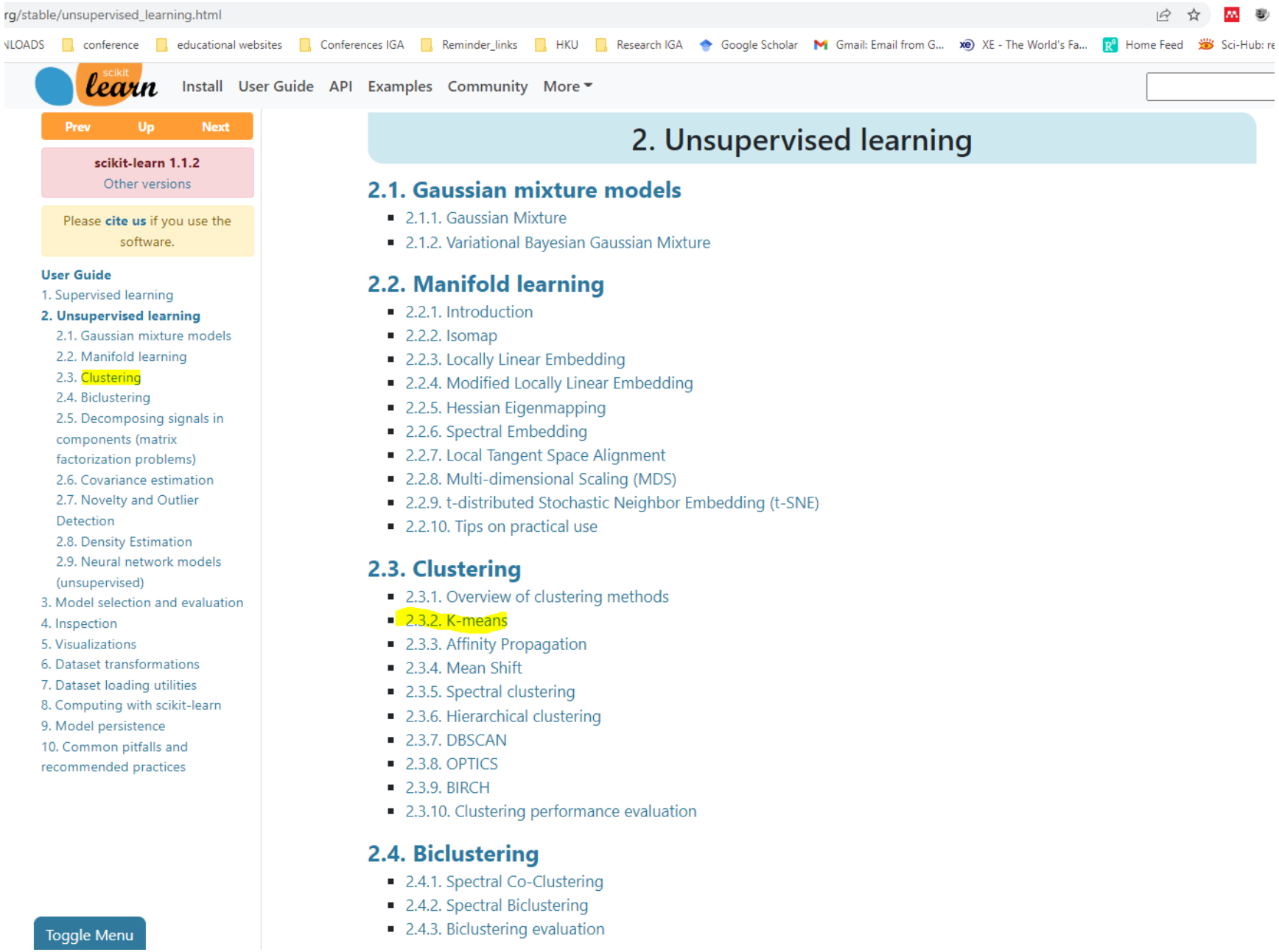
Python Scikit-learn Library



https://scikit-learn.org/stable/



https://scikit-learn.org/stable/



https://scikit-learn.org/stable/

How to import it on Jupyter Notebook

```
#Importing required packages.
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn import svm
from sklearn.neural_network import MLPClassifier
#from sklearn.linear_model import SGDClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
%matplotlib inline
```

Supervised Learning Algorithms

Linear and Logistic Regression

Regression

- Regression Analysis is a predictive modeling technique.
- It estimates the relationship between a dependent (target) and an independent variable (predictor).

Uses of Regression

Three major uses for regression analysis are

- Determining the strength of predictors
- Forecasting an effect, and
- Trend forecasting.

Types of Regression

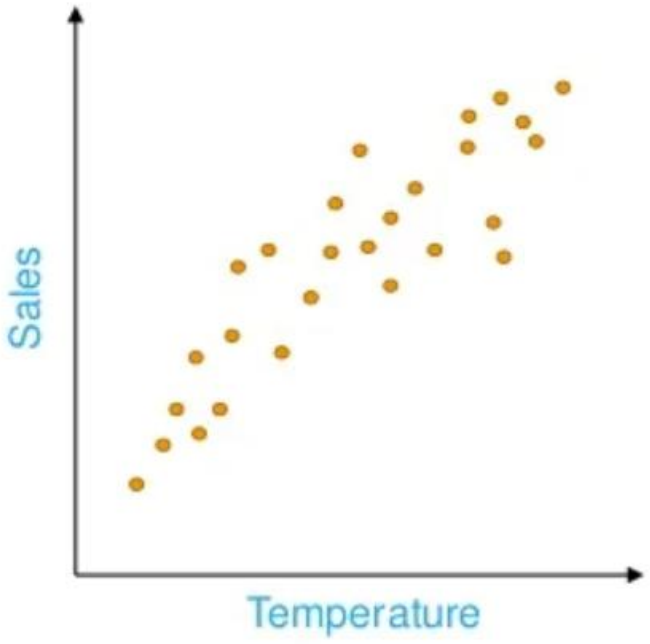
- Linear Regression
- Logistic Regression
- Polynomial Regression

Basis	Linear Regression	Logistic Regression
Core Concept	The data is modelled using a straight line	The probability of some obtained event is represented as a linear function of a combination of predictor variables.
Used with	Continuous Variable	Categorical Variable
Output/Prediction	Value of the variable	Probability of occurrence of event
Accuracy and Goodness of fit	measured by loss, R squared, Adjusted R squared etc.	Accuracy, Precision, Recall, F1 score, ROC curve, Confusion Matrix, etc

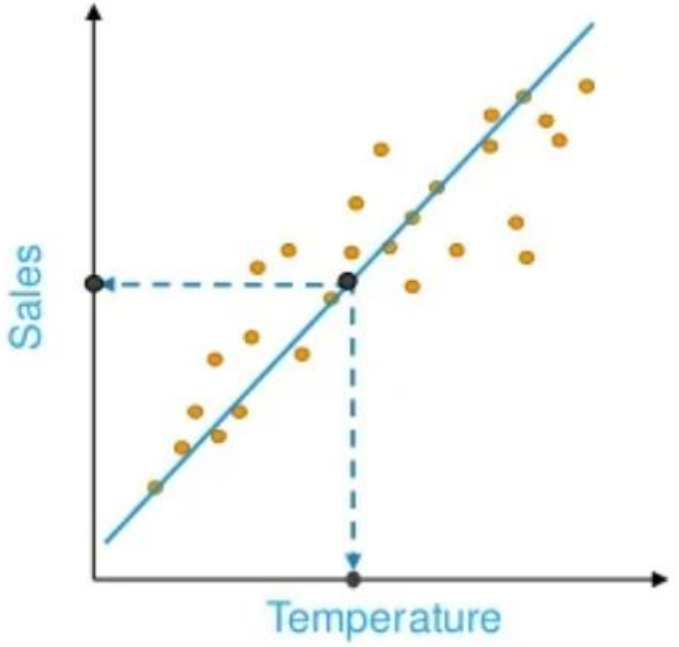
Linear Regression

- Linear regression is a linear modelling approach to find relationship between one or more independent variables (predictors) denoted as X and dependent variable (target) denoted as Y.





Plotting the sales of ice cream based on temperature

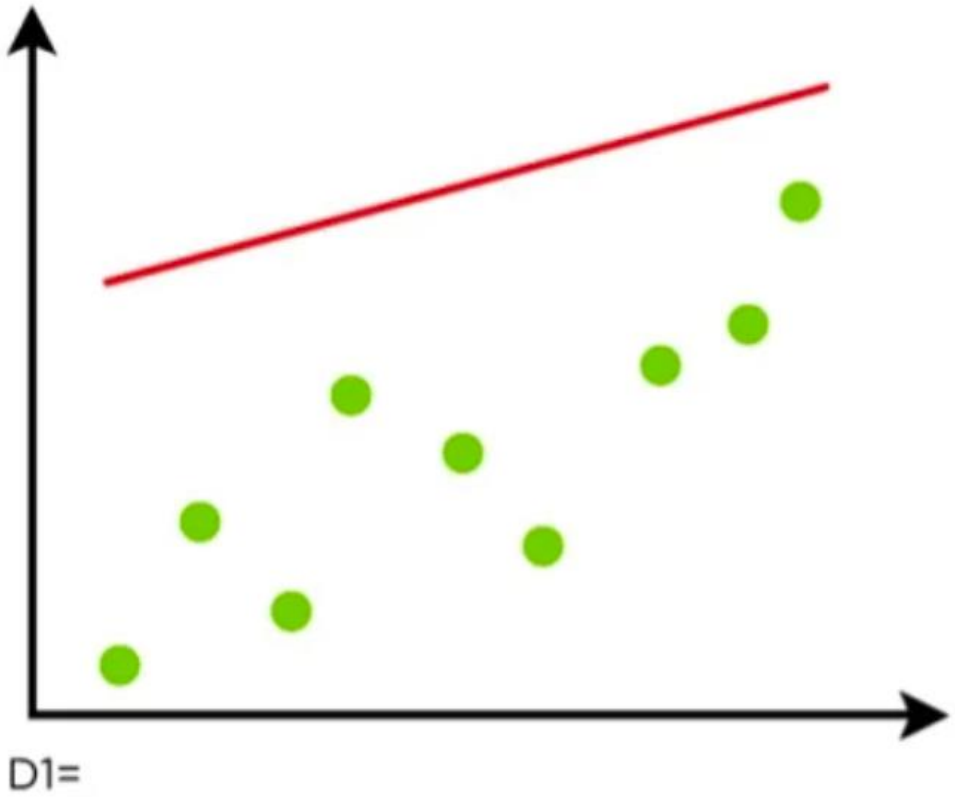


Regression line to predict the sales

Predict sales of Ice Cream based on temperature.

Linear Regression

- **Finding the best fit line:** The best fit line can be found by minimizing the distance between all the data points and the distance to the regression line. Ways to minimize this distance are the sum of squared errors, the sum of absolute errors, etc.

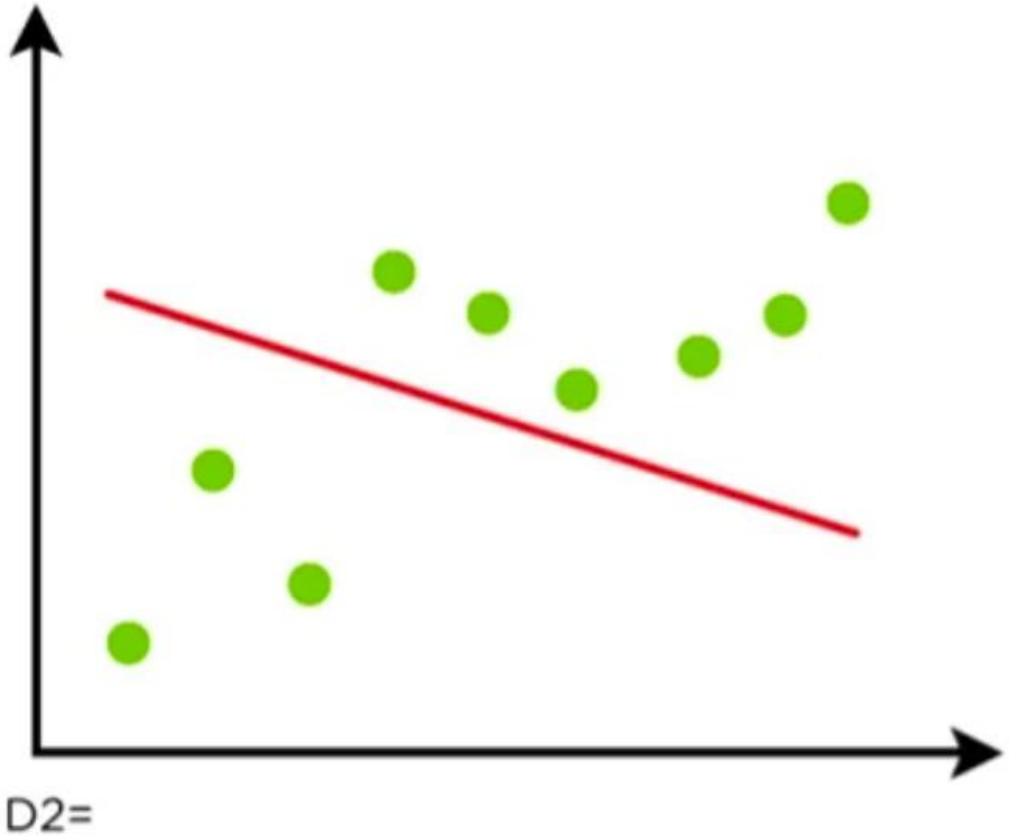


Not the best fit line

Distance between the data points and the line is maximum

Linear Regression

- **<u>Finding the best fit line:</u>** The best fit line can be found by minimizing the distance between all the data points and the distance to the regression line. Ways to minimize this distance are the sum of squared errors, the sum of absolute errors, etc.



Not the best fit line

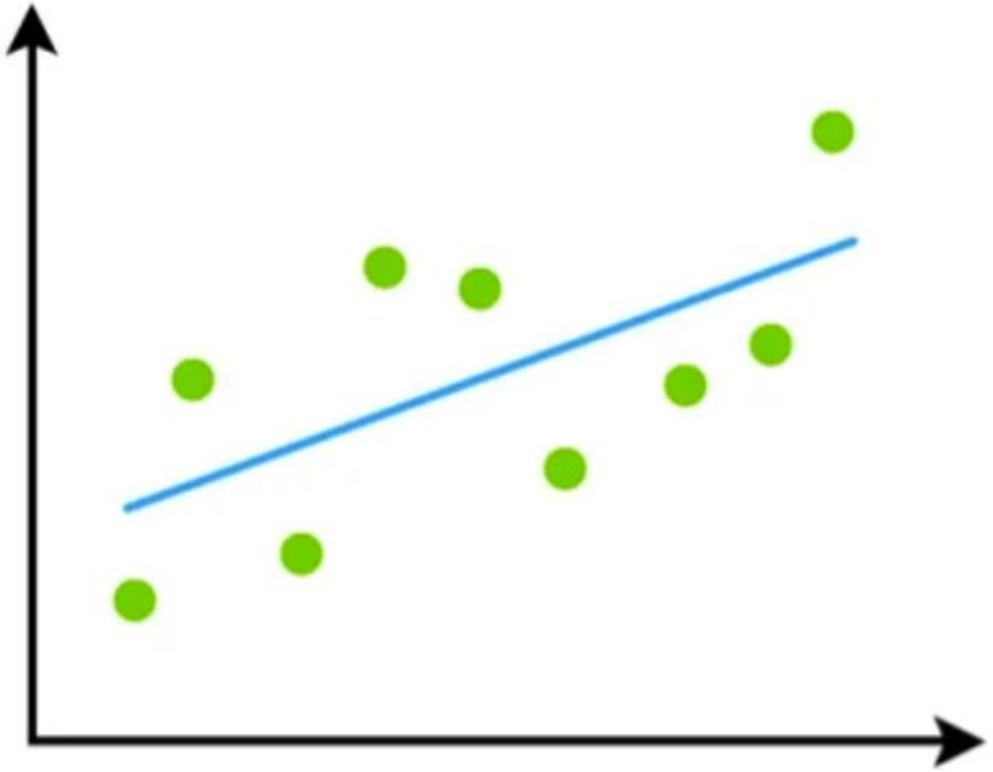
Distance between the data points and the line can still be reduced.

Linear Regression

- **<u>Finding the best fit line:</u>** The best fit line can be found by minimizing the distance between all the data points and the distance to the regression line. Ways to minimize this distance are the sum of squared errors, the sum of absolute errors, etc.

$y = mx + c$

y = dependent variable
x = independent variable
m = slope of the line
c = y-intercept



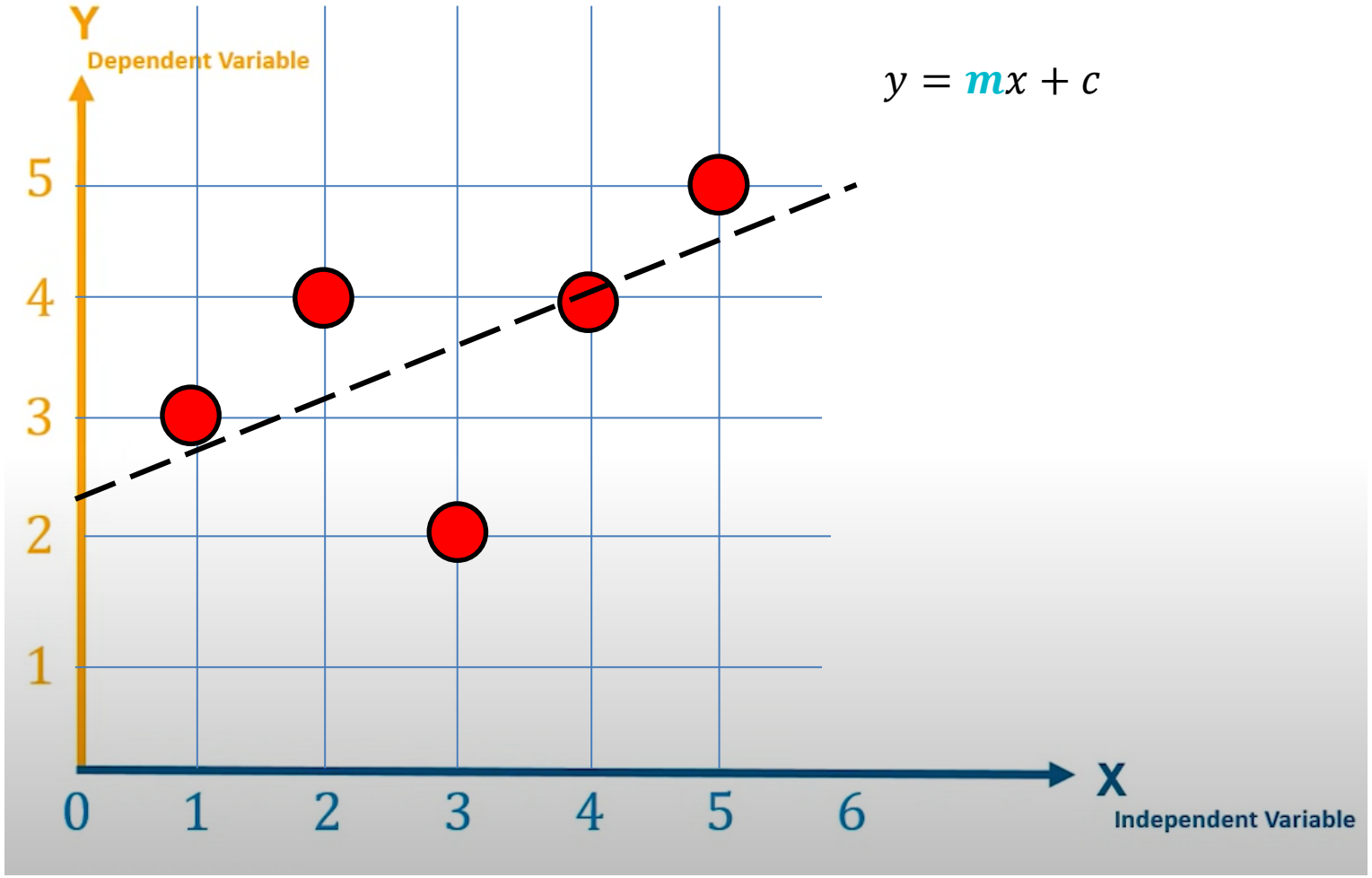
Best fit regression line

Distance between the data points and the line is the least.

D=

The regression line (blue) has the least value of D

Understanding Linear Regression



$$y = mx + c$$

x	y
1	3
2	4
3	2
4	4
5	5

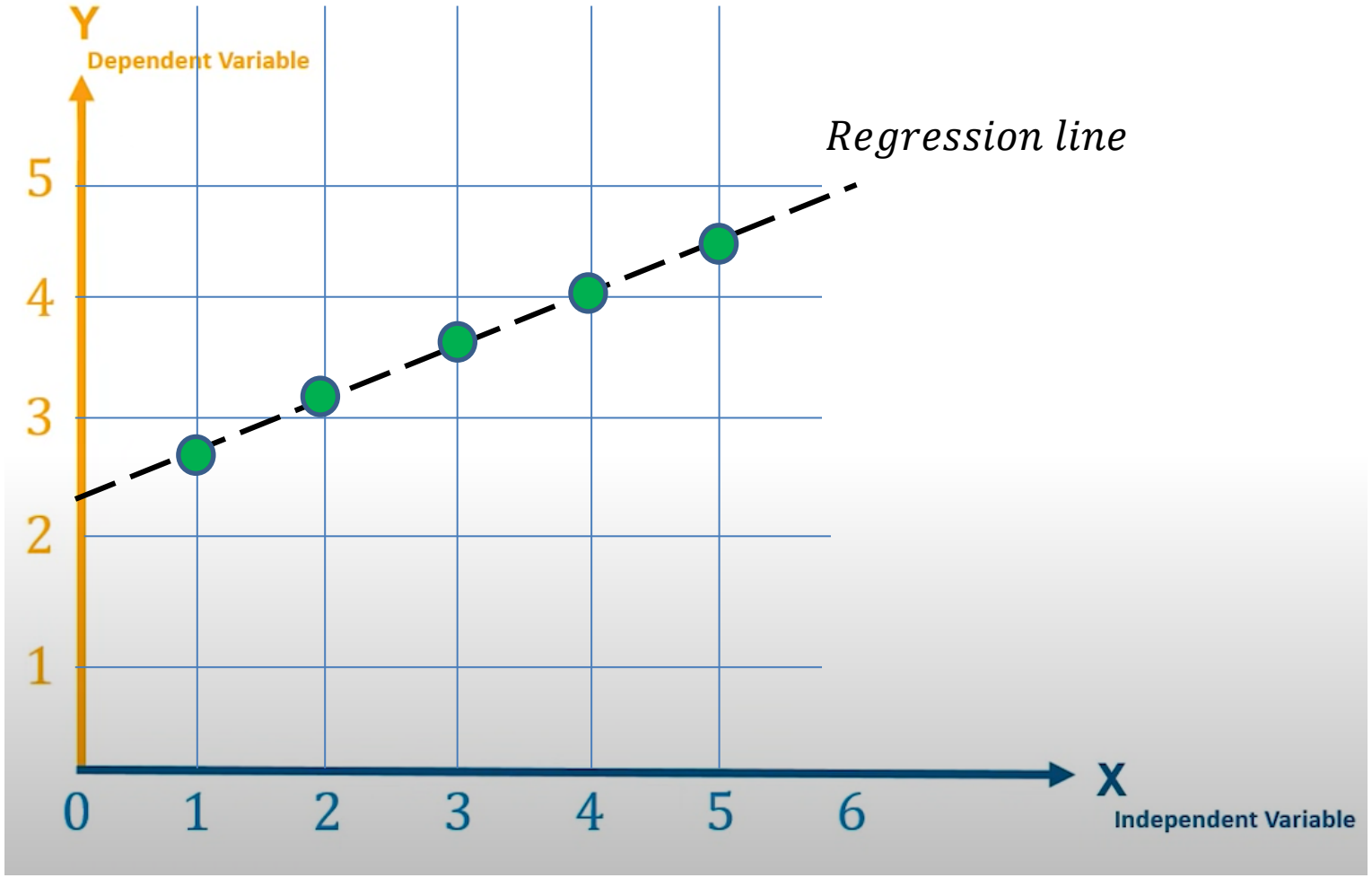
$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2} = 0.4$$

$\bar{x} =$ Mean of x

$\bar{y} =$ Mean of y

What is R-square

- **R-squared** value is a statistical measure of how close the data are to the fitted regression line.
- It is also known as the **coefficient of determination**, or the **coefficient of multiple determination.**



x	y
1	3
2	4
3	2
4	4
5	5

x	y_p
1	2.8
2	3.2
3	3.6
4	4.0
5	4.4

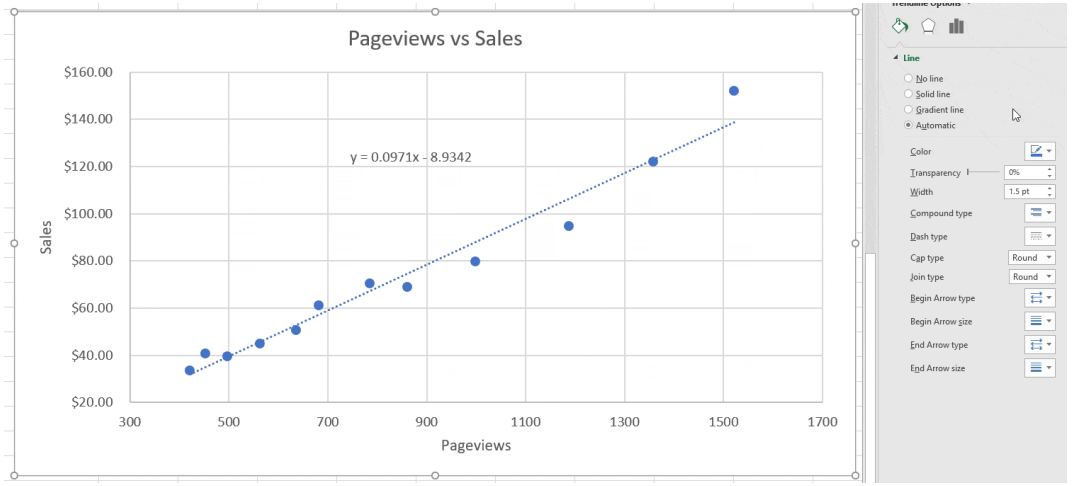
$$R^2 = \frac{\sum(y_p - \bar{y})^2}{\sum(y - \bar{y})^2}$$

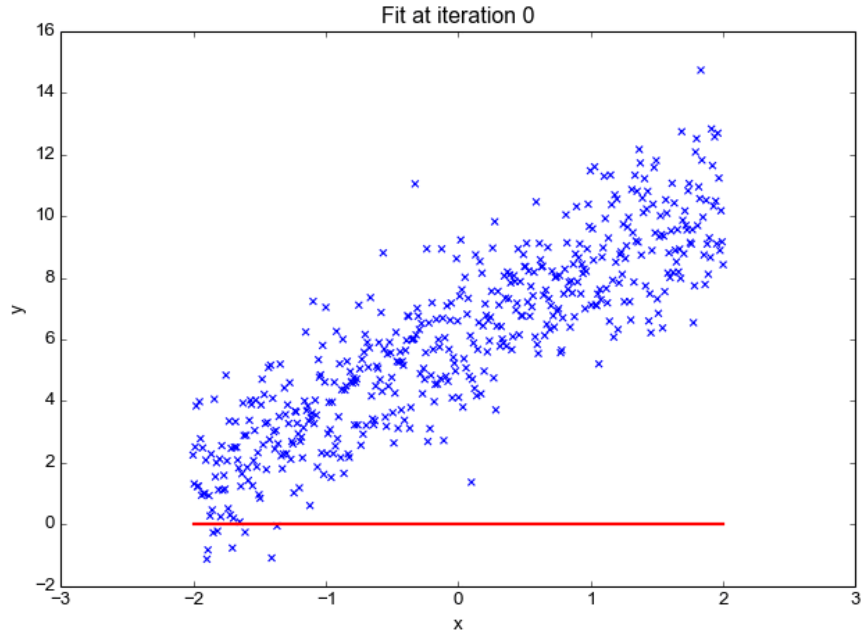
$$R^2 = 1$$

Very good linear regression

Application of Linear Regression

- Evaluating Trends and Sales Estimates.
- Analyzing the Impact of Price Changes.
- Assessment of risk in financial services and insurance domain.





Implementation of Linear Regression

- **Linear Regression:** Predict Employee Salary based on Years of Experience.

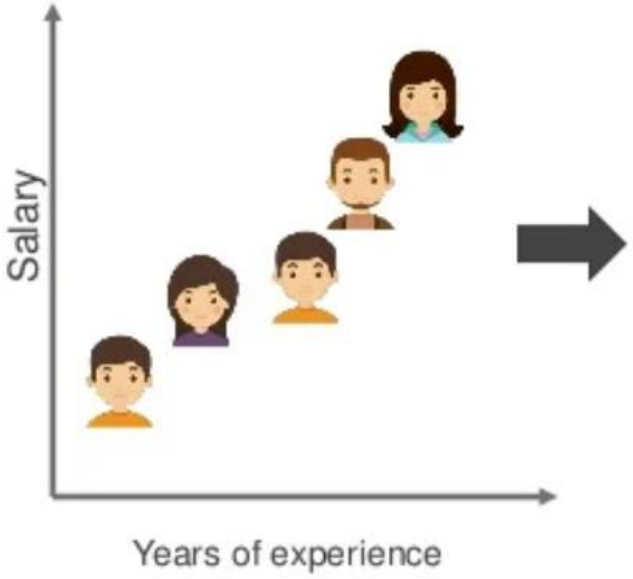


New Joinee

How much do we pay her?



HR team trying to figure out how much to pay to a new joinee?



Employee data of Salary based on years of experience



Predict Salary

https://www.kaggle.com/datasets/rohankayan/years-of-experience-and-salary-dataset

Implementation of Linear Regression

In [1]:
```
# Step-1: Load the relevent Libraries:

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

In [2]:
```
# Step-2: Import the Dataset
salary_data=pd.read_csv("C:/Users/itsme_000/Desktop/HKU
x = salary_data.iloc[:,:-1].values
y = salary_data.iloc[:,1].values
```

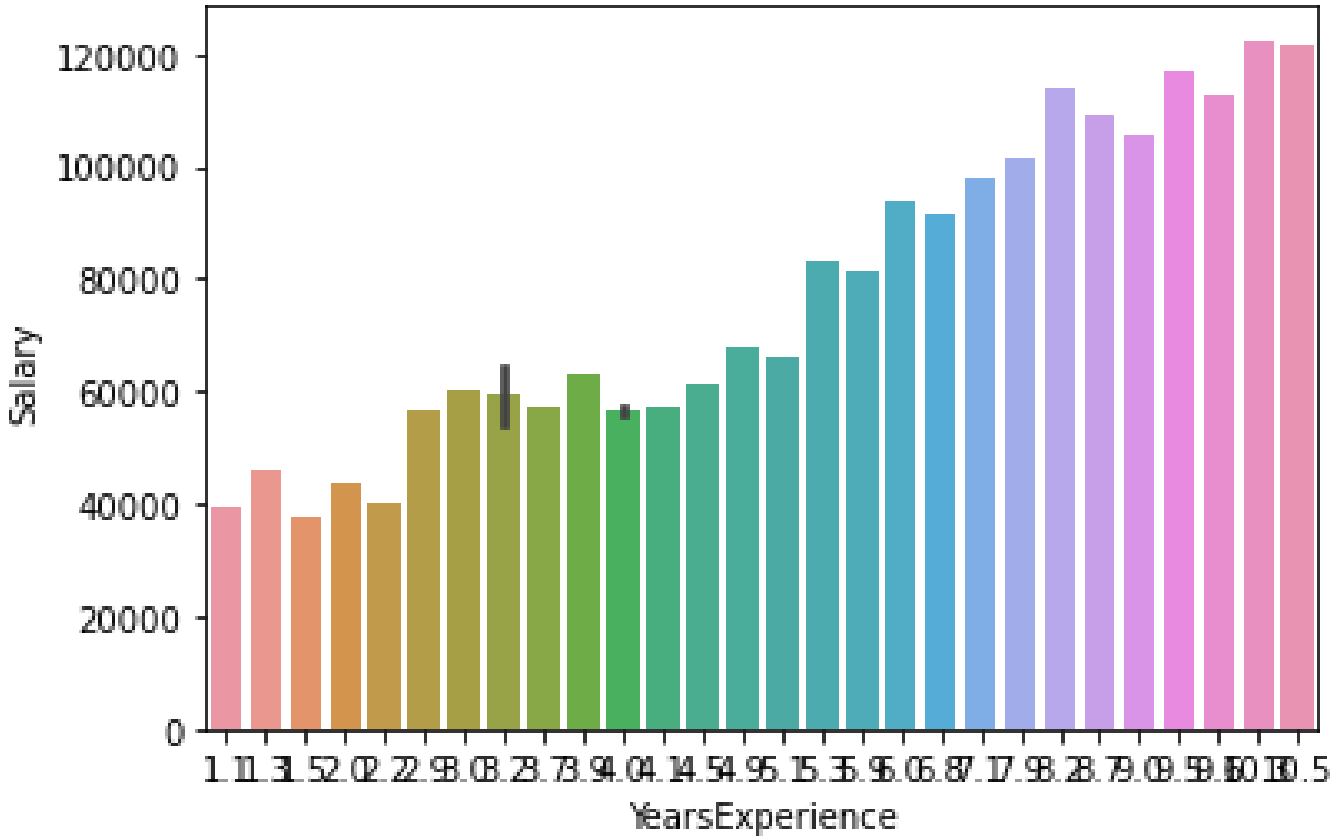
In [3]:
```
# Step-3: Visualize the dataset
plt.plot(x, y,'o')
plt.xlabel('Years of Experience')
plt.ylabel('Salary!')
plt.title('Salary vs Experience')
```



Implementation of Linear Regression

```
In [4]: # Step-3: Visualize the dataset
        sns.barplot(x='YearsExperience', y='Salary', data=salary_data)
```

```
Out[4]: <AxesSubplot:xlabel='YearsExperience', ylabel='Salary'>
```



Implementation of Linear Regression

Split the dataset into training and test dataset using sklearn

In [5]:
```
# Step-4:Split the data into training and testing set
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=1/3,random_state=0)
```

In [6]:
```
# Step-5: Fit Simple Linear Regression to the training dataset
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(x_train,y_train)
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1, normalize=False)
```

Out[6]:
```
LinearRegression(n_jobs=1, normalize=False)
```

In [7]:
```
# Step-6:Predict the test set results
y_pred = lr.predict(x_test)

y_pred
```

Out[7]:
```
array([ 40835.10590871, 123079.39940819,  65134.55626083,  63265.36777221,
       115602.64545369, 108125.8914992 , 116537.23969801,  64199.96201652,
        76349.68719258, 100649.1375447 ])
```

Implementation of Linear Regression

```
# Step-7: Visualize the train set results:
plt.scatter(x_train,y_train,color='blue')
plt.plot(x_train,lr.predict(x_train),color='red')
plt.title('Salary~Experience (Train set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```




```
# Step-8: Visualize the test set results:
plt.scatter(x_test,y_test,color='blue')
plt.plot(x_train,lr.predict(x_train),color='red')
plt.title('Salary~Experience (Test set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```



Implementation of Linear Regression

```
# Step 9 : Calculting the residuals:
from sklearn import metrics
print('MAE:', metrics.mean_absolute_error(y_test,y_pred))
print('MSE:', metrics.mean_squared_error(y_test,y_pred))
print('RMSE:',np.sqrt(metrics.mean_absolute_error(y_test,y_pred)))
```

```
MAE: 3426.4269374307123
MSE: 21026037.329511296
RMSE: 58.535689433291l4
```

$$MAE = \frac{\sum_{i=1}^{N} \left| y_{test_i} - y_{pred_i} \right|}{N}$$

$$MSE = \frac{\sum_{i=1}^{N} \left(y_{test_i} - y_{pred_i} \right)^2}{N}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N} \left(y_{test_i} - y_{pred_i} \right)^2}{N}}$$

- MSE values closer to zero consider better.
- $RMSE$ between 0.2 and 0.5 consider reasonable and the model can relatively predict the data accurately. However, close the score is to 0 the better the performing model is.

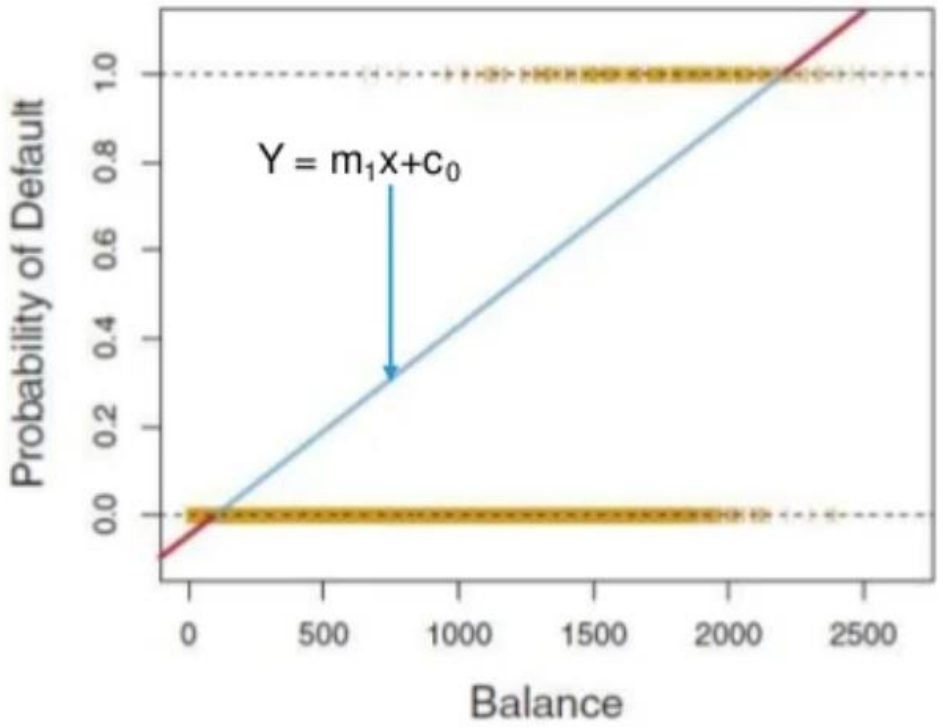
Logistic Regression

Logistic Regression is a classification algorithm that produces results in a binary format which is used to predict the outcome of a categorical dependent variable. So, the outcome should be discrete/categorical such as

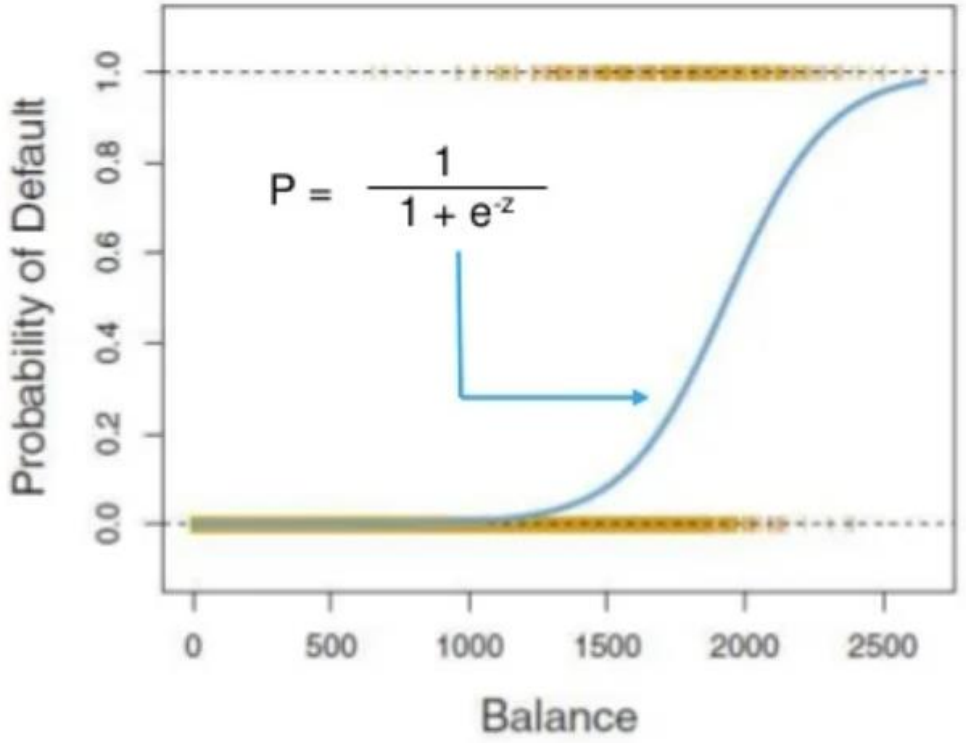


Logistic Regression

Plotting the Logistic Regression Curve: The logistic regression curve is known as the sigmoid curve (S curve)



Predicted Y can exceed the range 0 and 1



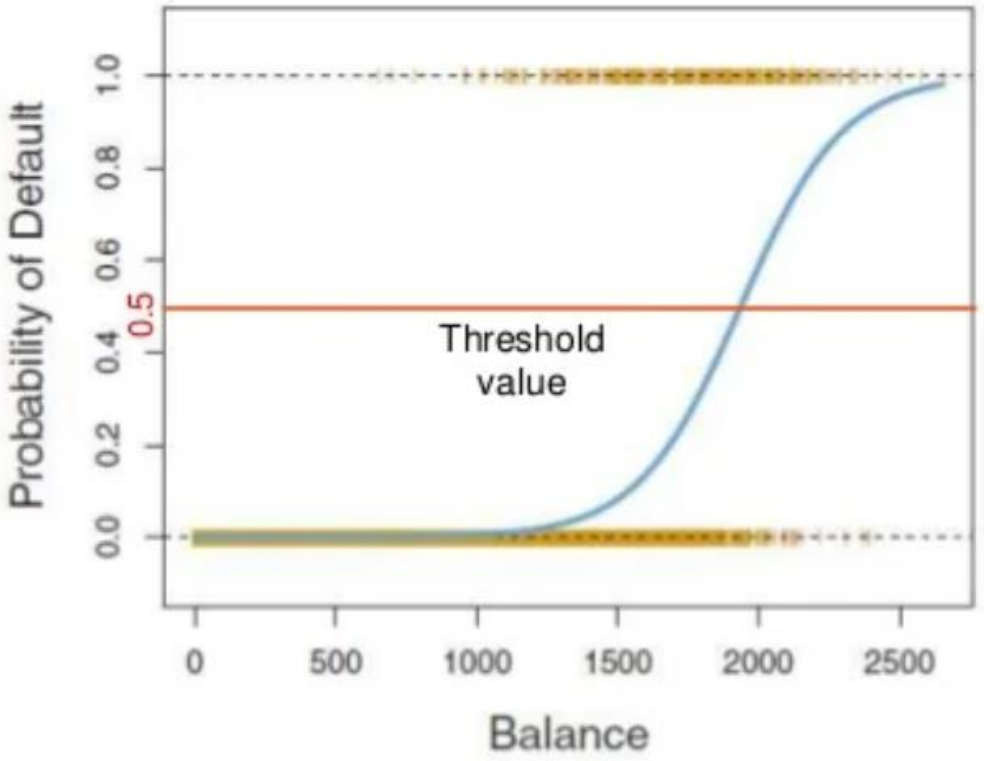
Predicted Y lies in the range 0 and 1

$$z = \theta x$$

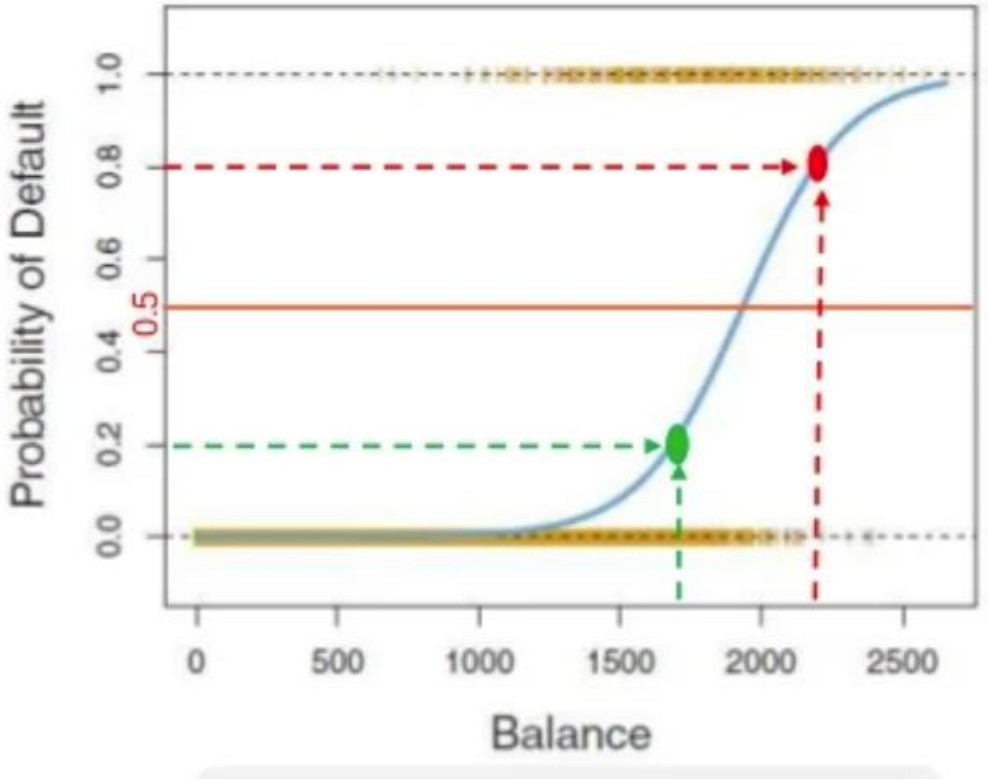
$$z = \theta_1 x_1 + \theta_2 x_2$$

Logistic Regression

Plotting the Logistic Regression Curve: The logistic regression curve is known as the sigmoid curve (S curve)



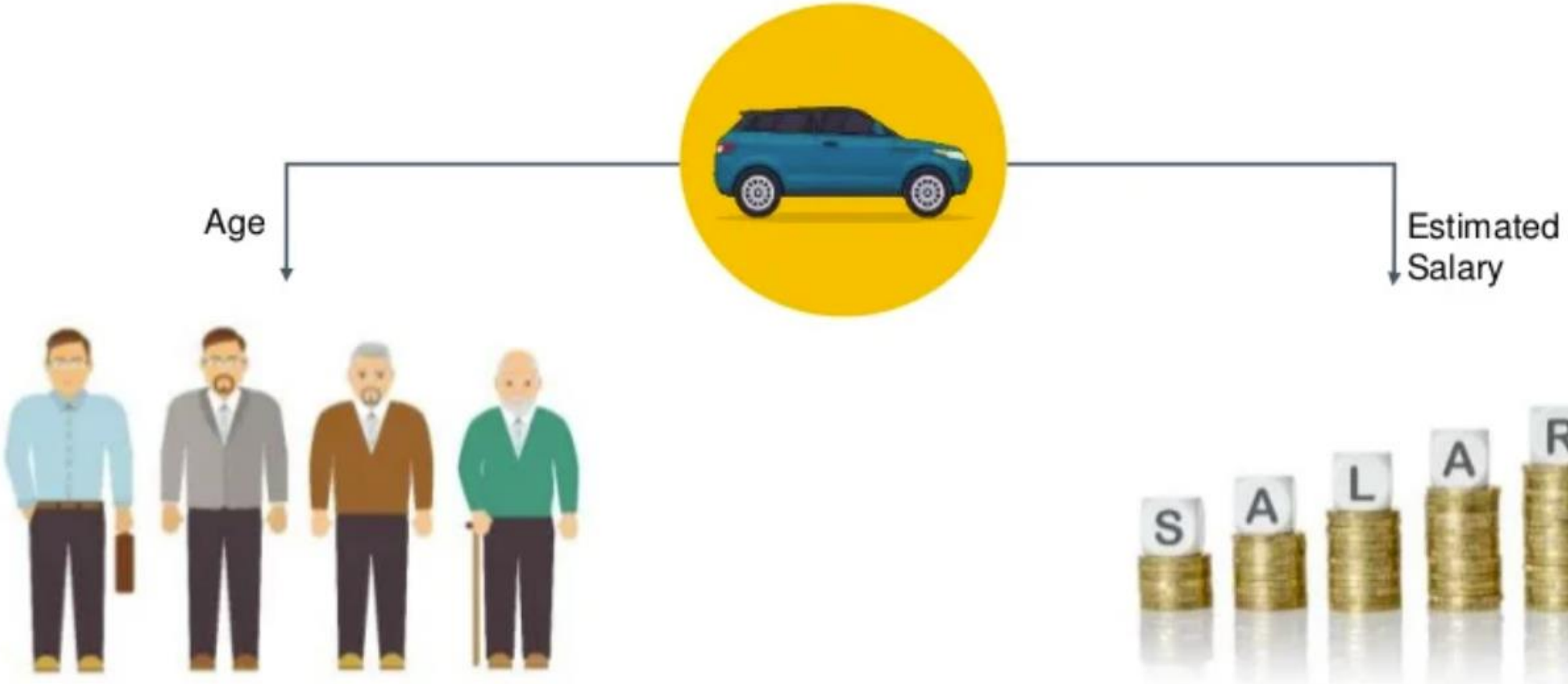
Cutoff point at 0.5, anything below it results in 0 and above is 1



The red data point will be default as it is above the threshold value of 0.5 and the green data point won't as it is below the threshold value

Implementation of Logistic Regression

- **Logistic Regression:** Predict if a person will buy a car based on their age and estimated salary.



https://www.kaggle.com/general/21221

Implementation of Logistic Regression

Step-1: Load the relevant libraries

In [1]:
```
# Step-1: Load the relevent Libraries:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Step-2: Import the dataset and extract the independent and dependent variables

In [2]:
```
# Step-2: Import the Dataset & extract the independent and dependent variables:
social_network=pd.read_csv("C:/Users/itsme_000/Desktop/HKU/Teaching/IDAT 7215 Computer Programming/

social_network.head(5)
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

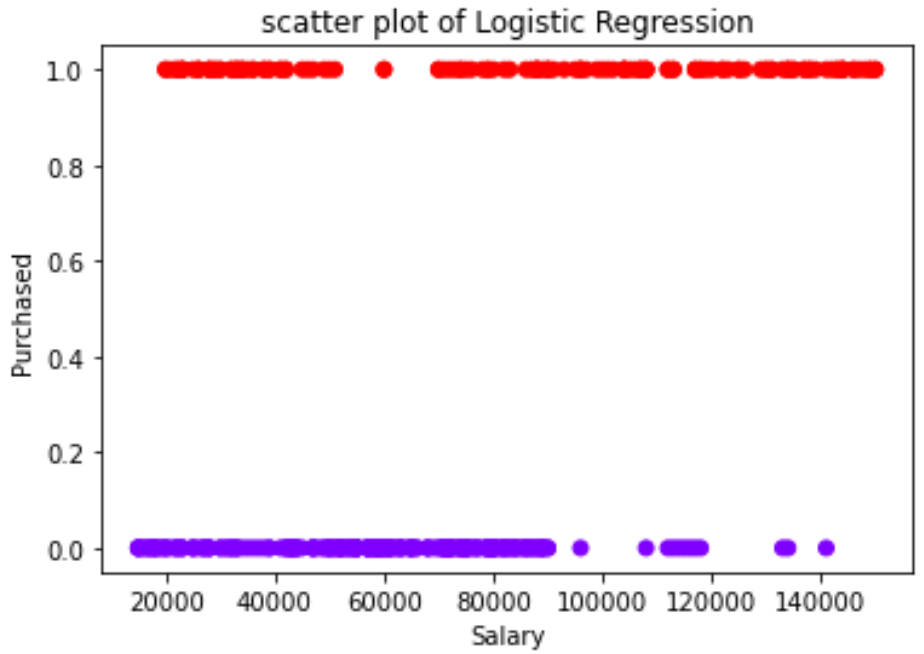
In [3]:
```
x = social_network.iloc[:,[2,3]].values  # Extracting the independent variables  (age, salary)
y = social_network.iloc[:,4].values      # Extracting the dependent variables    (purchased)
```

Implementation of Logistic Regression

Step-3: Visualize the dataset

```
In [4]: # Step-3: Visualize the dataset:
        # create scatter plot
        plt.scatter(x[:,1],y,c=y, cmap='rainbow')
        plt.xlabel('Salary')
        plt.ylabel('Purchased')
        plt.title('scatter plot of Logistic Regression')
        plt.show
```

```
Out[4]: <function matplotlib.pyplot.show(close=None, block=None)>
```



Implementation of Logistic Regression

Step-4: Split the dataset into Training and Testing Set

```
In [6]: # Step-4: Split the dataset into Training and Testing set:
        from sklearn.model_selection import train_test_split
        x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=1/3,random_state=0)
```

Step-5: Feature Scaling

```
In [7]: # Step-5: Feature Scaling:
        from sklearn.preprocessing import StandardScaler
        sc_x=StandardScaler()
        x_train=sc_x.fit_transform(x_train)
        x_test=sc_x.transform(x_test)
```

Feature scaling is a method used to normalize the range of independent variables or features of data. In data processing, it is also known as data normalization and is generally performed during the data preprocessing step.

Implementation of Logistic Regression

Step-6: Fit Logistic Regression to Training dataset:

In [8]:
```
# Step 6: Fit Logistic Regression to Training dataset
from sklearn.linear_model import LogisticRegression
logR = LogisticRegression(random_state=0)
logR.fit(x_train, y_train)
```

Out[8]:
```
LogisticRegression(random_state=0)
```

Step-7: Predicting the Test set results

In [9]:
```
# Step 7: Predicting the Test set results:
y_pred = logR.predict(x_test)
y_pred
```

Out[9]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1,
       0, 1], dtype=int64)
```

Implementation of Logistic Regression

Step-8: Display the confusion matrix:

```
In [10]: # Step 8: Displacy the Confusion Matrix
         from sklearn.metrics import confusion_matrix
         cm=confusion_matrix(y_test, y_pred)
         cm

Out[10]: array([[79,  6],
                [11, 38]], dtype=int64)
```

A **confusion matrix**, also known as an error matrix, is a specific table layout that allows visualization of the performance of an algorithm.

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Accuracy:

$$\frac{(TP+TN)}{P+N} = \frac{(79+38)}{134} = 0.87$$

Misclassification Rate:

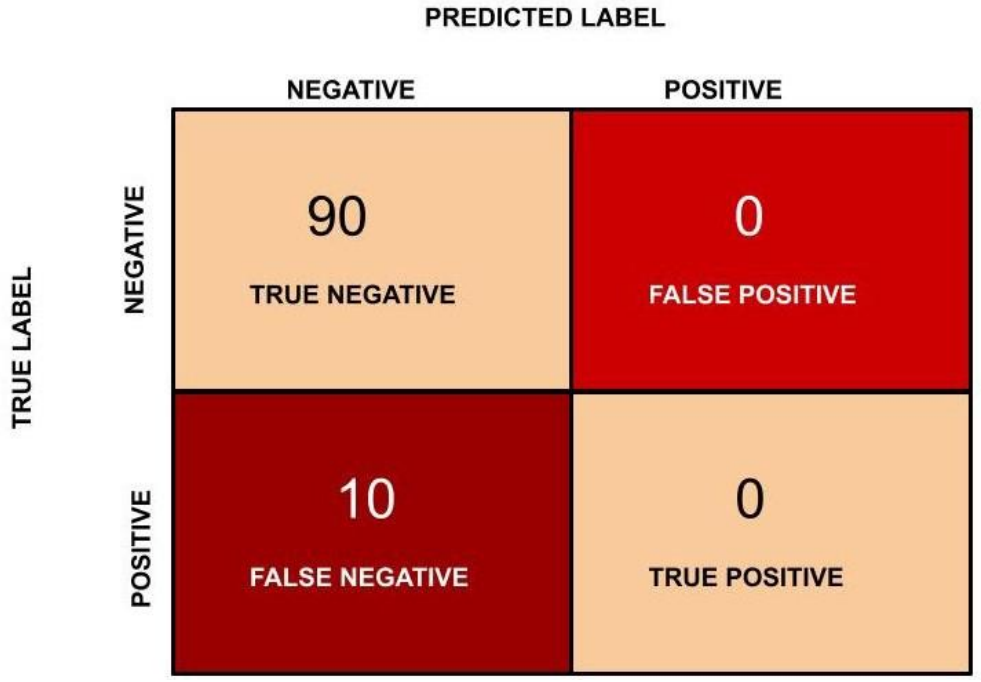
$$\frac{(FP+FN)}{P+N} = \frac{(11+6)}{134} = 0.13$$

Confusion Matrix

Is accuracy is the best measure?

Accuracy may not be a good measure if the dataset is not balanced (both negative and positive classes have different numbers of data instances). We will explain this with an example.

Consider the following scenario: There are 90 people who are healthy (negative) and 10 people who have some disease (positive). Now let's say our machine learning model perfectly classified the 90 people as healthy but it also classified the unhealthy people as healthy. What will happen in this scenario? Let us see the confusion matrix and find out the accuracy.



Confusion matrix for healthy vs unhealthy people classification task.

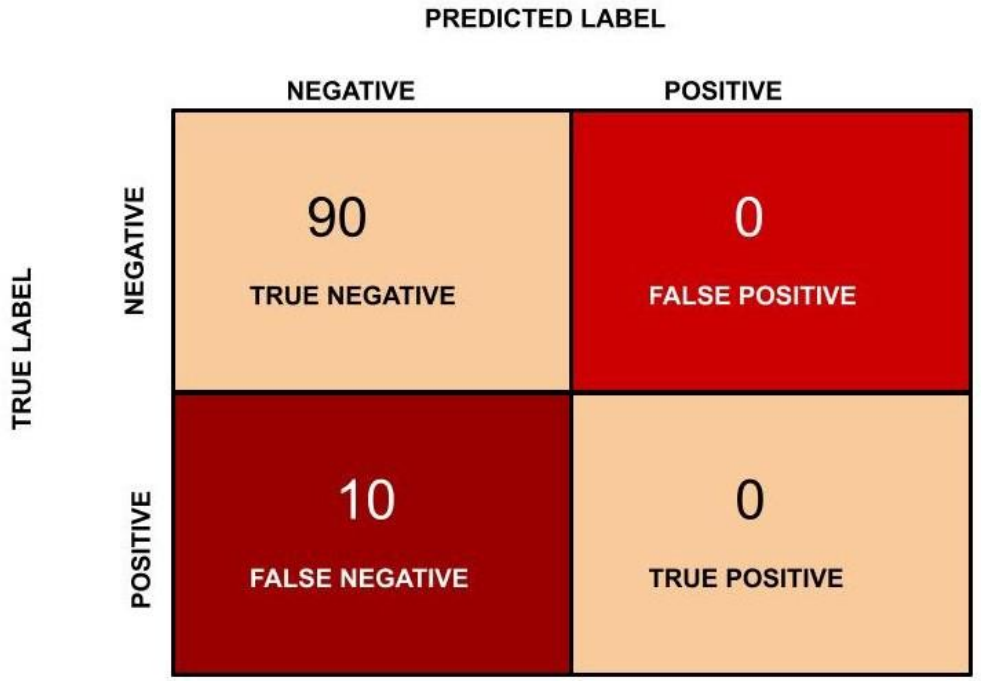
The accuracy, in this case, is 90 % but this model is very poor because all the 10 people who are unhealthy are classified as healthy.

Confusion Matrix Metrics

What does precision mean?

$$Precision = \frac{TP}{TP + FP}$$

Precision should ideally be 1 (high) for a good classifier. Precision becomes 1 only when the numerator and denominator are equal i.e TP = TP +FP, this also means FP is zero. As FP increases the value of the denominator becomes greater than the numerator and precision value decreases (which we don't want).



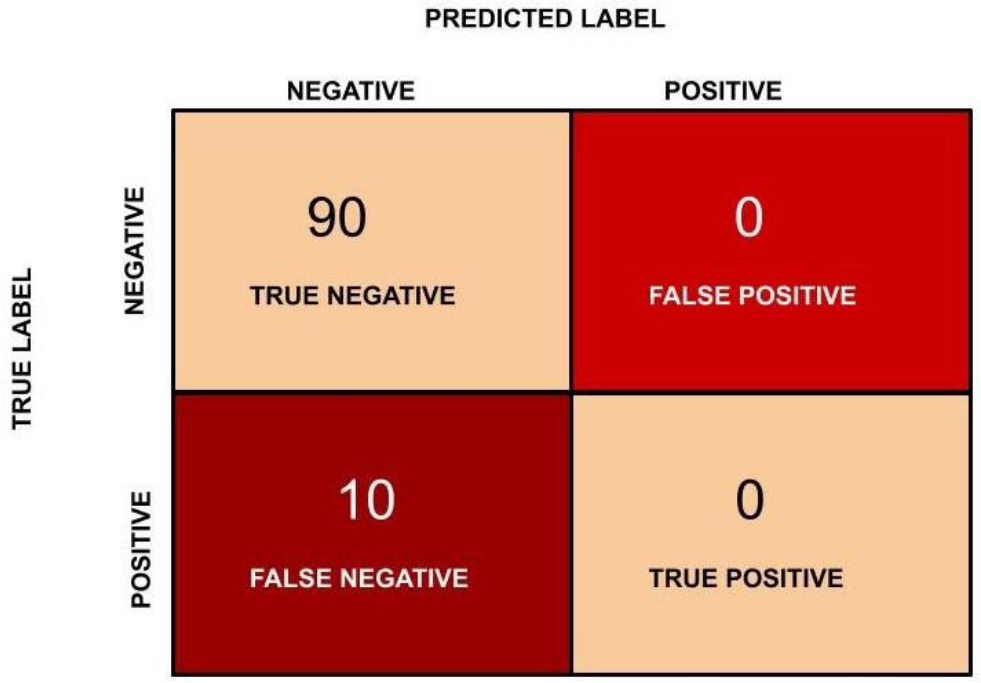
Confusion matrix for healthy vs unhealthy people classification task.

Confusion Matrix Metrics

What does recall mean?

Now we will introduce another important metric called ***recall***. ***Recall*** is also known as ***sensitivity*** or ***true positive rate*** and is defined as follows

$$Recall = \frac{TP}{TP + FN}$$



Confusion matrix for healthy vs unhealthy people classification task.

So ideally in a good classifier, we want both ***precision*** and ***recall*** to be one which also means ***FP*** and ***FN*** are zero. Therefore, we need a metric that takes into account both ***precision*** and ***recall***.

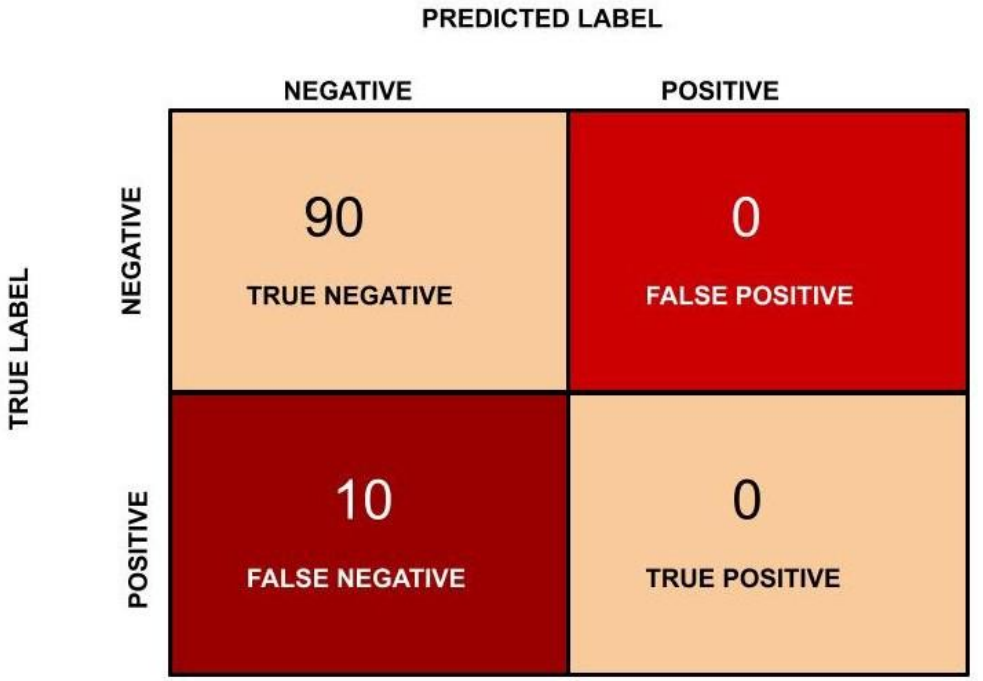
Confusion Matrix Metrics

What does F1 Score?

F1-score is a metric which takes into account both ***precision*** and ***recall*** and is defined as follows

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1 Score becomes 1 only when ***precision*** and ***recall*** are both 1. ***F1 score*** becomes high only when both ***precision*** and ***recall*** are high. ***F1 score*** is the harmonic mean of ***precision*** and ***recall*** and is a better measure than ***accuracy***.

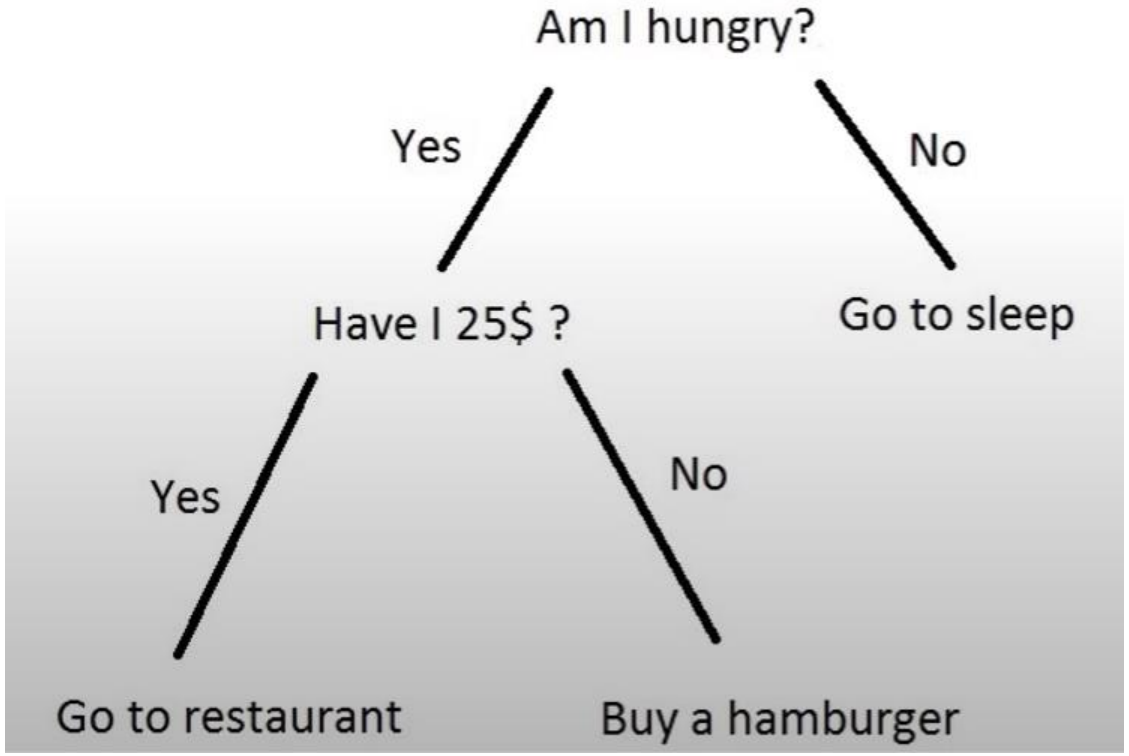


Confusion matrix for healthy vs unhealthy people classification task.

Decision Tree and Random Forest

Decision Tree

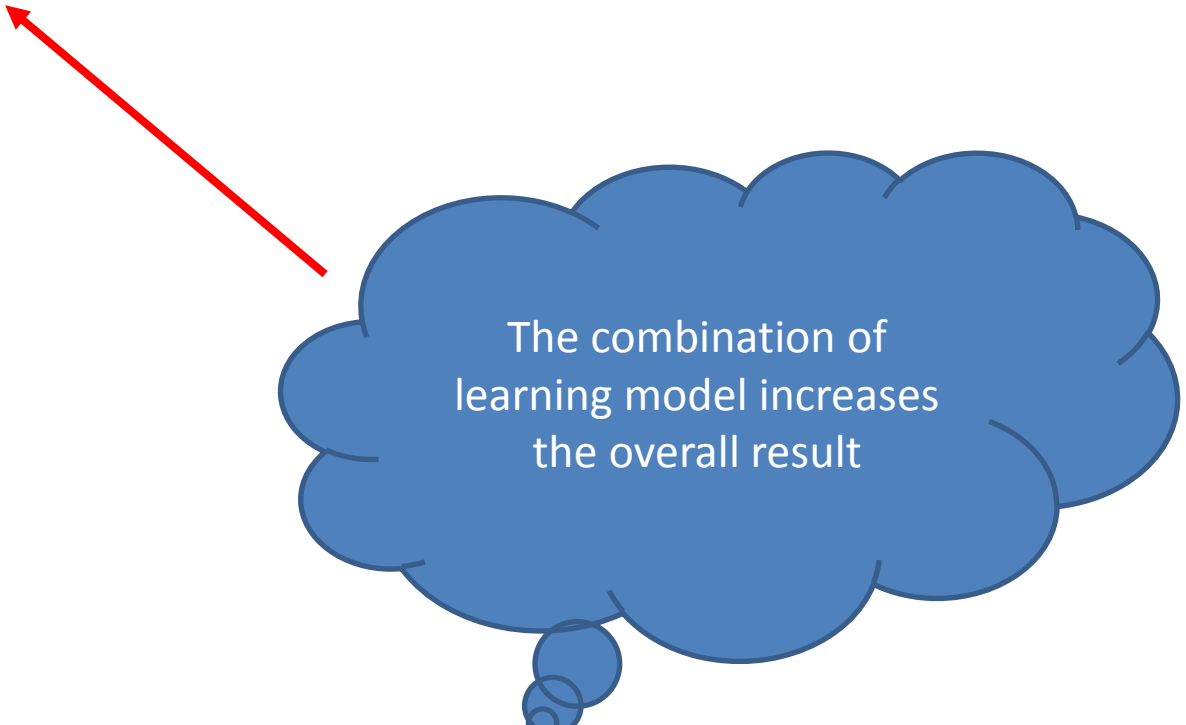
- Graphical representation of all the possible solutions to a decision.
- Decisions are based on some conditions.
- Decisions made can be easily explained.



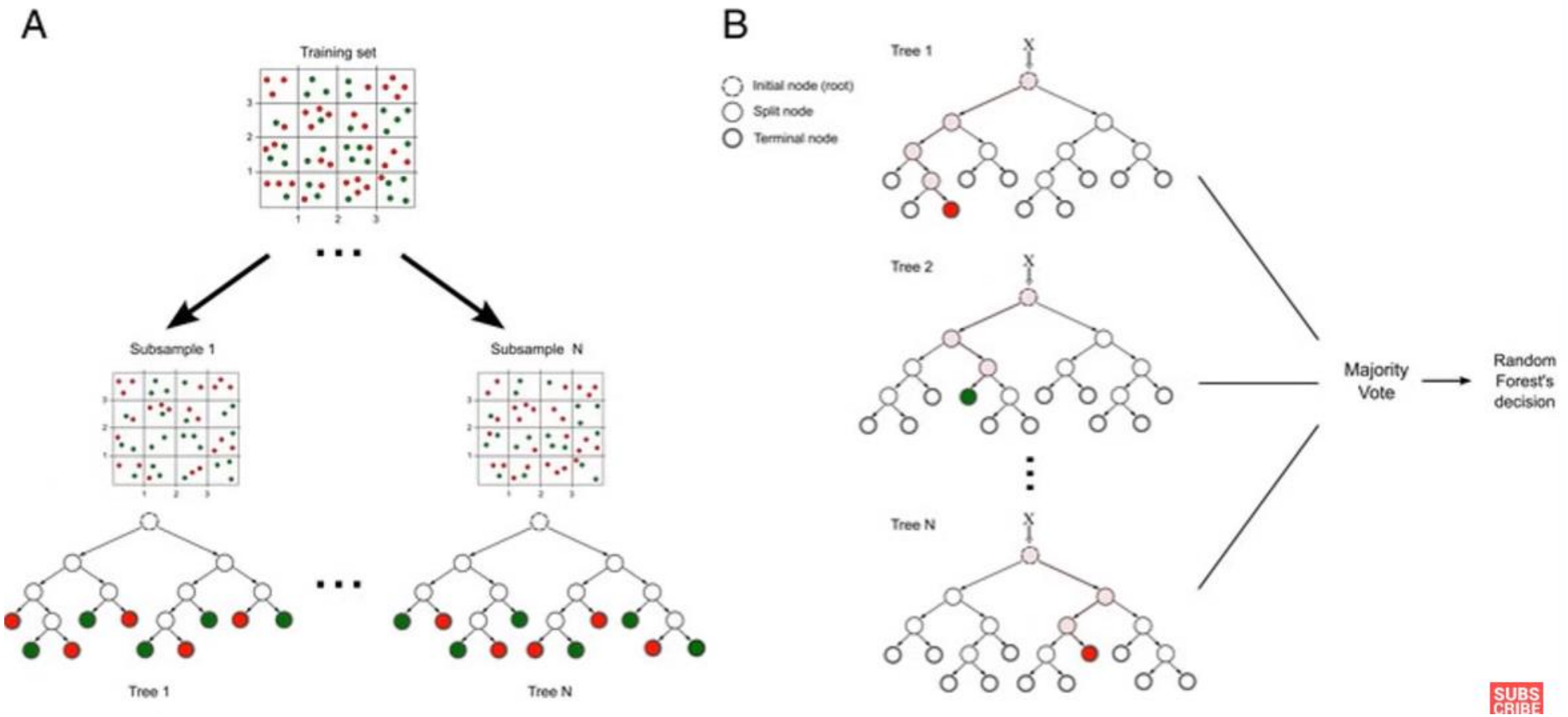
Should the person go to a restaurant?

Random Forest

- Builds multiple decision trees and merges them together.
- More accurate and stable prediction.
- Random decision forests correct for decision trees' habit of overfitting to their training set.
- Trained with the "bagging" method.



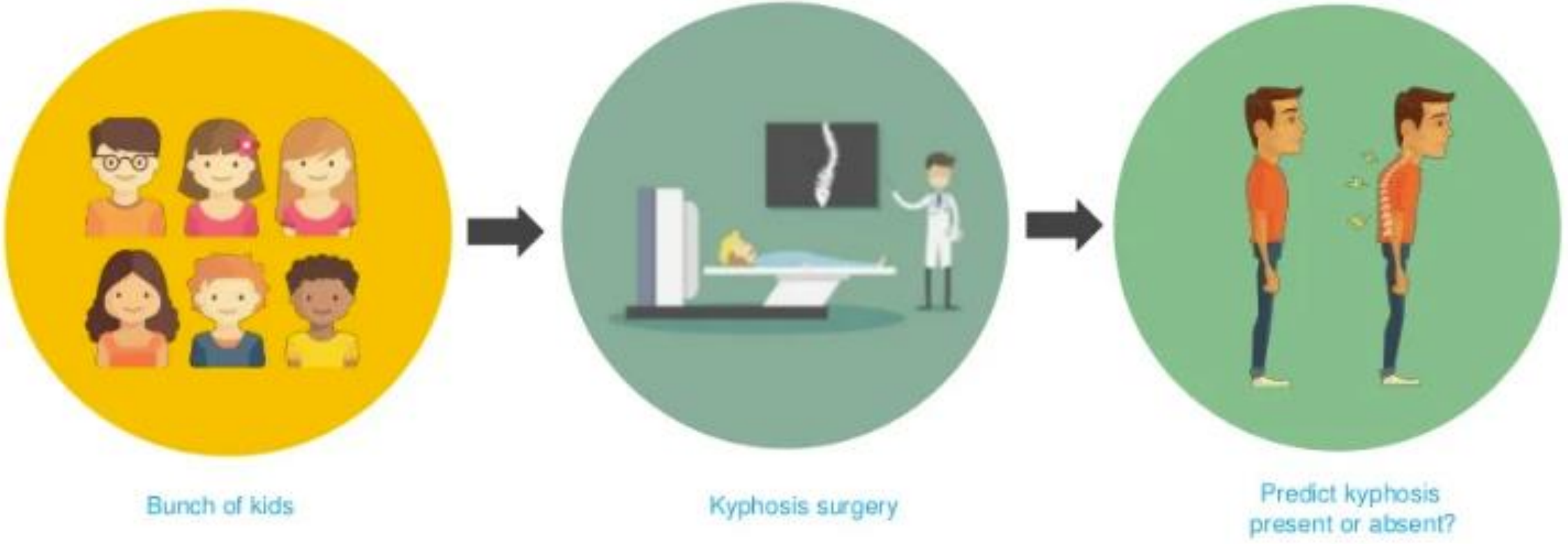
Random Forest



If the size of your dataset is huge then one single tree would lead to a overfit model

Implementation of Decision Tree

Decision Tree and Random Forest: Does Kyphosis exist after a surgery



https://www.kaggle.com/datasets/abbasit/kyphosis-dataset

Implementation of Decision Tree

Step-1: Load the relevant libraries

In [1]:
```
# Step-1: Load the relevent Libraries:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Step-2: Import the dataset and extract the independent and dependent variables

In [2]:
```
# Step-2: Importing the Dataset
Kyphosis=pd.read_csv("C:/Users/itsme_000/Desktop/HKU/Teaching/IDAT 7215 Computer Programming/Jupiter Notebbook/L
Kyphosis.head()
```

Out[2]:

	Kyphosis	Age	Number	Start
0	absent	71	3	5
1	absent	158	3	14
2	present	128	4	5
3	absent	2	5	1
4	absent	1	4	15

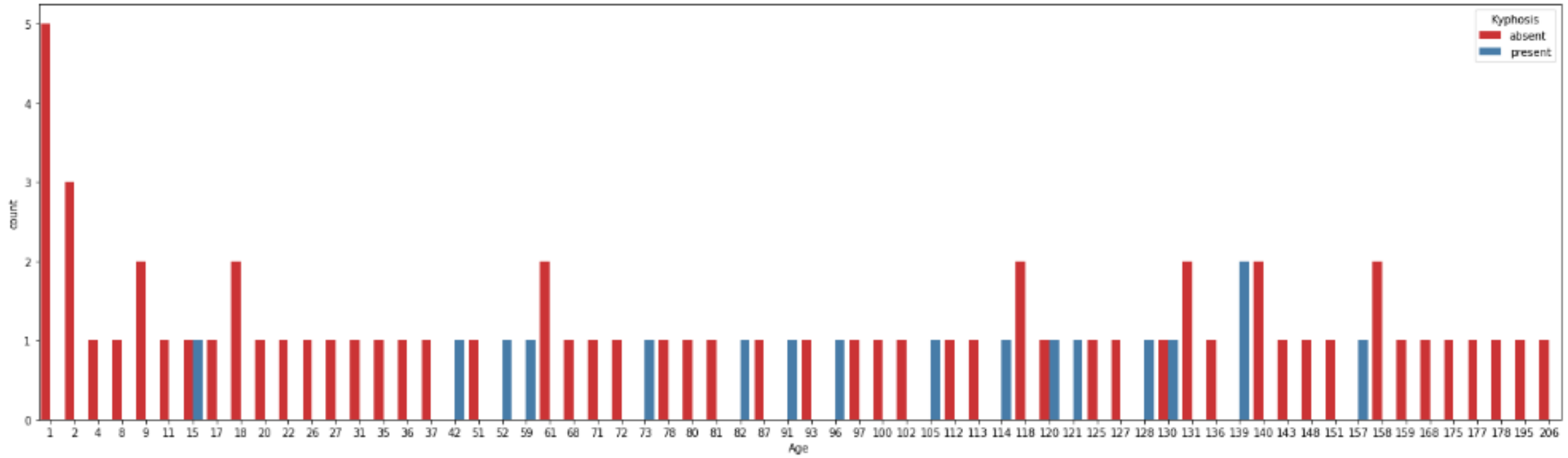
In [3]:
```
# Extracting Dependent and Independent Variables
x = Kyphosis.drop('Kyphosis',axis=1)  # Extracting the independent variables
y = Kyphosis['Kyphosis']              # Extracting the dependent variables
```

Implementation of Decision Tree

Step-3: Visualize the dataset

In [4]:
```
# Step 3: Visualize the datset
plt.figure(figsize=(25,7))
sns.countplot(x='Age', hue='Kyphosis', data=Kyphosis, palette='Set1')
```

Out[4]:
```
<AxesSubplot:xlabel='Age', ylabel='count'>
```



Implementation of Decision Tree

Step-4: Split the dataset into Training and Testing Set

In [5]:
```
# Step-4:Split the data into training and testing set
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.3,random_state=100)
```

Step-5: Train Decision Tree

In [6]:
```
# Step-5: Train Decision Tree
from sklearn.tree import DecisionTreeClassifier
dtree=DecisionTreeClassifier()
dtree.fit(x_train,y_train)
```

Out[6]:
```
DecisionTreeClassifier()
```

Implementation of Decision Tree

Step-6: Predict the model:

```
In [7]: # Step-6: Predict the Model
        y_pred = dtree.predict(x_test)
        y_pred

Out[7]: array(['absent', 'absent', 'present', 'absent', 'present', 'absent',
               'absent', 'absent', 'present', 'absent', 'absent', 'absent',
               'absent', 'absent', 'present', 'present', 'absent', 'absent',
               'present', 'present', 'absent', 'absent', 'present', 'absent',
               'present'], dtype=object)
```

Step-7: Display the confusion matrix

```
In [8]: # Step 7: Displacy the Confusion Matrix (To evaluate the model)
        from sklearn.metrics import classification_report,confusion_matrix
        print(classification_report(y_test,y_pred))

                      precision    recall  f1-score   support

              absent       0.88      0.64      0.74        22
             present       0.11      0.33      0.17         3

            accuracy                           0.60        25
           macro avg       0.49      0.48      0.45        25
        weighted avg       0.78      0.60      0.67        25
```

Accuracy:

$$\frac{(TP+TN)}{P+N} = \frac{(14+1)}{25} = 0.6$$

Misclassification Rate:

$$\frac{(FP+FN)}{P+N} = \frac{(2+8)}{25} = 0.4$$

```
In [9]: # Confusion matrix
        cm=confusion_matrix(y_test, y_pred)
        cm

Out[9]: array([[14,  8],
               [ 2,  1]], dtype=int64)
```

Implementation of Random Forest Tree

Step-8: Comparison with Random Forest

```
In [10]: # Step 9: Compare the model using Random Forest Tree
         from sklearn.ensemble import RandomForestClassifier
         rfc = RandomForestClassifier(n_estimators=100)
         rfc.fit(x_train,y_train)

Out[10]: RandomForestClassifier()

In [11]: rfc_pred=rfc.predict(x_test)

In [12]: # Confusion matrix
         cm_rfc=confusion_matrix(y_test, rfc_pred)
         cm_rfc

Out[12]: array([[19,  3],
                [ 3,  0]], dtype=int64)
```

Accuracy:

$$\frac{(TP+TN)}{P+N} = \frac{(19+0)}{25} = 0.76$$

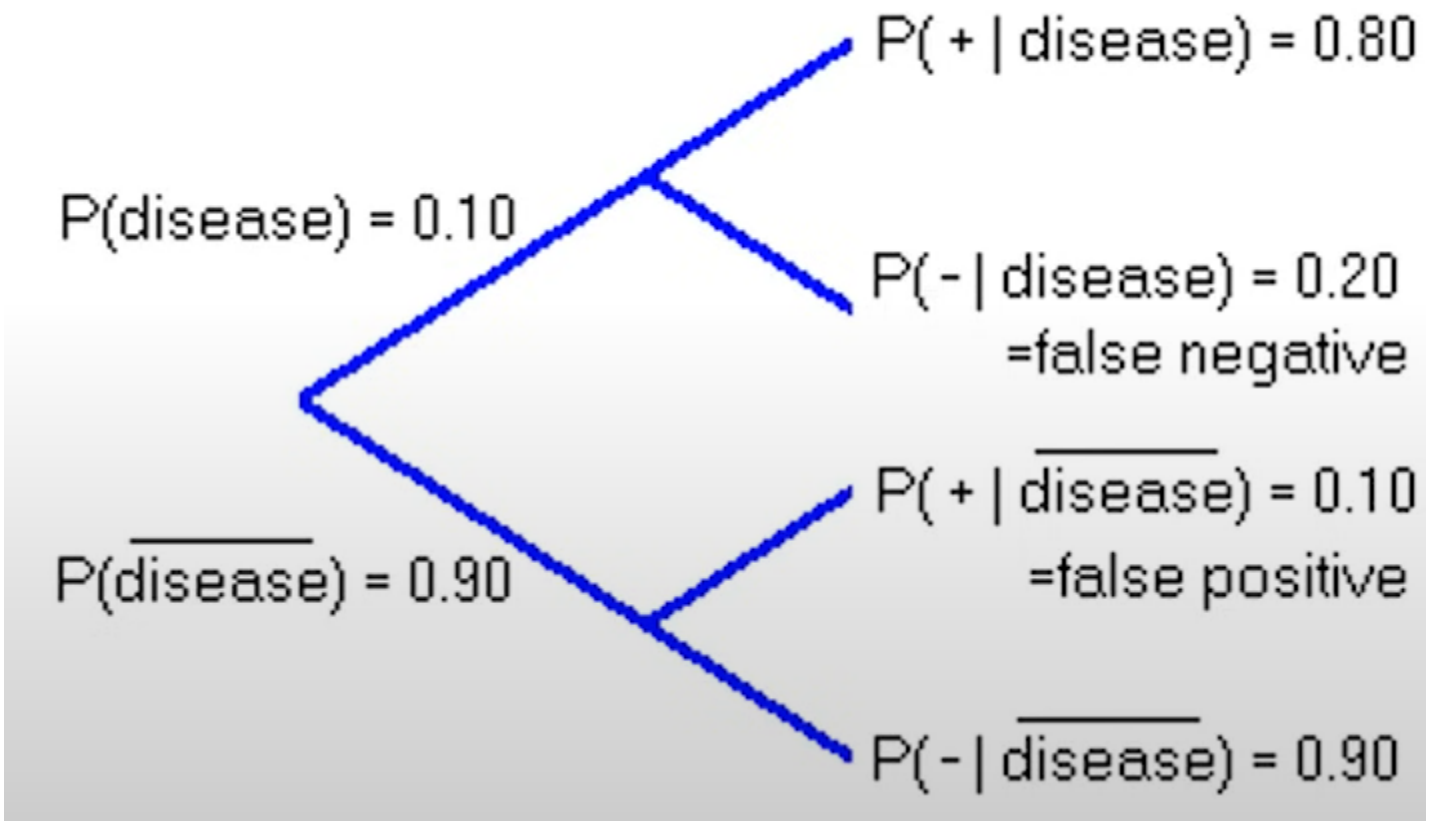
Misclassification Rate:

$$\frac{(FP+FN)}{P+N} = \frac{(3+3)}{25} = 0.24$$

Naïve Bayes

What is Naïve Bayes

Naïve Bayes is a simple but surprisingly powerful algorithm for predictive modeling. It is a classification technique based on the Bayes theorem.



What is Naïve Bayes

Naïve Bayes is a simple but surprisingly powerful algorithm for predictive modeling. It is a classification technique based on the Bayes theorem.

Bayes' Theorem

Given a hypothesis $\boldsymbol{H}$ and evidence $\boldsymbol{E}$, Bayes theorem states that the relationship between the probability of the hypothesis before getting the evidence $\mathbf{P}(\boldsymbol{H})$ and the probability of the hypothesis after getting the evidence $\mathbf{P}(\boldsymbol{H}|\boldsymbol{E})$ is

$$\mathbf{P}(\boldsymbol{H}|\boldsymbol{E}) = \frac{\mathbf{P}(\boldsymbol{E}|\boldsymbol{H}).\boldsymbol{P}(\boldsymbol{H})}{\mathbf{P}(\boldsymbol{E})}$$

Bayes' Theorem Example







$$P(King) = \frac{4}{52} = \frac{1}{13}$$

$$P(King|Face) = ?$$

Apply Bayes Theorem

$$P(King|Face) = \frac{P(Face|King).P(King)}{P(Face)}$$

A king, queen, or jack in a deck of cards is known as a face card.

$$P(Face|King) = 1$$

$$P(Face) = \frac{12}{52} = \frac{3}{13}$$

$$P(King|Face) = \frac{1.1/13}{3/13} = 1/3$$

Classification Steps

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

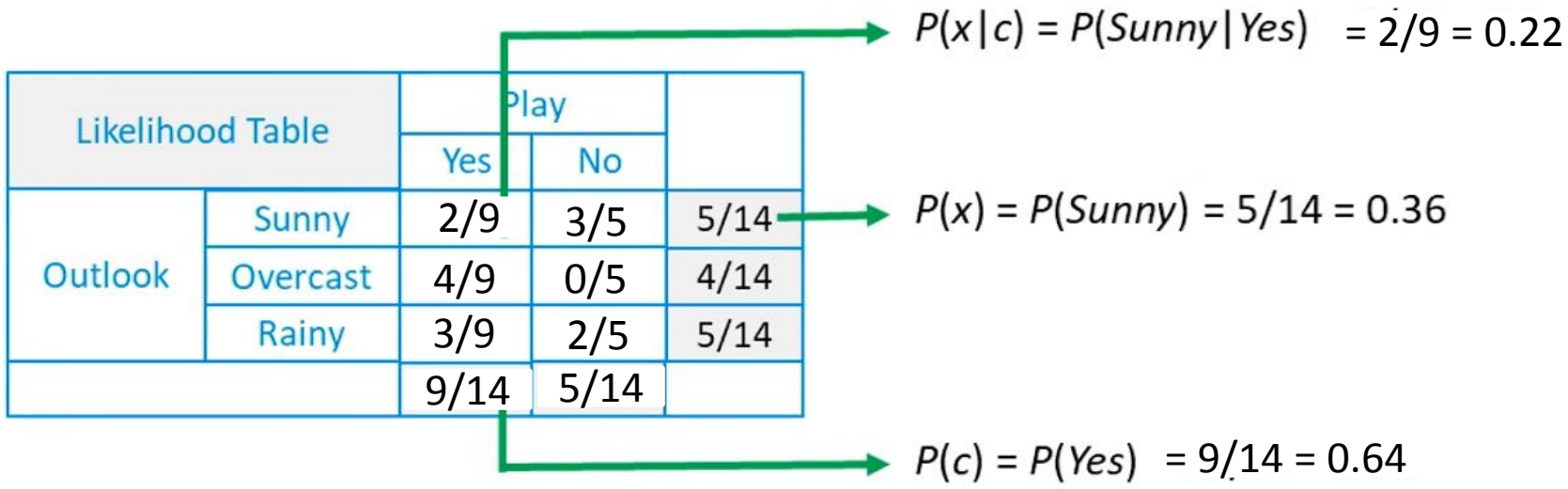


Frequency Table		Play	
		Yes	No
Outlook	Sunny	**2**	**3**
	Overcast	4	0
	Rainy	3	2

Frequency Table		Play	
		Yes	No
Humidity	High	3	4
	Normal	6	1

Frequency Table		Play	
		Yes	No
Wind	Strong	6	2
	Weak	3	3

Classification Steps



Likelihood of 'Yes' given Sunny is

$$P(c|x) = P(Yes|Sunny) = \frac{P(Sunny|Yes).P(Yes)}{P(Sunny)} = \frac{\frac{2}{9} \times \frac{9}{14}}{\frac{5}{14}} = 0.4$$

Likelihood of 'No' given Sunny is

$$P(c|x) = P(No|Sunny) = \frac{P(Sunny|No).P(No)}{P(Sunny)} = \frac{\frac{3}{5} \times \frac{5}{14}}{\frac{5}{14}} = 0.6$$

Classification Steps

Likelihood table for Humidity

Likelihood Table		Play		
		Yes	No	
Humidity	High	3/9	4/5	7/14
	Normal	6/9	1/5	7/14
		9/14	5/14	

Likelihood table for Wind

Likelihood Table		Play		
		Yes	No	
Wind	Weak	6/9	2/5	8/14
	Strong	3/9	3/5	6/14
		9/14	5/14	

$$P(Yes|High) = \frac{\frac{3}{9} \times \frac{9}{14}}{\frac{7}{14}} = 0.42$$

$$P(No|High) = \frac{\frac{4}{5} \times \frac{5}{14}}{\frac{7}{14}} = 0.58$$

$$P(Yes|Weak) = \frac{\frac{6}{9} \times \frac{9}{14}}{\frac{8}{14}} = 0.75$$

$$P(No|Weak) = \frac{\frac{2}{5} \times \frac{5}{14}}{\frac{8}{14}} = 0.25$$

Classification Steps

Suppose we have a day with the following values

Outlook	=	Rain
Humidity	=	High
Wind	=	Weak
Play	=	?

Likelihood of 'Yes' on that Day = $P(Outlook = Rain|Yes) \times P(Humidity = High|Yes) \times P(Wind = Weak|Yes) \times P(Yes)$

$$= \frac{3}{9} \times \frac{3}{9} \times \frac{6}{9} \times \frac{9}{14} = 0.048$$

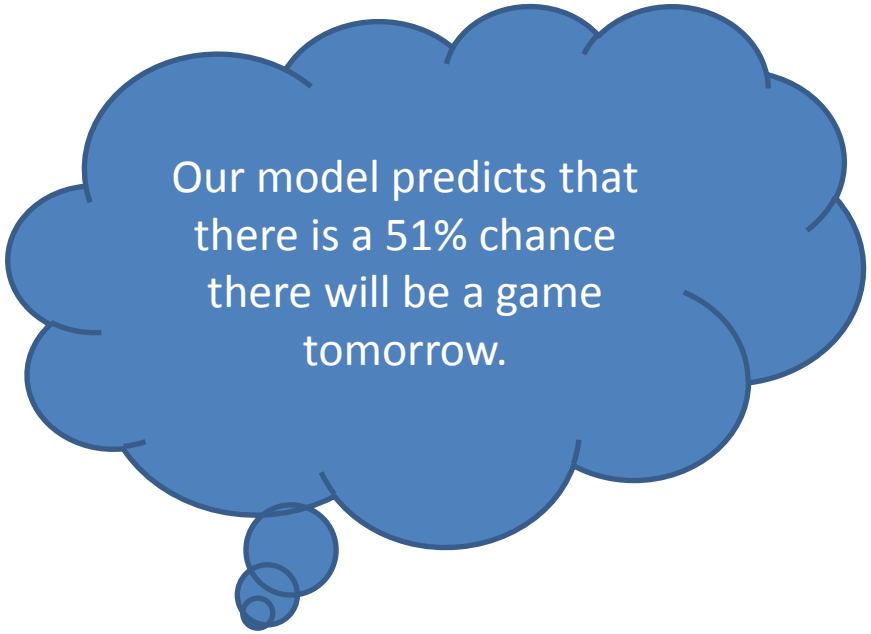
Likelihood of 'No' on that Day = $P(Outlook = Rain|No) \times P(Humidity = High|No) \times P(Wind = Weak|No) \times P(No)$

$$= \frac{2}{5} \times \frac{4}{5} \times \frac{2}{5} \times \frac{5}{14} = 0.046$$

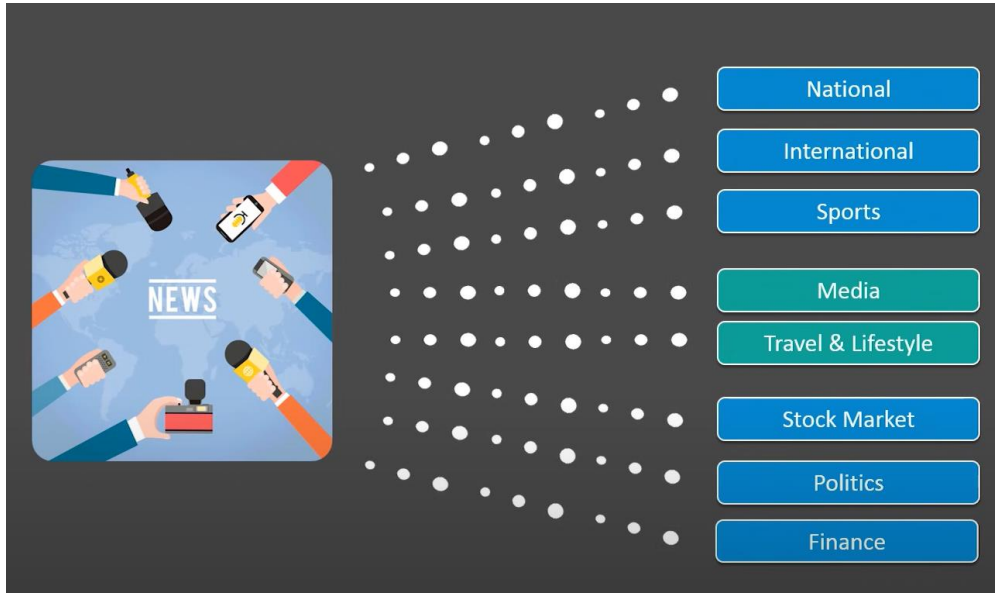
Classification Steps

$$P(Yes) = \frac{0.048}{(0.048 + 0.046)} = 0.51$$

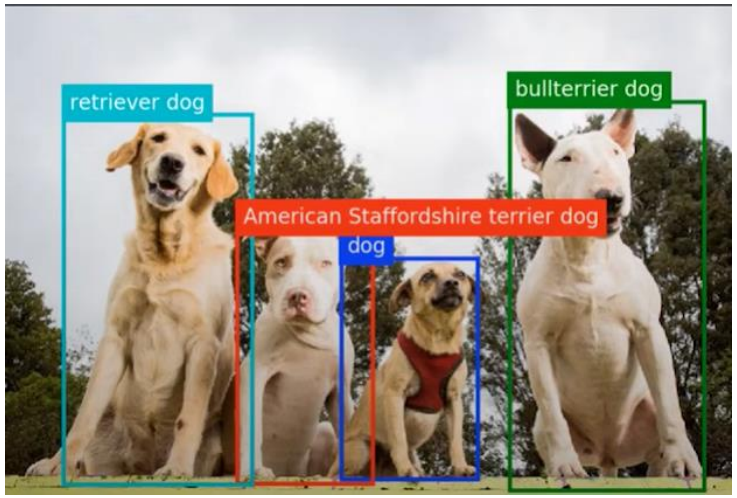
$$P(No) = \frac{0.046}{(0.048 + 0.046)} = 0.49$$



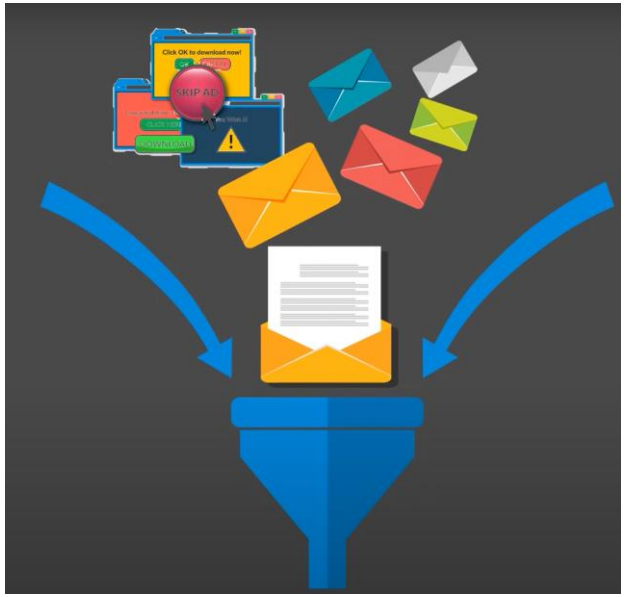
Applications of Naïve Base Algorithm



News Categorization



Object & Face Recognition



SPAM Filtering



Weather Prediction

Types of Naïve Bayes

1. **Gaussian**: It is used in classification and assumes that features follow a normal distribution.

2. **Multinomial**: It is used for discrete counts. For example, let's say, we have a text classification problem. Here we can consider Bernoulli trials which is one step further and instead of "word occurring in the document", we have "count how often word occurs in the document", you can think of it as "number of times outcome number x_i is observed over the n trials".

3. **Bernoulli**: The binomial model is useful if your feature vectors are binary (i.e. zeros and ones). One application would be text classification with 'bag of words' model where the 1s & 0s are "word occurs in the document" and "word does not occur in the document" respectively.

Implementation of Naïve Bayes

Load the library

```
In [1]: from sklearn import datasets
        from sklearn import metrics
        from sklearn.naive_bayes import GaussianNB
```

Load the IRIS Dataset

```
In [2]: dataset=datasets.load_iris()   # Loading the IRIS Datasets
```

Fit a Gaussian Naïve Base Model on IRIS Dataset

```
In [3]: model = GaussianNB()
        model.fit(dataset.data, dataset.target)
Out[3]: GaussianNB()

In [4]: print(model)

        GaussianNB()
```

Implementation of Naïve Bayes

Make Predictions

```
In [5]: expected = dataset.target
        predicted = model.predict(dataset.data)
```

```
In [6]: print(metrics.classification_report(expected, predicted))
        print(metrics.confusion_matrix(expected, predicted))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        50
           1       0.94      0.94      0.94        50
           2       0.94      0.94      0.94        50

    accuracy                           0.96       150
   macro avg       0.96      0.96      0.96       150
weighted avg       0.96      0.96      0.96       150

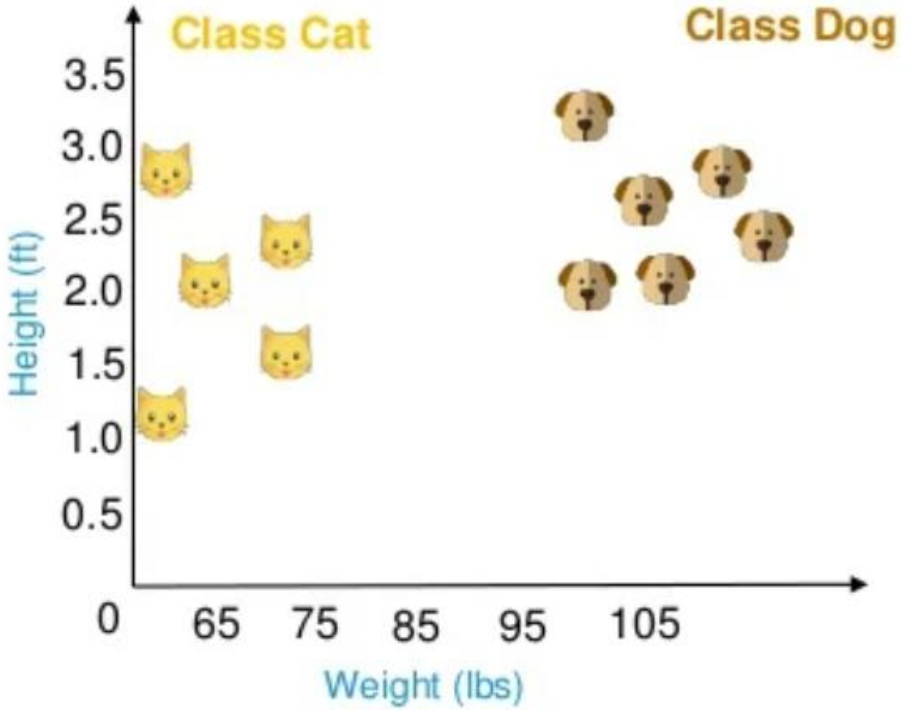
[[50  0  0]
 [ 0 47  3]
 [ 0  3 47]]
```

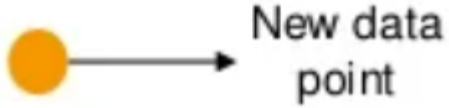
KNNs Algorithm

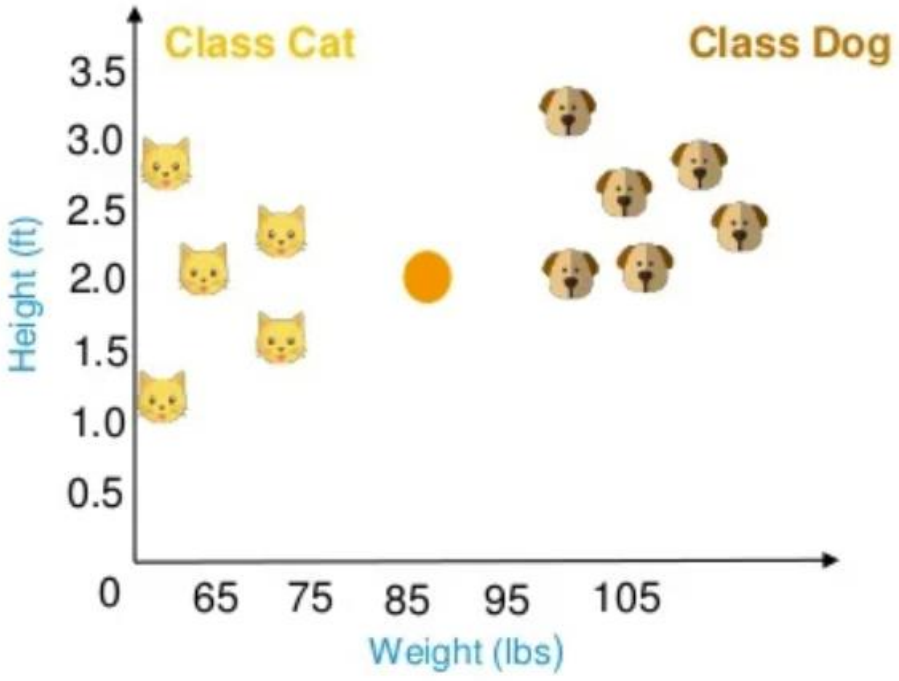
K Nearest Neighbors

- KNN is a classification algorithm generally used to predict categorical values.
- It stores all the available cases and classifies new cases based on a similarity measure.

To find if a new data is  

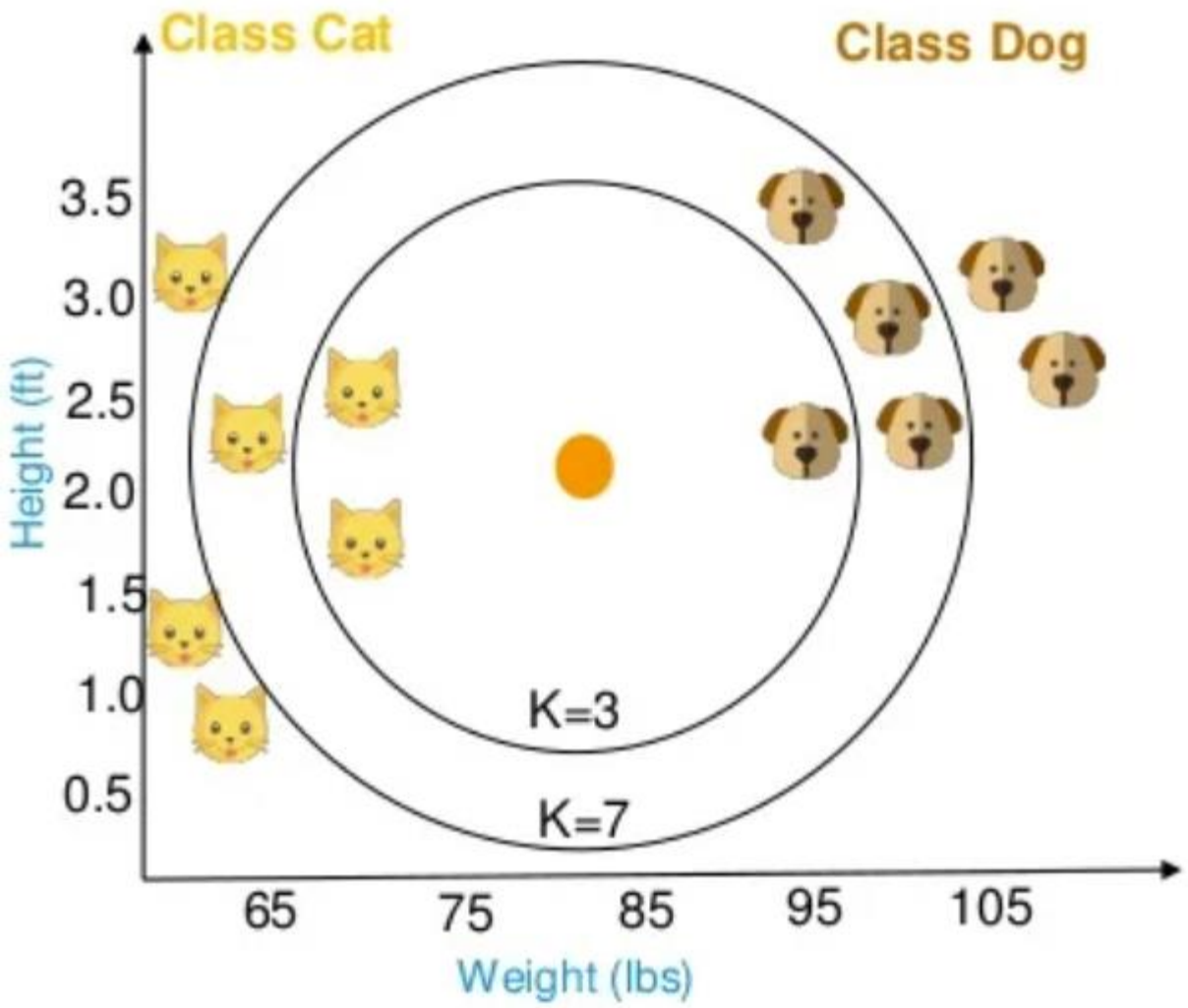






K Nearest Neighbors

- Choosing a K will define what class a new data point is assigned to.
- K is the number of nearest neighbors.

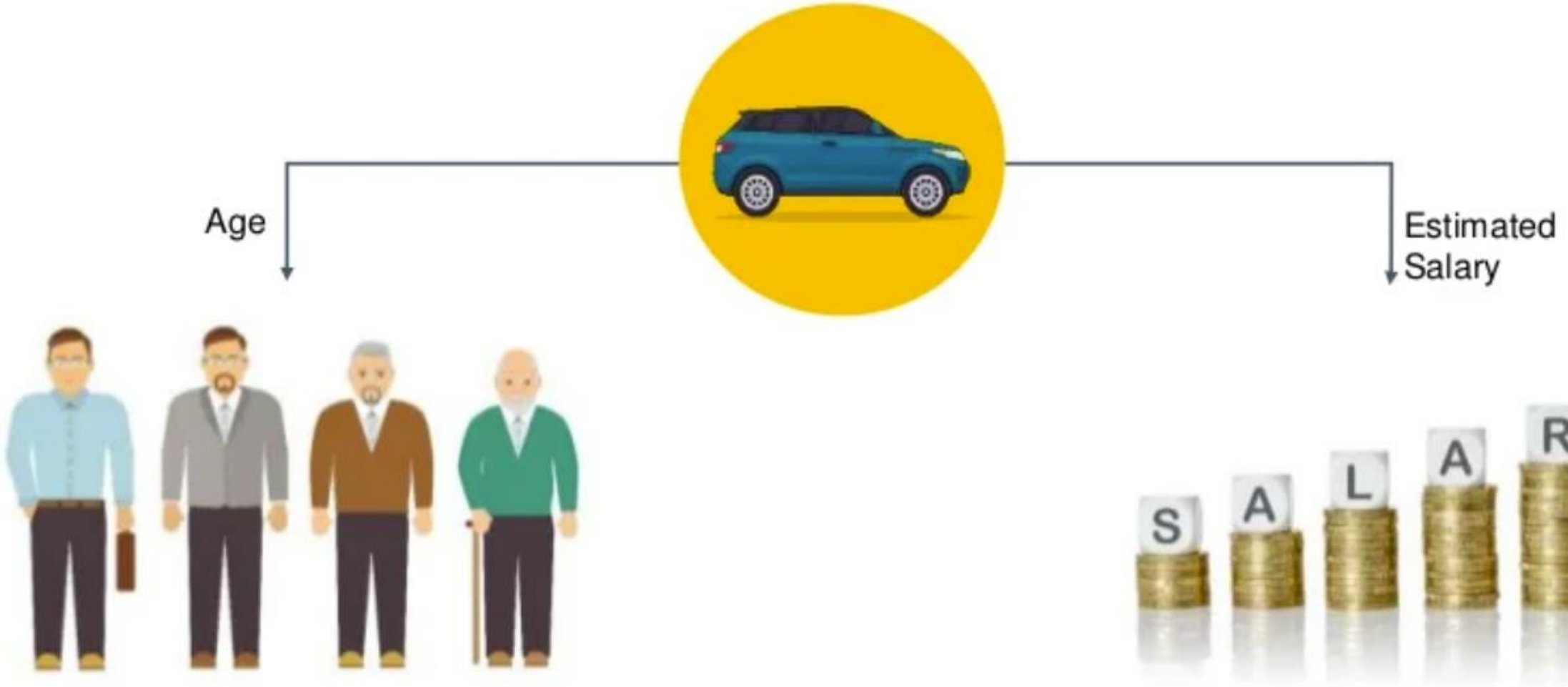


Calculate the least distance from new points

If K=3 the new data point belongs to class Cat

If K=7, the new data point belongs to class Dog

Implementation of KNN Algorithm



https://www.kaggle.com/general/21221

Implementation of KNN Algorithm

Step-1: Load the relevant libraries

In [1]:
```
# Step-1: Load the relevent Libraries:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Step-2: Import the dataset and extract the independent and dependent variables

In [2]:
```
# Step-2: Import the Dataset & extract the independent and dependent variables:
social_network=pd.read_csv("C:/Users/itsme_000/Desktop/HKU/Teaching/IDAT 7215 Computer Programming/

social_network.head(5)
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

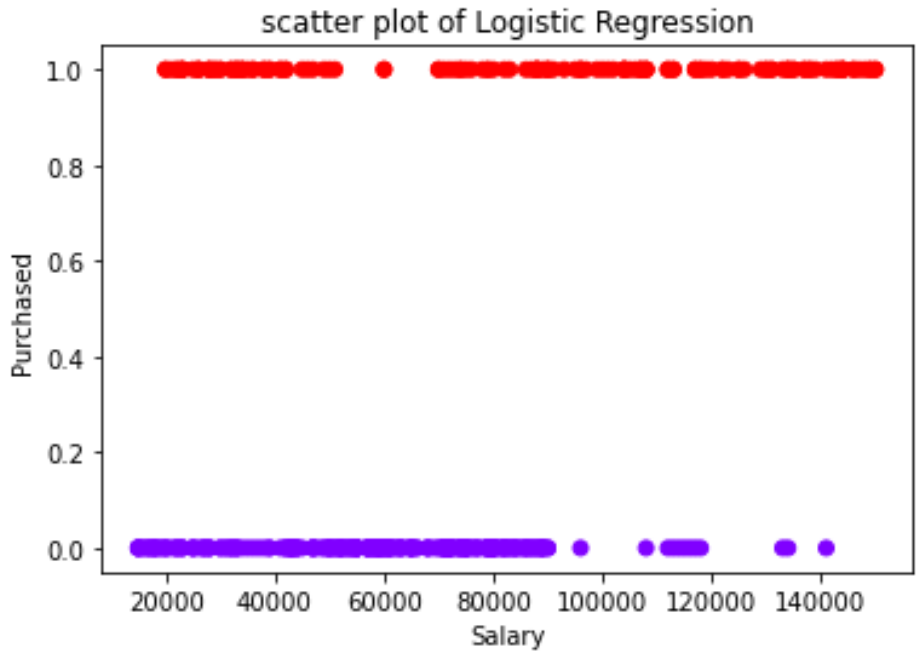
In [3]:
```
x = social_network.iloc[:,[2,3]].values  # Extracting the independent variables  (age, salary)
y = social_network.iloc[:,4].values      # Extracting the dependent variables    (purchased)
```

Implementation of KNN Algorithm

Step-3: Visualize the dataset

```
In [4]: # Step-3: Visualize the dataset:
        # create scatter plot
        plt.scatter(x[:,1],y,c=y, cmap='rainbow')
        plt.xlabel('Salary')
        plt.ylabel('Purchased')
        plt.title('scatter plot of Logistic Regression')
        plt.show
```

```
Out[4]: <function matplotlib.pyplot.show(close=None, block=None)>
```



Implementation of KNN Algorithm

Step-4: Split the dataset into Training and Testing Set

In [6]:
```
# Step-4: Split the dataset into Training and Testing set:
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=1/3,random_state=0)
```

Step-5: Feature Scaling

In [7]:
```
# Step-5: Feature Scaling:
from sklearn.preprocessing import StandardScaler
sc_x=StandardScaler()
x_train=sc_x.fit_transform(x_train)
x_test=sc_x.transform(x_test)
```

Implementation of KNN Algorithm

Step-6: Fit KNN model to Train dataset:

In [9]:
```
# Step-6: Fit KNN to Train set:
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p=2)
classifier.fit(x_train,y_train)
```

Out[9]:
```
KNeighborsClassifier()
```

Step-7: Predicting the Test set results

In [13]:
```
# Step 7: Predicting the Test set results:
y_pred = classifier.predict(x_test)
y_pred
```

Out[13]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1], dtype=int64)
```

Implementation of KNN Algorithm

Step-8: Display the confusion matrix:

```
# Step 8: Displacy the Confusion Matrix (To evaluate the model)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test, y_pred)
cm
```

```
array([[64,  4],
       [ 3, 29]], dtype=int64)
```

Accuracy:

$$\frac{(TP+TN)}{P+N} = \frac{(64+29)}{100} = 0.93$$

Misclassification Rate:

$$\frac{(FP+FN)}{P+N} = \frac{(3+4)}{100} = 0.07$$

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

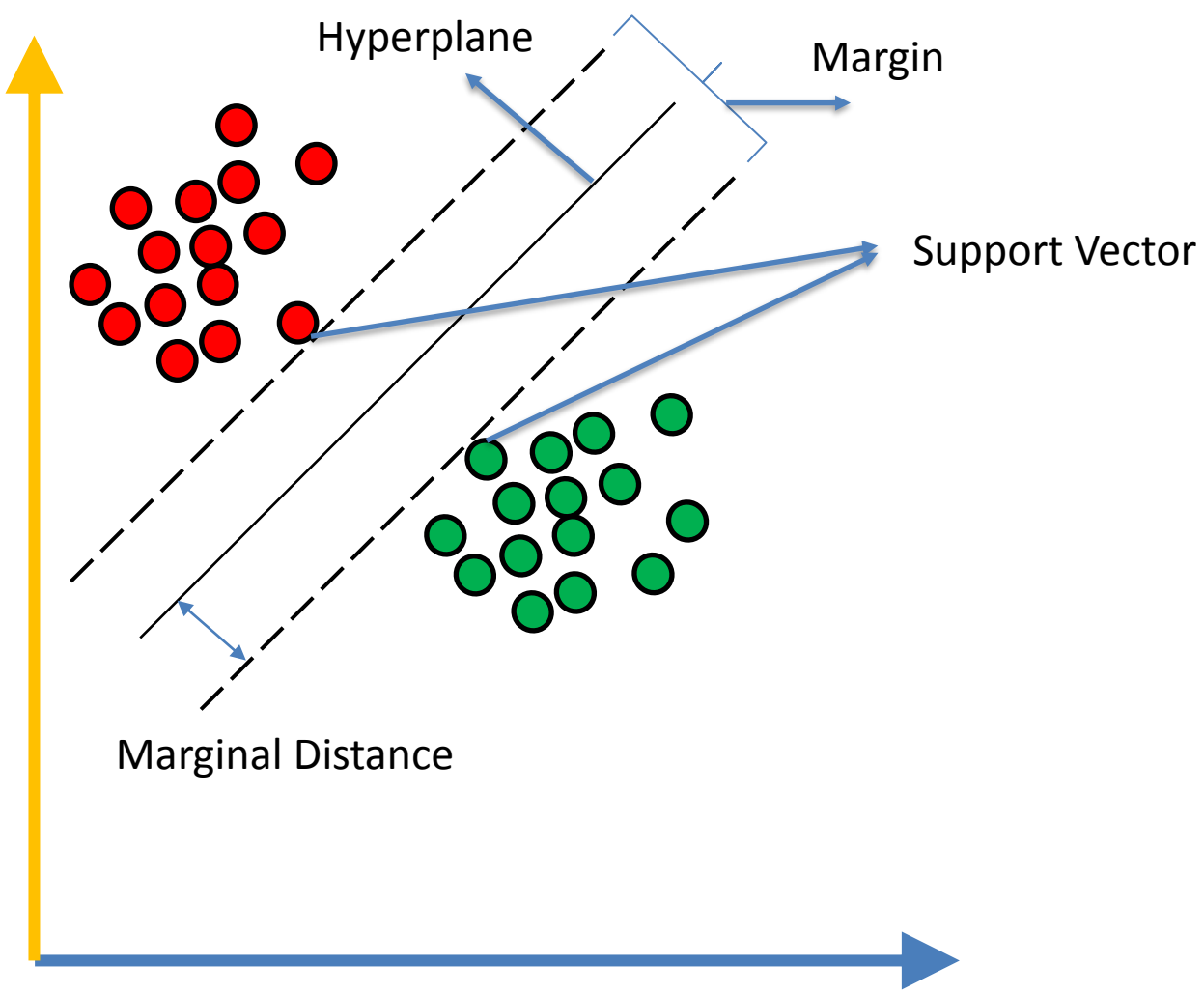
Support Vector Machine (SVM)

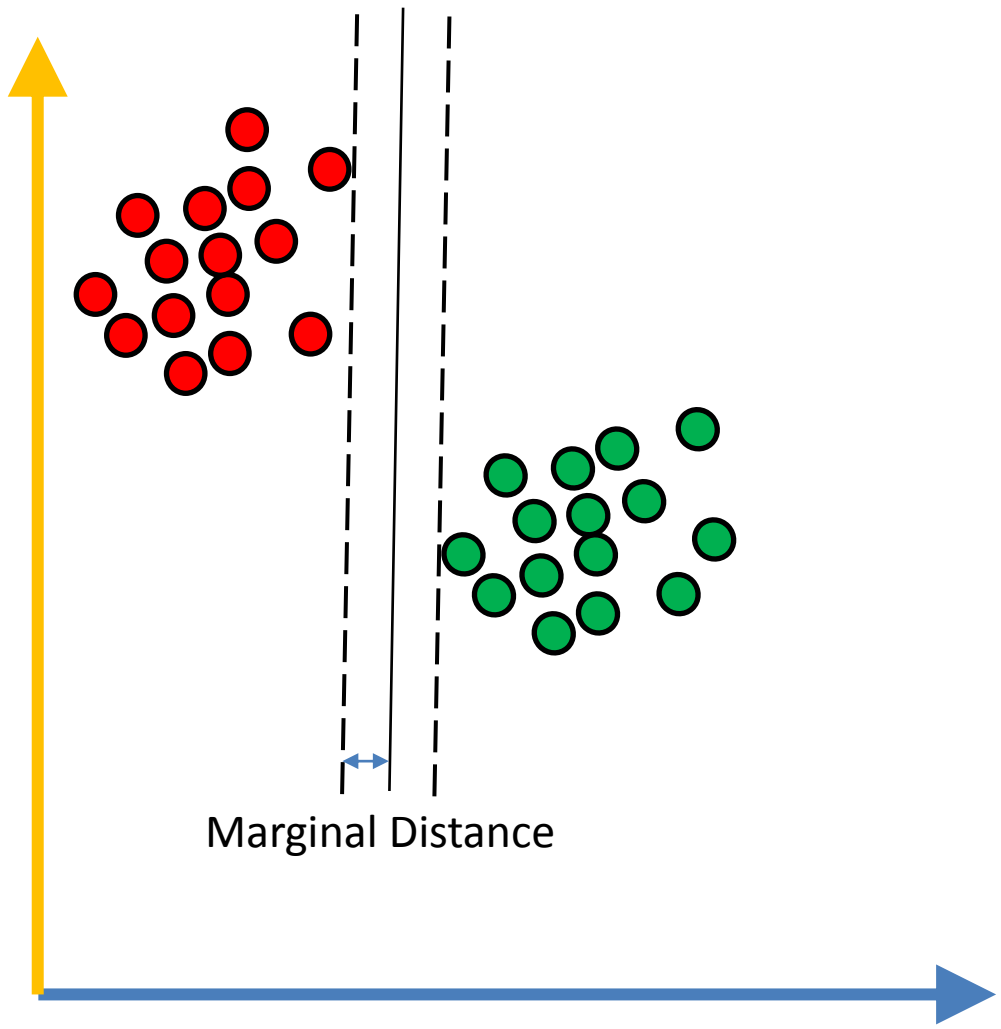
What is SVM

- Support Vector Machine (SVM) is a discriminative classifier that is formally designed by a separative hyperplane.
- Useful for solving both classification and regression-type problems.
- It is a representation of examples as points in space that are mapped so that the points of different categories are separated by a gap as wide as possible.
- The main objective of SVM is to segregate the given data in the best possible way.

How does SVM Work?

Linearly Separable data

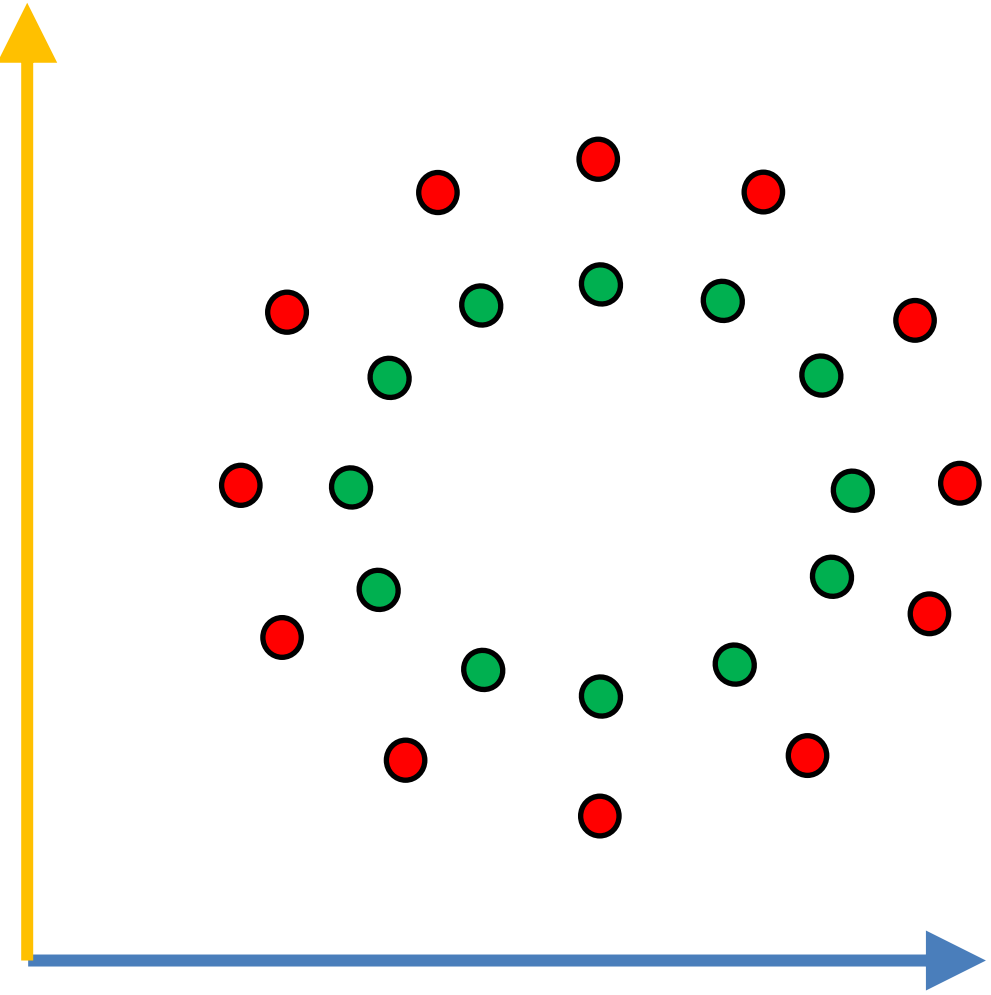


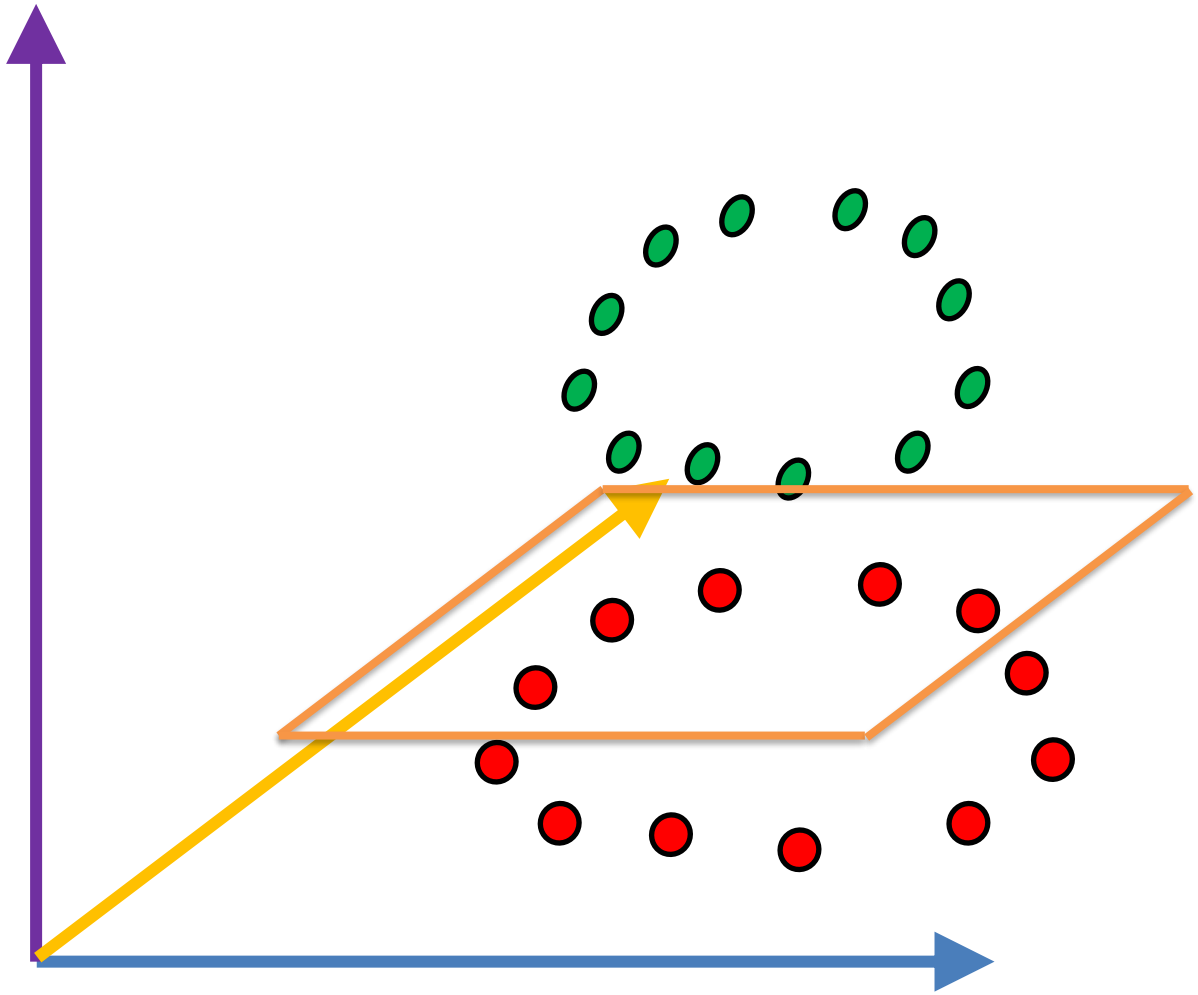


Choose a hyperplane that has the maximum marginal distance .

How does SVM Work?

Non-linearly Separable data





Convert the 2D dimensional problem into a 3D dimensional using SVM Kernels

SVM Kernels

Kernel does a transformation from lower dimension to higher dimension.

- Linear Kernels
- Polynomial Kernels
- Radial Basis Function (RBF) Kernel
- Sigmoid Kernel

SVM Use Cases

- Face detection
- Text and Hypertext Categorization
- Classification of Images
- Bioinformatics
- Remote Homology Detection
- Handwriting Detection
- Generalized Predictive Control

Implementation of SVM

IDAT7215 Lab 5 Tutorial- Supervised Machine Learning Algorithms

Support Vector Machine (SVM) on Breast Cancer Dataset Tutorial

In [1]:
```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn import metrics
```

In [2]:
```
cancer_data = datasets.load_breast_cancer()
```


```
{'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer wisconsin (diagnostic) dataset\n--------------------------------------------\n\n**
 'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
         1.189e-01],
        [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
         8.902e-02],
        ...,
        [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
         7.820e-02],
        [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
         1.240e-01],
        [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
         7.039e-02]]),
 'feature_names': array(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
        'mean smoothness', 'mean compactness', 'mean concavity',
        'mean concave points', 'mean symmetry', 'mean fractal dimension',
        'radius error', 'texture error', 'perimeter error', 'area error',
        'smoothness error', 'compactness error', 'concavity error',
        'concave points error', 'symmetry error',
        'fractal dimension error', 'worst radius', 'worst texture',
        'worst perimeter', 'worst area', 'worst smoothness',
        'worst compactness', 'worst concavity', 'worst concave points',
        'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
 'filename': '/usr/local/lib/python3.6/dist-packages/sklearn/datasets/data/breast_cancer.csv',
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0,
        1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
        1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
        1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
        0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
        1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
        0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
        1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
```

Implementation of SVM

IDAT7215 Lab 5 Tutorial- Supervised Machine Learning Algorithms

Support Vector Machine (SVM) on Breast Cancer Dataset Tutorial

In [1]:
```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn import metrics
```

In [2]:
```
cancer_data = datasets.load_breast_cancer()
```

In [3]:
```
x_train, x_test, y_train, y_test = train_test_split(cancer_data.data,cancer_data.target,test_size=0.4,random_state=209)
```

In [4]:
```
cls = svm.SVC(kernel='linear') # Choose the kernal
```

In [5]:
```
# Train the model
cls.fit(x_train, y_train)
```

Out[5]:
```
SVC(kernel='linear')
```

In [6]:
```
# Predict the response
pred = cls.predict(x_test)
```

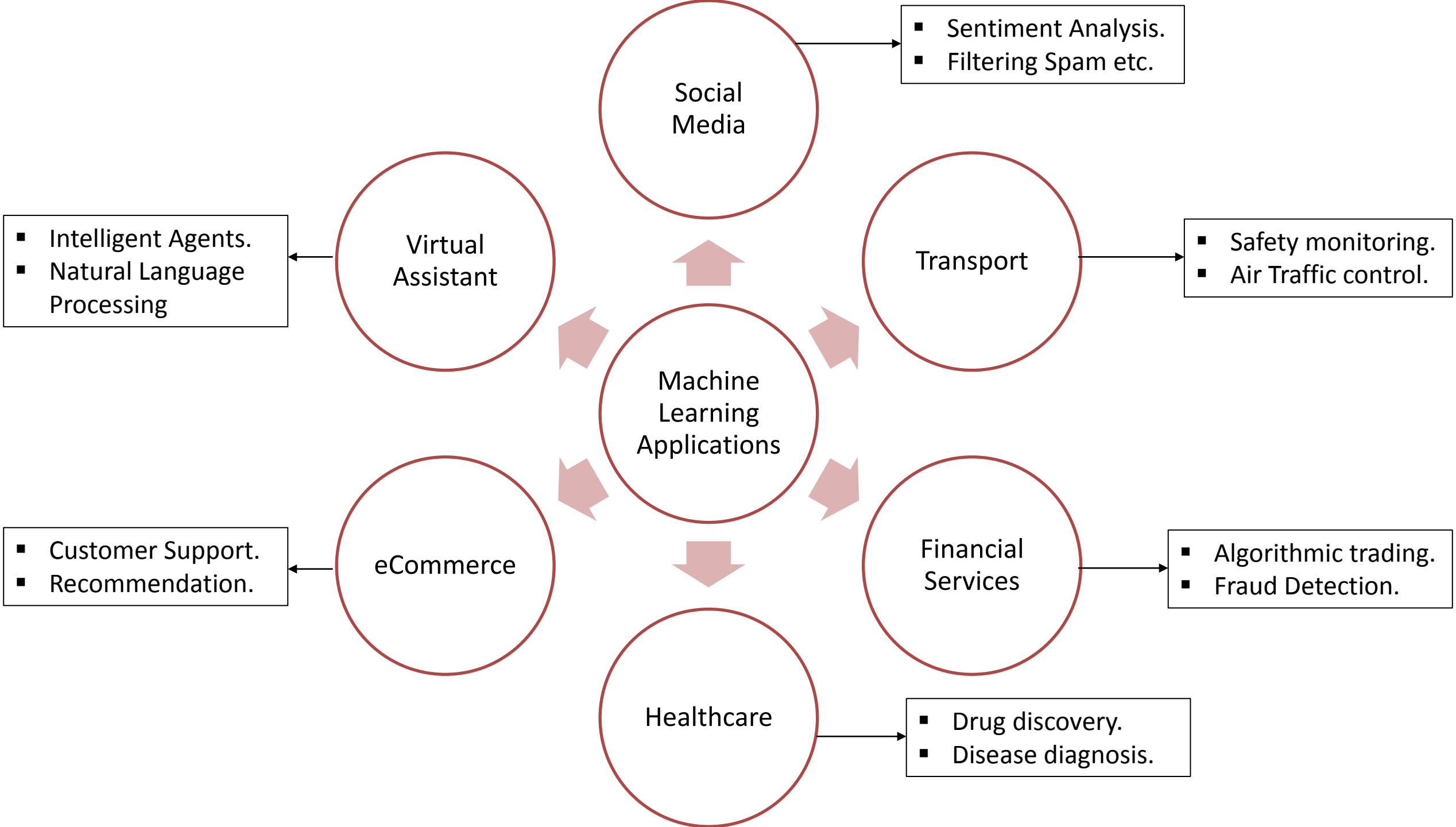
Implementation of SVM

In [11]:
```
# Performance Measure
print("accuracy:", metrics.accuracy_score(y_test,y_pred=pred))     # Accuracy
print("precision:", metrics.precision_score(y_test,y_pred=pred))   # Precision
print("recall:", metrics.recall_score(y_test,y_pred=pred))         # recall
print(metrics.classification_report(y_test,y_pred=pred))           # Total Performance report
```

```
accuracy: 0.9254385964912281
precision: 0.9333333333333333
recall: 0.9402985074626866
              precision    recall  f1-score   support

           0       0.91      0.90      0.91        94
           1       0.93      0.94      0.94       134

    accuracy                           0.93       228
   macro avg       0.92      0.92      0.92       228
weighted avg       0.93      0.93      0.93       228
```


Machine Learning Applications
Social Media
Sentiment Analysis.
Filtering Spam etc.
Transport
Safety monitoring.
Air Traffic control.
Financial Services
Algorithmic trading.
Fraud Detection.
Healthcare
Drug discovery.
Disease diagnosis.
eCommerce
Customer Support.
Recommendation.
Virtual Assistant
Intelligent Agents.
Natural Language Processing

References

1. Alpaydin, E. (2020). Introduction to machine learning. MIT press.

Resources: Datasets

- UCI Repository: http://www.ics.uci.edu/~mlearn/MLRepository.html
- UCI KDD Archive: http://kdd.ics.uci.edu/summary.data.application.html
- Statlib: http://lib.stat.cmu.edu/
- Delve: http://www.cs.utoronto.ca/~delve/